

Université Amadou Mahtar Mbow



Cours de Langage C

UFR STA

L1 MPI & SML / Semestre 2

Pr Amadou Dahirou GUEYE ADG

Enseignant-Chercheur en informatique à l'UAM

Année académique 2023-2024

Objectifs du cours

À l'issu de ce cours, l'étudiant sera capable de :

- Comprendre les concepts fondamentaux du langage C.
- Écrire, compiler et exécuter des programmes en C.
- Utiliser efficacement les structures de contrôle, les fonctions, et les pointeurs.
- Gérer la mémoire dynamiquement.
- Programmer de manière modulaire
- Créer et Manipuler des fichiers
- Appliquer le C pour résoudre des problèmes de programmation.
- Faire la programmation graphique

Plan du cours

1. Introduction au Langage C

- Historique et caractéristiques du langage C
- Installation de l'environnement de développement (TP)
- Structure d'un programme C, compilation, et exécution
- Identificateurs, Variables, types de données, et opérateurs

2. Structures de Contrôle

- Instructions conditionnelles (if, else)
- Boucles (for, while, do-while)
- Instructions de saut (break, continue, goto)

Plan du cours

3. Les Types composites

- Tableaux et chaînes
- Structures , Unions et Enumérations

4. Pointeurs

- Compréhension des pointeurs et de l'adressage mémoire
- Opérations sur les pointeurs
- Pointeurs vers des fonctions et des tableaux

5. Fonctions

- Définition et appel de fonctions
- Passage de paramètres par valeur et par référence
- Portée des variables, variables locales et globales

Plan du cours

6. Les listes chaînées

- Définition et utilisation des structures
- Gestion des unions
- Utilisation des énumérations

7. Entrées/Sorties en C

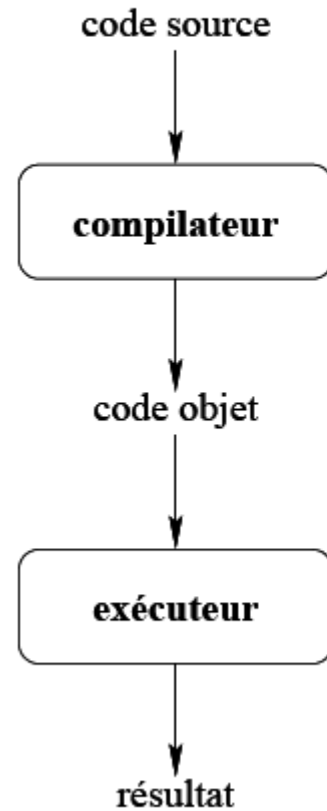
- Utilisation des fonctions de la bibliothèque stdio pour lire et écrire des fichiers
- Gestion des erreurs d'entrée/sortie

Chapitre 1:

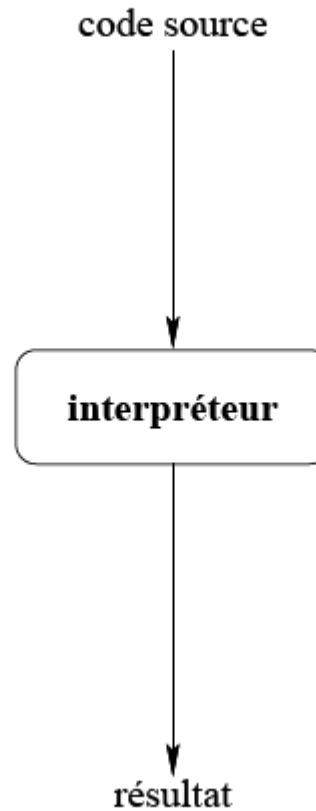
Introduction au langage C

Notion de langage de programmation

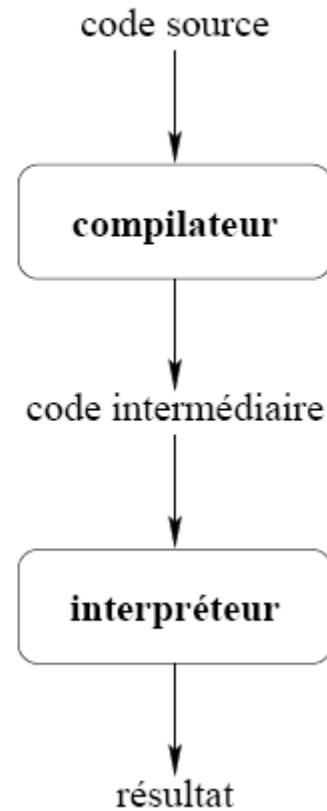
compilation
ADA, C/C++, Fortran



interprétation
Lisp, Prolog



semi-compilation
Java, Python



Introduction au langage C

- C'est un langage compilé qui a les caractéristiques suivantes :
 - C'est un langage typé: tout objet doit avoir un type (caractère, entier, réel...)
 - C'est un langage déclaratif: tout objet doit être déclaré avant son utilisation.
- Le langage C propose un certain nombre de fonctions prédéfinies dans des bibliothèques.

1.1 Historique

Le développement du C a commencé en **1972** sur un système d'exploitation UNIX, également développé chez Bell Labs. À cette époque, UNIX était écrit en langage d'assemblage, et l'un des objectifs du C était de recoder UNIX en un langage de haut niveau pour augmenter la **portabilité** du système d'exploitation.

Le langage C est l'un des langages les plus influents et les plus largement utilisés depuis sa création. Développé au début des années 1970 par Dennis Ritchie et Ken Thompson chez Bell Labs, le C a été conçu pour être à la fois **simple** et **puissant**, des qualités qui ont contribué à sa popularité et à sa longévité.

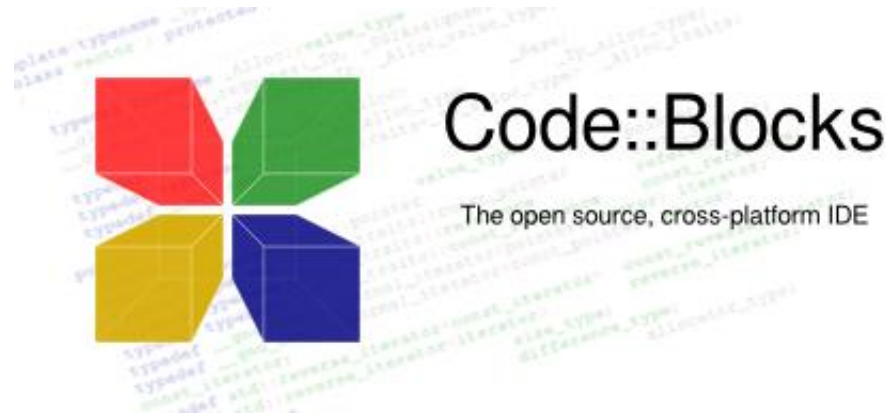
En **1983**, l'ANSI (American National Standards Institute) décida de normaliser le langage ; ce travail s'acheva en **1989** par la définition de la norme ANSI C.

1.2 Caractéristiques du langage C

- **Simplicité** : Le C est un langage minimaliste avec un petit ensemble de mots-clés. Cette simplicité facilite l'apprentissage du langage pour les nouveaux programmeurs.
- **Portabilité** : Le C est conçu pour être facilement portable, ce qui permet aux programmes écrits en C d'être compilés et exécutés sur différentes plateformes avec peu ou pas de modification.
- **Efficient et Rapide** : Les programmes en C, s'ils sont bien écrits, peuvent être très efficaces en termes de performances. Le C permet une manipulation directe de la mémoire et un accès bas niveau aux ressources système, ce qui est particulièrement utile pour le développement de systèmes embarqués et de systèmes d'exploitation.
- **Langage de programmation structurée** : Le C supporte la programmation structurée, ce qui facilite l'écriture de code clair et maintenable.
- **Accès bas niveau** : Le C offre des opérations de bas niveau qui ne sont généralement disponibles que dans les langages d'assemblage, comme la manipulation directe de bits et l'accès direct à la mémoire adressée.

1.3 Installation de l'environnement de développement

IDE: **CodeBlocks**



Aller sur le site officiel de **CodeBlocks**:

<https://www.codeblocks.org/downloads/binaries/>

Télécharger la version qui correspond à votre système d'exploitation

Autres éditeurs: vscode, etc.

1.4 Structure d'un programme C

```
#include <stdio.h>
#include <conio.h>
#define FIN 10
#define FIN 10
#define MGS "Premier programme C"

int main(void)
{ /*Begin*/

    printf(" Bonjour L1 UAM ");

    getch() ;    /* Attente d'une saisie clavier */

    return 0;    /* En principe un code d'erreur nul signifie "pas d'erreur". */

/*End */
```

Directives du **préprocesseur**
: accès avant la compilation

Programme principal

- Un programme écrit en C est une succession d'instructions
- Une instruction est une suite de mots prédéfinis, terminée par un point virgule.
- Ces instructions sont regroupées dans une fonction.
- Un bloc regroupe un ensemble d'instructions entre accolades
- Il existe une fonction particulière : En tête du programme, **main()**

1.4 Structure d'un programme C

- Le nom du programme principal est main; son type de retour est entier (int).
- Il ne prend en général pas d'arguments. Dans ce cas, on met le mot void entre les parenthèses (ou rien du tout)
- Dans certains cas il peut en prendre des arguments.

1.4 Structure d'un programme C

Préprocesseur

Le préprocesseur effectue un prétraitement du programme source avant qu'il soit compilé.
Ce préprocesseur exécute des instructions particulières appelées **directives**.

Ces directives sont identifiées par le caractère **#** en tête.

Inclusion de fichiers

#include <nom-de-fichier>	/* répertoire standard */
#include "nom-de-fichier"	/* répertoire courant */

La directive #include

- Cette directive permet l'inclusion de bibliothèques dont les éléments seront utilisés dans le programme source.
- Le compilateur C fournit un ensemble de bibliothèques mais le programmeur peut aussi créer ses propres bibliothèques.

Syntaxe de cette directive :

- `# include <fichier>` ou `# include "fichier"`

`<fichier>` : Indique l'utilisation d'une bibliothèque, fournie par le compilateur C, se trouvant dans un répertoire particulier du système.

- `#include "C:\MesProgrammes\definitions.h"`

1.4 Structure d'un programme C

Préprocesseur

La gestion des fichiers (**stdio.h**)

Les fonctions mathématiques (**math.h**)

Taille des types entiers (**limits.h**)

Limites des type réels (**float.h**)

Traitement de chaînes de caractères (**string.h**)

Le traitement de caractères (**ctype.h**)

Utilitaires généraux (**stdlib.h**)

Date et heure (**time.h**)

Tracé de figures (**graphics.h**)

.....

/* Entrees-sorties standard */

1.4 Structure d'un programme C

La directive #define

Cette directive permet de remplacer dans le programme toutes les occurrences d'une suite de caractères par un nom de substitution.

Cette possibilité permet une meilleure lisibilité du programme source, évite de devoir réécrire à chaque fois une suite longue de caractères souvent utilisés au cours du programme source et permet de définir des constantes pour le programme.

Syntaxe de cette directive :

#define nom_de_substitution suite_de_caractères

représenter la suite de caractères *suite_de_caractères*.

Exemple:

#define VRAI 1

Le mot VRAI sera utilisé pour représenter le chiffre 1 dans le programme source.

Mais rien ne vous empêche d'utiliser le chiffre 1.

#define MAX 100

1.4 Structure d'un programme C

La fonction **printf()** : permet d'afficher des informations à l'écran

Les formats d'affichage sont

- %d ou %ld pour les entiers (int, short, long)
- %f ou %lf pour les réels (float, double)
- %s pour les chaînes de caractères
- %c pour les caractères

`printf("La valeur des variables X et Y sont : %d et %d",X,Y) ;`

Les deux %d seront respectivement remplacés à l'écran par la valeur de X et de Y.

1.4 Structure d'un programme C

NB : sur les caractères non imprimables

Les caractères précédés de \ sont interprétés comme suit :

**** : caractère \ **\"** : caractère "

\n : retour à la ligne

\t : tabulateur.

\r : retour chariot sans saut de ligne

Exemple avec les :

```
printf("Qu'il est agreable d'utiliser printf en\t C,\nlorsqu'on l'utilise \"proprement\".\n");
```

Résultat sur la sortie :

```
Qu'il est agreable d'utiliser printf en   C,  
lorsqu'on l'utilise "proprement".
```

1.4 Structure d'un programme C

Les constantes de type caractère ont une valeur entière dans la table ASCII

```
char c1 = 'A', c2 = '\x41'; /* représentation hexadécimale */
```

caractères	nom	symbole	code hexa	décimal
\n	newline	LF	10	
\t	tabulation	HT	9	
\b	backspace	BS	8	
\r	return	CR		13
\f	form feed	FF	12	
\\	backslash	5C	92	
\'	single quote	27	39	
\"	double quote	22	34	

1.4 Structure d'un programme C

La fonction **scanf()** :

Librairie : **stdio.h**.

#include <stdio.h>

L'instruction scanf permet au programme de lire des informations saisies au clavier par l'utilisateur.

Syntaxe : scanf (" chaîne de formatage " , &variable1 , &variable2 , . . .)

Exemple : scanf(" %d %d",&X, &Y) ;

Le & permet d'accéder à l'adresse des variables.(A omettre dans le cas de la lecture d'une chaîne de caractères).

Format des paramètres passés en lecture et écriture.

"%c" : lecture d'un caractère.

"%d" ou "%i" : entier signé.

"%e" : réel avec un exposant.

"%f" : réel sans exposant.

"%g" : réel avec ou sans exposant suivant les besoins.

"%G" : identique à g sauf un E à la place de e.

"%o" : le nombre est écrit en base 8.

"%s" : chaîne de caractère.

"%u" : entier non signé.

"%x" ou "%X" : entier base 16 avec respect majuscule/minuscule.

Lecture et affichage de caractères

- Les fonctions **getchar** et **putchar** appartiennent à la bibliothèque **stdio.h**
 - **getchar** : permet de lire au clavier
 - **Putchar**: permet d'imprimer à l'écran
- Ce sont des fonctions Entrées/Sorties non formatées
- La fonction **getchar** permet au programme de lire un caractère saisi au clavier par l'utilisateur. Cette fonction retourne un entier (code ASCII) correspondant au caractère lu. Pour mettre le caractère lu dans une variable c de type caractère, on écrit
 - `c = getchar( ) ;`

Lecture et affichage de chaines de caractères

- Nous avons les fonctions `gets` et `puts` pour lire et afficher respectivement une chaîne de caractères

- Exemple 1

```
#include<stdio.h>
```

```
int main() {
```

```
    char str[]="Hello, world!"; / * declaration et initialization d'une chaine de caractères
```

```
    nommée str * /
```

```
    puts(str); // Output the string to the console with a newline
```

```
    return 0;
```

- Exemple 2

```
#include<stdio.h>
```

```
int main() {
```

```
    char str[50];
```

```
    printf("Enter a string: ");
```

```
    gets(str); // Read input from user
```

```
    printf("You entered: ");
```

```
    puts(str); // Display the string
```

```
    return 0;
```

```
}
```

Remarques

- La lecture avec la fonction `getchar` se fait par l'intermédiaire d'une zone mémoire appelée "buffer d'entrée" qui sert à contenir les caractères tapés au clavier par l'utilisateur et qui fonctionne selon le principe d'une "file" (premier arrivé = premier sorti).
- Si le buffer d'entrée n'est pas vide, l'instruction **`getchar();`** lit le caractère le plus ancien se trouvant dans le buffer d'entrée sans attendre l'utilisateur, et enlève ce caractère du buffer d'entrée.
- Si le buffer d'entrée est vide, l'instruction **`getchar();`** stoppe le déroulement du programme jusqu'à ce que le buffer d'entrée reçoive un ou plusieurs caractères. L'ajout de caractères dans le buffer d'entrée ne se fait qu'au moment où l'utilisateur tape un retour-chariot (touche **Entrée**).
- Pour vider le buffer d'entrée, il faut utiliser l'instruction **`fflush(stdin);`**



Remarques

- Certains compilateurs proposent une bibliothèque appelée **conio.h** qui contient deux fonctions de lecture et d'écriture de caractère. Ce sont respectivement les fonctions **getch** et **putch**.

```
c = getch( ) ;  
putch(c) ;
```

- La fonction getch lit directement le caractère tapé par l'utilisateur sans utiliser de buffer d'entrée.
- Ces deux fonctions ne font pas partie du langage C standard.

1.5 Compilation & Exécution

Le C est un langage **compilé** (par opposition aux langages interprétés). Cela signifie qu'un programme C est décrit par un fichier texte, appelé **fichier source**. Ce fichier n'étant évidemment pas directement exécutable par le microprocesseur, il faut le traduire en langage machine. Cette opération est effectuée par un programme appelé **compilateur**. Cette étape se décompose en fait en 2 phases successives :

- **Compilation** : l'opération de compilation transforme le fichier source en un fichier binaire, c'est-à-dire en instructions directement compréhensibles par le processeur. Le fichier produit par l'assemblage est appelé **fichier objet**.
- **L'édition de liens** : un programme est souvent séparé en plusieurs fichiers sources, pour des raisons de clarté mais aussi parce qu'il fait généralement appel à des bibliothèques de fonctions standard déjà écrites. Une fois chaque code source assemblé, il faut donc lier entre eux les différents fichiers objets. L'édition de liens produit alors un **fichier exécutable**.

1.6 Les identificateurs

Le rôle d'un identificateur est de donner un nom à une entité du programme.

Plus précisément, un identificateur peut désigner :

- un nom de variable ou de fonction,
- un type défini par typedef, struct, union ou enum,
- une étiquette.

Un identificateur est une suite de caractères parmi :

- les lettres (minuscules ou majuscules, mais non accentuées),
- les chiffres,
- le « blanc souligné » ().

Les majuscules et minuscules sont différenciées.

1.7 Les variables

Définition : Une variable est une zone mémoire à laquelle on affecte une valeur.

En C, toute variable doit être déclarée

Remarque : l'emplacement en zone mémoire dépend de l'endroit où est déclarée la variable dans le programme

Déclaration : Une variable se déclare en précisant le type de la variable suivi du nom donné à la variable.

Syntaxe : **Type** identificateur1, identificateur2, ...,.... ;

Exemple : **char** c1, c2, c3;
 int i, j, var_ent;

1.8 Les commentaires

`/* ..... */` ligne de commentaires

`/* ...ceci est un commentaire... ..`

Plusieurs lignes de commentaires */

`// .....` ligne de commentaires (Uniquement en C++)

1.9 Les mots réservés

Un identificateur ne peut pas être un **mot réservé** du langage :

auto	double	int	struct
break	else	long	switch
case	enum	register	typedef
char	extern	return	union
const	float	short	unsigned
continue	for	signed	void
default	goto	sizeof	volatile
do	if	static	while

RQ : Les **mot réservés du langage C doivent être écrits en minuscules.**

1.8 Les types de base

Type	Signification	Exemples de valeur	Codage en mémoire	Peut être
char	Caractère unique	'a' 'A' 'z' 'Z' '\n' 'a' 'A' 'z' 'Z' '\n' Varie de -128 à 127	1 octet	signed, unsigned
int	Nombre entier	0 1 -1 4589 32000 -231 à 231 +1	2 ou 4 octets	Short, long, signed, unsigned
float	Nombre réel simple	0.0 1.0 3.14 5.32 -1.23	4 octets	
double	Nombre réel double précision	0.0 1.0E-10 1.0 - 1.34567896	8 octets	long

1.8 Les opérateurs: Affectation

- Une affectation consiste à donner une valeur à une variable
- L'opérateur d'affectation est le symbole '='

Remarques :

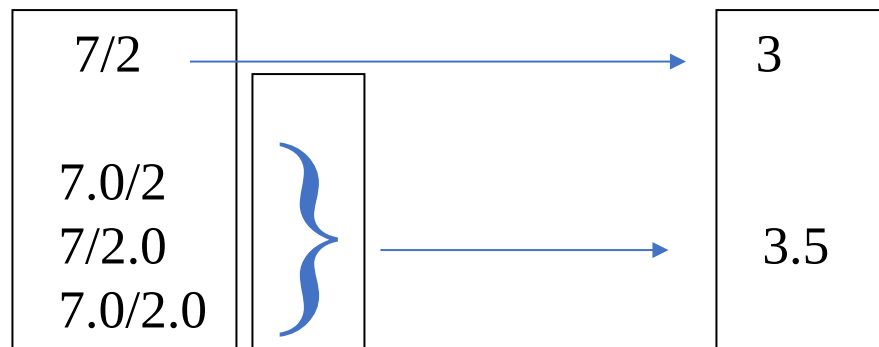
- On peut enchaîner plusieurs affectations : `i = j = 2 ;`
- NB: Opérateurs d'affectation composée: `+=`, `-=`, `x=`, `/=`, `%=`
Exemple: `Y+=5` équivaut à `Y=Y+5`

1.8 Les opérateurs: Opérateurs arithmétiques

+	addition
-	soustraction
*	multiplication
/	division
%	reste de la division (modulo)

Exemple :

% ne peut être utilisé qu'avec des entiers



```
main()
{
    int i=10, j=4;
    float x;
    x = i / j;      //x=2.0
    x=float(i) / j; //x=2.5
}
```


1.8 Les opérateurs: Opérateurs relationnels

>	strictement supérieur
>=	supérieur ou égal
<	strictement inférieur
<=	inférieur ou égal
==	égal
!=	différent

Exemple :

```
int i;  
float f;  
char c;  
  
i = 7;    f = 5.5;    c = 'w';  
  
f > 5                =====> vrai (1)  
(i + f) <= 1         =====> faux (0)  
c == 'w'             =====> vrai (1)  
c != 'w'             =====> faux (0)  
c >= 10*(i + f)       =====> faux (0)  
(i >= 6) && (c == 'w') =====> vrai (1)  
(i >= 6) || (c == 119) =====> vrai (1)
```

Leur syntaxe est : **Expression_1** **op** **expression_2**

Les deux expressions sont évaluées puis comparées. La valeur rendue est de type **int** (**il n'y a pas de type booléen en C**); elle vaut **1** si la condition est vraie, et **0** sinon.

Attention à ne pas confondre l'opérateur de test d'égalité **==** avec l'opérateur d'affectation **=**.

1.8 Les opérateurs: Opérateurs logiques

Opérateurs	Fonction
!	non
	ou
&&	et

Comme pour les opérateurs de comparaison, la valeur retournée par ces opérateurs est un int qui vaut 1 si la condition est vraie et 0 sinon.

Dans une expression de type : **Expression_1** *op_1* **expression_2** *op_2* ...**expression_n**

L'évaluation se fait de gauche à droite et s'arrête dès que le résultat final est déterminé.

Par exemple dans :

```
int i;
```

```
int p[10];
```

```
...
```

```
if ((i >= 0) && (i <= 9) && !(p[i] == 0))
```

```
...
```

La dernière clause ne sera pas évaluée si i n'est pas entre 0 et 9.

1.8 Les opérateurs: Incrémentation & Décrémentation

Incrémentation : $i = i + 1$

- $i++$: post incrémentation
- $++i$: pré incrémentation

Décrémentation : $i = i - 1$

- $i--$: post décrémentation
- $--i$: pré décrémentation

$j = i++$ ($j = i$; puis $i = i + 1$;) affectation puis incrémentation

$j = ++i$ ($i = i + 1$; puis $j = i$;) incrémentation puis affectation

1.9 Les opérateurs: Opérateurs logiques de traitement de bits

Le langage C définit six opérateurs permettant de manipuler les bits :

l'opérateur « et » : $\&$

l'opérateur « ou inclusif » : $|$

l'opérateur « ou exclusif » : $\wedge$

l'opérateur de négation ou de complément : $\sim$

l'opérateur de décalage à droite : $>>$

l'opérateur de décalage à gauche : $<<$

1.9 Les opérateurs: Opérateurs logiques de traitement de bits

Les opérateurs « et », « ou inclusif » et « ou exclusif »

Chacun de ces trois opérateurs attend deux opérandes entiers et produit un nouvel entier en appliquant une table de vérité à chaque paire de bits formée à partir des bits des deux nombres de départs. Plus précisément :

- L'opérateur « et » (&) donnera 1 si les deux bits de la paire sont à 1 ;
- L'opérateur « ou inclusif » (|) donnera 1 si au moins un des deux bits de la paire est à 1 ;
- L'opérateur « ou exclusif » (^) donnera 1 si un seul des deux bits de la paire est à 1.

1.9 Les opérateurs: Opérateurs logiques de traitement de bits

Bit 1	Bit 2	Opérateur « et »	Opérateur « ou inclusif »	Opérateur « ou exclusif »
0	0	0	0	0
1	0	0	1	1
0	1	0	1	1
1	1	1	1	0

1.9 Les opérateurs: Opérateurs logiques de traitement de bits

L'opérateur de négation ou de complément

L'opérateur de négation a un fonctionnement assez simple : il inverse les *bits* de son opérande (les uns deviennent des zéros et les zéros des uns)

1.9 Les opérateurs: Opérateurs logiques de traitement de bits

Les opérateurs de décalage

Les opérateurs de décalage, comme leur nom l'indique, décalent la valeur des *bits* d'un objet d'une certaine quantité, soit vers la gauche (c'est-à-dire vers le *bit* de poids fort), soit vers la droite (autrement dit, vers le *bit* de poids faible). Ils attendent deux opérandes : le nombre dont les *bits* doivent être décalés et la grandeur du décalage.

1.9 Les opérateurs: Opérateurs logiques de traitement de bits

L'opérateur de décalage à gauche

L'opérateur de décalage à gauche translate la valeur des *bits* vers le *bit* de poids fort. Les *bits* de poids faibles perdant leur valeur durant l'opération sont mis à zéro.

L'opérateur de décalage à droite

L'opérateur de décalage à droite translate la valeur des *bits* vers le *bit* de poids faible. Dans le cas où le premier opérande est un entier *non signé* ou un entier signé *positif*, les *bits* de poids forts perdant leur valeur durant l'opération sont mis à zéro. Si en revanche il s'agit d'un nombre signé *négatif*, les *bits* perdant leur valeur se voient mis à zéro ou un suivant la machine cible.

1.9 Les opérateurs: Opérateur de conversion de type

L'opérateur de conversion de type, appelé **cast**, permet de modifier explicitement le type d'un objet.

On écrit : **(type) objet**

Exemple :

```
main()
{
    int i = 3, j = 2;
    printf("%f \n", (float)i/j);
}
```

retourne la valeur 1.5.

1.9 Les opérateurs: Opérateur adresse « & »

L'opérateur d'adresse **&** appliqué à une variable retourne l'adresse mémoire de cette variable.

La syntaxe est **&var**

Chapitre 2: Structures de contrôle

Introduction

- Les structures de contrôle permettent d'agir sur l'ordre ou la fréquence d'exécution des instructions d'un programme.
- Il existe deux grands types de structures de contrôle:
 - les *structures conditionnelles*
 - les *structures répétitives*, encore appelées boucles.

Introduction

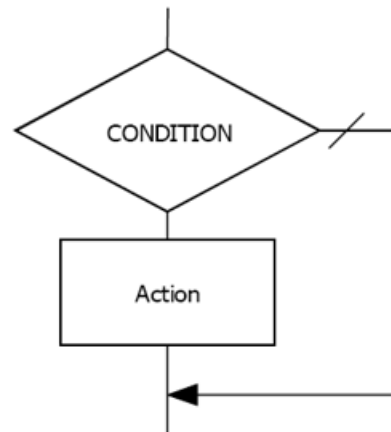
Alternative:	<code>if-else</code>
Choix Multiple:	<code>switch-case</code>
Itérations:	<code>for, while, do-while</code>
Rupture de Contrôle:	<code>break, continue, goto</code>

2.1 Structures Alternative: Rappel Si

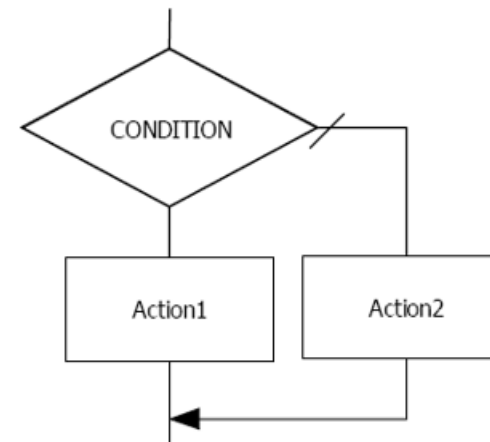
- La structure **Si**

- permet de conditionner l'exécution d'un ensemble d'instructions à la valeur d'une condition (expression booléenne).
- La syntaxe générale de cette structure est la suivante:

Si (condition) **Alors**
 Action
FinSi



Si (condition) **Alors**
 Action1
Sinon
 Action2
FinSi



2.1 Structures Alternative: Rappel Si

- La structure **Si**: Utilisation de plusieurs **Si**

Si (condition1) **Alors**

Action1

Sinon Si (condition2) **Alors**

Action2

...

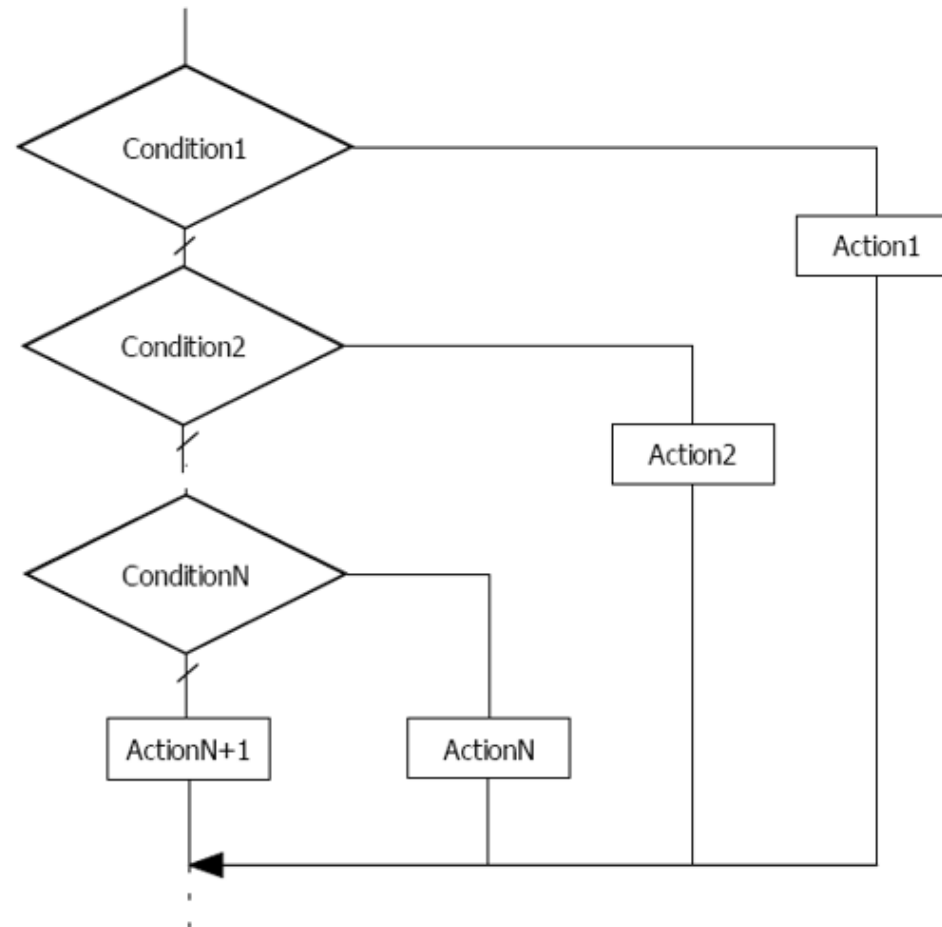
Sinon Si (conditionN) **Alors**

ActionN

Sinon

Action

FinSi



2.1 Structures Alternative: Si Traduction en C

Le mot-clé **if** permet d'effectuer une instruction (simple ou composée) de manière conditionnelle.

Pas de **then** en C

Le bloc "**else**"
est optionnel.

```
if (condition) {
```

```
/* si la condition est vraie */
```

```
instruction 1
```

```
...
```

```
instruction n
```

```
}
```

2.1 Structures Alternative: « **if-else** »

Il peut être suivi du mot-clé **else** pour spécifier l'instruction à effectuer lorsque la condition est fausse.

Le **else** est optionnel

```
if (condition)
{
    /* si la condition est vraie */
    instruction 1
    ...
    instruction n
} else {
    /* si la condition est fausse */
    instruction 1
    ...
    instruction n
}
```

2.1 Structures Alternative: Utilisation de plusieurs if

```
if(condition1)
{
    traitement1
}
else if (condition2)
{
    traitement2
}
...
else if (conditionN)
{
    traitementN
}
else
{
    traitement
}
```

NB : conditions doivent être entre parenthèses.

Si un traitement comporte une seule instruction, on peut omettre les accolades.

2.1 Structures Alternative: « if-else »

Exemple 1 :

```
...  
printf("entrez un nombre");  
  
scanf("%d", &n);  
  
if (n > 0)  
    printf("valeur positive");  
  
else  
    printf("valeur négative ou nulle");  
...
```

2.1 Structures Alternative: « if-else »

Exemple 2

```
#include <stdio.h>
int main( )
{
    int nb1,nb2, res;
    char op;
    printf("Entrez deux nombres");
    scanf("%d%d", &nb1, &nb2);
    printf("entrez la première lettre de l'opération voulue: somme ou produit");
    fflush (stdin);
    op = getchar( );
    if (op == 's')
    {
        res = nb1 + nb2;
        printf("la somme est %d", res);
    }
    else
    {
        res = nb1 * nb2;
        printf("le produit est %d", res);
    }
    return 0;
}
```

2.1 Structures Alternative: « if-else »

Exemple 3 :

```
if (i < 10) i++;
```

La variable **i** ne sera incrémentée que si elle a une valeur inférieure à **10**.

```
if (i == 10) i++;
```

== et pas =

La variable **i** ne sera incrémentée que si elle est égale à **10**.

```
if (!recu) printf ("rien reçu\n");
```

Le message "rien reçu" est affiché si **recu** vaut zéro.

```
if ((!recu) && (i < 10)) i++
```

i ne sera incrémentée que si **recu** vaut zéro et **i < 10**.

Si plusieurs instructions, il faut les mettre entre accolades.

```
if ((!recu) && (i < 10) && (n!=0) ) {  
    i++;  
    moy = som/n;  
    printf(" la valeur de i =%d et moy=%f\n", i,moy) ;  
} else {  
    printf ("erreur \n");  
    i = i +2;           // i +=2 ;  
}
```

2.1 Structures Alternative: « if-else »

Il est possible d'enchaîner plusieurs tests successifs grâce à **else if** :

```
if (condition1)
    /* si condition1 est vraie */
    instruction
else if (condition2)
    /* si condition1 est fausse et condition2 est vraie */
    instruction
else
    /* si les deux conditions sont fausses */
    instruction
```

2.1 Structures Alternative: « if-else »

```
#include <stdio.h>

int main() {
    int age;
    printf("Veuillez saisir votre age : "); // Demande à l'utilisateur de saisir son âge
    scanf("%d", &age);

    // Vérifie si l'utilisateur est éligible pour voter et conduire en utilisant des opérateurs logiques
    if (age >= 18 && age >= 16) {
        printf("Vous êtes éligible pour voter et pour conduire.\n");
    } else if (age >= 18) {
        printf("Vous etes éligible pour voter mais pas pour conduire.\n");
    } else if (age >= 16) {
        printf("Vous êtes éligible pour conduire mais pas pour voter.\n");
    } else {
        printf("Vous n'êtes ni éligible pour voter ni pour conduire.\n");
    }

    return 0;
}
```


2.2 Choix Multiple: « switch-case »

- Le mot-clé **switch** permet d'effectuer un choix selon les valeurs possibles d'une expression.
- On sélectionne une valeur particulière en la faisant précéder du mot-clé **case**.
- Le mot-clé **default** permet de spécifier le comportement par défaut (c'est à dire lorsqu'aucune des valeurs spécifiées n'est pas la bonne).

Attention:

N'oubliez pas le **break** entre chaque cas sinon l'exécution continue sur le cas suivant.
Ceci permet de spécifier plusieurs valeurs pour une même série d'instructions.

```
switch (expression)
{
    case Constante1 :
        /* si expression est egale a Constante1 */
        instruction 1
        ...
        instruction n
        break;
    case Constante2 :
        /* si expression est egale a Constante2 */
        instruction 1
        ...
        instruction n
        break;

    ...

    default :
        /* cas par default */
        instruction 1
        ...
        instruction n
        break;
}
```

2.2 Choix Multiple: « switch-case »

Exemple:

Exemple

```
...  
printf("Donner le numéro du mois");  
scanf("%d", &mois);  
switch (mois)  
{  
  case 1 : printf("Janvier"); break;  
  case 2 : printf("Février"); break;  
  ...  
  case 12 : printf("Décembre"); break;  
  default : printf("Un numéro de mois doit être  
compris entre 1 et 12");  
}
```

2.3.1 Les Itérations: « for »

Le mot-clé **for** permet d'effectuer des boucles.

- **expr1** est l'expression d'initialisation. La seconde
- **expr2** est le test de continuation. La troisième
- **expr3** est appelée expression d'incrément.

```
for (expr1; expr2; expr3)
{
    instruction 1
    ...
    instruction N
}
```

Exemple: cette boucle for affiche:

La valeur de i est : 0

La valeur de i est : 2

.....

La valeur de i est : 9

```
for (i = 0; i < 10; i++)
{
    printf("La valeur i est : %d\n", i);
}
```

2.3.2 Les Itérations: « while »

La boucle **while** évalue une condition puis exécute le bloc d'instruction qui suit si la condition est vraie. Cette boucle continue tant que la condition reste vraie.

initialisation

while(condition)

{

//Si la condition est vraie, le bloc d'instructions sera exécutée

instruction_1

...

instruction_N

}

2.3.2 Les Itérations: « while »

Exemple:

```
Int i = 0;
while (i < 10)
{
    printf(« La valeur i est : %d\n", i);
    i++;
}
```

cette boucle **while** affiche:

La valeur de i est : 0
La valeur de i est : 2
.....
La valeur de i est : 9

2.3.3 Les Itérations: « do-while »

- La boucle **do-while** exécute tout d'abord l'instruction .
- Puis la condition est évaluée et, tant qu'elle reste vraie, l'instruction est exécutée à nouveau.

```
do {  
    instruction 1  
    ...  
    instruction n  
} while (condition);
```

Exemple:

```
Int i = 0;  
do {  
    printf(« La valeur i est : %d\n", i);  
    i++;  
} while (i < 10) ;
```

cette boucle **do-while** affiche:

```
La valeur de i est : 0  
La valeur de i est : 2  
.....  
La valeur de i est : 9
```

2.4.1 Rupture de Contrôle: « break »

- Elle est utilisée à l'intérieure d'une **boucle** ou d'une instruction **switch**.
- Elle provoque l'arrêt du déroulement de la boucle ou de l'instruction **switch** en cours et donne le contrôle à l'instruction située après la fin de cette boucle ou de cette instruction switch.
- Cette instruction n'a d'intérêt que si son exécution est conditionnée par un choix.
- Quand l'instruction **break** se trouve dans une **boucle imbriquée**, l'exécution se poursuit avec la suite de la boucle extérieure.

Exemple: Dans une boucle for

```
#include

main () {
    int i;
    for (i = 1 ; i<=10 ; i++ ) {
        printf("Début tour %d \n", i) ;
        printf("UAM \n") ;

        if (i == 3)
            break ;

        printf("Fin tour %d \n", i) ;
    }
    printf("Après la boucle\n") ;
}
```

2.4.2 Rupture de Contrôle: « continue »

- Elle doit être utilisée dans une boucle.
- Elle provoque l'exécution immédiate de la prochaine itération de la boucle.
- Les instructions situées entre continue et la fin de la boucle sont ignorées.
- Lorsque l'instruction continue est utilisée dans une boucle for, elle effectue alors un branchement sur l'évaluation de la condition.
- En cas de boucles imbriquées, l'instruction continue ne concerne que la boucle la plus interne

Exemple: Dans une boucle for

```
#include

main () {
    int i ;
    for (i = 1 ; i <= 10 ; i++ ) {
        printf("Début tour d \n", i) ;
        printf("« UAM \n") ;

        if (i == 5)
            continue ;

        printf("Fin tour %d \n", i) ;
    }
    printf("Après la boucle\n") ;
}
```


2.4.3 Rupture de Contrôle: « goto »

- C'est l'instruction la plus inconditionnelle. Si elle est rencontrée dans un programme, elle provoque le branchement immédiat dans l'emplacement indiqué.
- Elle est une instruction label (étiquette) qui identifie un emplacement du programme pour la suite de l'exécution.
- Une instruction label est constituée d'un identificateur (étiquette) suivi de deux points, puis d'une instruction **C**.
- L'instruction **goto** et son **étiquette** peuvent se trouver dans des blocs différents. L'étiquette associée à une instruction **goto** peut se situer avant ou après cette instruction dans le code.

2.4.3 Exemple simple de l'utilisation de : « goto »

```
#include <stdio.h>
int main() {
    int nombre = 0;

    debut: // étiquette
    printf("Entrez un nombre entre 1 et 3 : ");
    scanf("%d", &nombre);

    if (nombre < 1 || nombre > 3) {
        printf("Nombre invalide. Réessayez.\n");
        goto debut; // sauter à l'étiquette 'debut'
    }

    printf("Vous avez entré le nombre %d.\n", nombre);
    return 0;
}
```

2.4.3

Exemple simple de l'utilisation de : « goto »

- Dans cet exemple, si l'utilisateur entre un nombre qui n'est pas compris entre 1 et 3, le programme utilise `goto` pour revenir à l'étiquette `debut` et demander à nouveau un nombre.

Chapitre 3: Les types composés

- **Tableaux & Chaines de caractères**
- **Structures & Typedef**
- **Union**
- **Enumérations**

3. Les types composés

A partir des types prédéfinis du C (caractères, entiers, flottants), on peut créer de nouveaux types, appelés types composés, qui permettent de représenter des ensembles de données organisées.

3.1. Les tableaux

3.1.1. Tableau à 1 dimension

Un **tableau** est un ensemble fini d'éléments de **même type**, stockés en mémoire à des adresses contiguës.

La déclaration d'un tableau à une dimension se fait de la façon suivante :

type nom-du-tableau[nombre_éléments];

Où **nombre_éléments** est une expression constante entière positive.

Exemple:

int tab[10];

La variable **tab** est un tableau de **10 éléments** de type **int**. Cette déclaration alloue donc en mémoire pour l'objet **tab** un espace de **10 × 4** octets consécutifs.

3.1.1. Tableau à 1 dimension

Pour plus de clarté, il est recommandé de donner un nom à la constante **nombre-éléments** par une directive au préprocesseur, par exemple :

#define nombre_éléments 10

Si le nombre de données dans la liste d'initialisation est inférieur à la dimension du tableau, seuls les premiers éléments seront initialisés. Les autres éléments seront mis à zéro si le tableau est une variable globale (extérieure à toute fonction) ou une variable locale de classe de mémorisation static.

Les éléments d'un tableau sont toujours numérotés de **0** à **nombre_éléments -1**. On accède à un élément du tableau en lui appliquant l'opérateur **[]**.

Exemple:

printf("tab[%d] = %d\n",i,tab[i]); == > tab[i-ème] = valeur à la i-ème case du tableau

3.1.1. Tableau à 1 dimension

❖ Initialisation et Affichage d'un tableau à 1 dimension:

On peut initialiser un tableau lors de sa déclaration par une liste de constantes de la façon suivante :

type nom-du-tableau[N] = {constante-1,constante-2,...,constante-N};

Par exemple, on peut écrire :

```
#include <stdio.h>
#include <stdlib.h>
#define N 4
int tab[N] = {1, 2, 3, 4};

int main() {
    int i;
    for (i = 0; i < N; i++){
        printf("tab[%d] = %d\n", i, tab[i]);
    }
    return 0;
}
```

==>

```
tab[0] = 1
tab[1] = 2
tab[2] = 3
tab[3] = 4
```

```
Process returned 0 (0x0)   execution time : 0.031 s
Press any key to continue.
```


3.1.1. Tableau à 1 dimension

❖ Saisie et affichage de t:

```
#include <stdio.h>
```

```
int main() {
```

```
    int tableau[5];
```

```
    // Saisie des éléments du tableau
```

```
    for(int i = 0; i < 5; i++) {
```

```
        printf("Entrez l'élément %d : ", i + 1);
```

```
        scanf("%d", &tableau[i]);
```

```
    }
```

```
    // Affichage des éléments du tableau
```

```
    printf("Les éléments du tableau sont :\n");
```

```
    for(int i = 0; i < 5; i++) {
```

```
        printf("%d ", tableau[i]);
```

```
    }
```

```
    printf("\n"); return 0; }
```

3.1.1. Tableau à 1 dimension

❖ Remarque:

Un tableau correspond en fait à un pointeur vers le premier élément du tableau. Ce pointeur est constant. Cela implique en particulier qu'aucune opération globale n'est autorisée sur un tableau. Notamment, un tableau ne peut pas figurer à gauche d'un opérateur d'affectation. Par exemple, on ne peut pas écrire **"tab1 = tab2;"**. Il faut effectuer l'affectation pour chacun des éléments du tableau :

```
#include <stdio.h>
#include <stdlib.h>
#define N 10

int main() {
    int tab1[N], tab2[N];
    int i;
    for (i = 0; i < N; i++)
        tab1[i] = tab2[i];
    return 0;
}
```

3.1.2. Les chaînes de caractères

Il n'existe pas de type prédéfini représentant une chaîne de caractères.

Les chaînes sont définies par des tableaux de caractères.

De la même manière un tableau de caractères peut être initialisé par une liste de caractères, mais aussi par une chaîne de caractères littéraux.

Remarque: Le compilateur complète toute chaîne de caractères avec un caractère nul **'\0'**. Il faut donc que le tableau ait au moins un élément de plus que le nombre de caractères de la chaîne littérale.

Exemple:

```
char chaine [10] = "BONJOUR";
```

== >

'B'	'o'	'n'	'j'	'o'	'u'	'r'	'\0'	'?'	'?'
-----	-----	-----	-----	-----	-----	-----	------	-----	-----

3.1.2. Les chaines de caractères

❖ Déclaration de chaines de caractères

On peut déclarer et initialiser des chaînes de caractères de différentes manières:

```
char chaine1 [] = "BONJOUR";  
char chaine2 [10] = "BONJOUR";  
char chaine3 [] = {'B', 'O', 'N', 'J', 'O', 'U', 'R', '\0'};  
char chaine4 [10] = {'B', 'O', 'N', 'J', 'O', 'U', 'R', '\0'};
```

3.1.2. Les chaines de caractères

❖ Lecture et Affichage de chaines de caractères

On peut utiliser les fonctions **scanf()** et **printf()** pour lire et afficher une chaîne de caractère. La fonction **scanf()** lit la séquence de caractères jusqu'à ce qu'elle rencontre un espace.

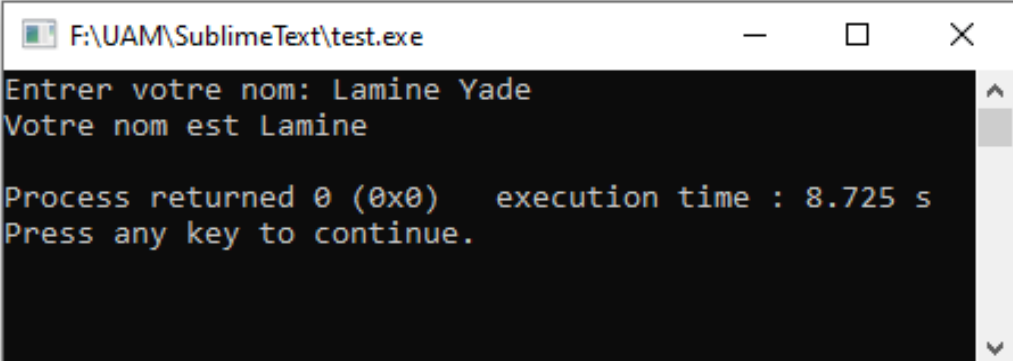
```
#include <stdio.h>
#include <stdlib.h>

int main() {
    //déclaration d'une chaine de 20 caractères
    char chaine [20];

    printf("Entrer votre nom: ");
    //Lecture de la chaine saisie
    scanf("%s", &chaine);
    //Affichage de la chaine
    printf("Votre nom est %s\n", chaine);

    return 0;
}
```

=== >



```
F:\UAM\SublimeText\test.exe
Entrer votre nom: Lamine Yade
Votre nom est Lamine

Process returned 0 (0x0)   execution time : 8.725 s
Press any key to continue.
```

Même si le nom « Lamine Yade » a été entré dans le programme ci-dessus, seul « Lamine » a été stocké dans le tableau chaine. C'est parce qu'il y avait un espace dans la chaîne.

3.1.2. Les chaines de caractères

❖ Lecture et Affichage de chaines de caractères

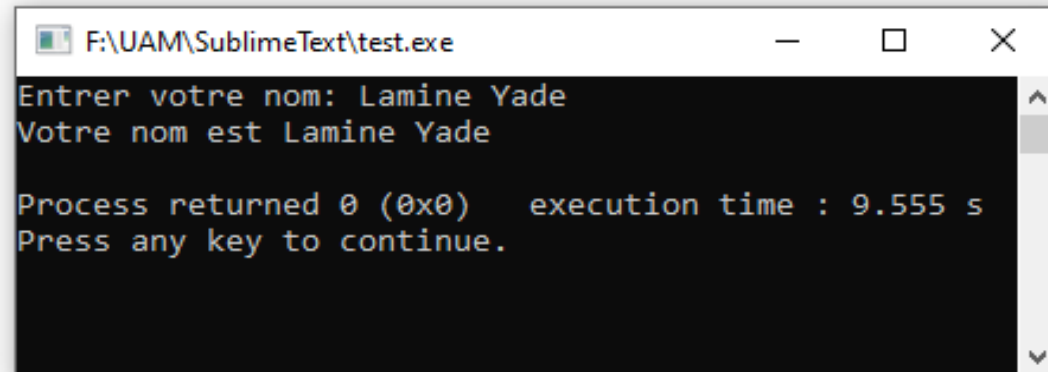
On peut utiliser la fonction **gets()** pour lire une chaîne de caractères avec des espaces. Et, utiliser la fonction **puts()** ou **printf()** pour afficher la chaîne:

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    //déclaration d'une chaine de 20 caractères
    char chaine [20];

    printf("Entrer votre nom: ");
    //Lecture de la chaine saisie
    gets(chaine);
    //Affichage de la chaine
    printf("Votre nom est ");
    puts(chaine);

    return 0;
}
```



```
F:\UAM\SublimeText\test.exe
Entrer votre nom: Lamine Yade
Votre nom est Lamine Yade

Process returned 0 (0x0)   execution time : 9.555 s
Press any key to continue.
```

3.1.2. Les chaines de caractères

❖ Parcourir une chaine de caractères

On peut parcourir une chaine avec une boucle comme un tableau d'entiers. Par exemple le code suivant parcourt la chaine et affiche le code ascii correspondant au **chaine[i-ème]** caractère. Avec la fonction **sizeof()** qui renvoie la taille du tableau:

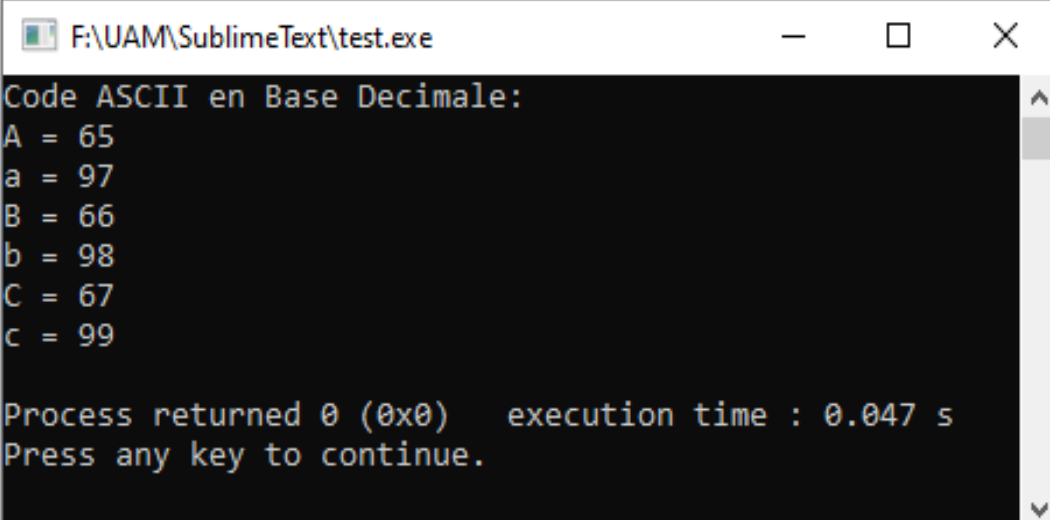
```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define N 10

int main() {

    char chaine [] = {'A', 'a', 'B', 'b', 'C', 'c'};

    printf("Code ASCII en Base Decimale:\n");
    for(int i=0; i<sizeof(chaine); i++){
        printf("%c = %d\n", chaine[i], chaine[i]);
    }

    return 0;
}
```



```
F:\UAM\SublimeText\test.exe
Code ASCII en Base Decimale:
A = 65
a = 97
B = 66
b = 98
C = 67
c = 99

Process returned 0 (0x0)   execution time : 0.047 s
Press any key to continue.
```

3.1.2. Les chaines de caractères

❖ Parcourir une chaine de caractères

Avec la fonction **strlen()** qui renvoie la longueur du tableau (la chaine):

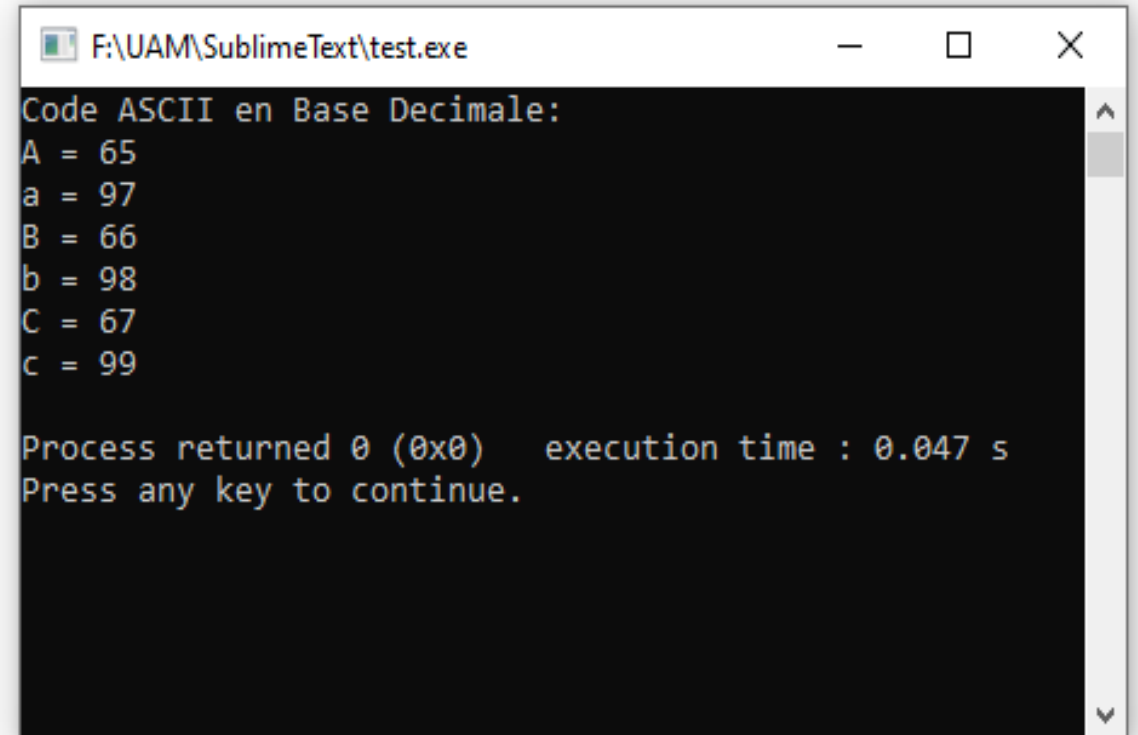
```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#define N 10

int main() {

    char chaine [] = {'A', 'a', 'B', 'b', 'C', 'c'};

    printf("Code ASCII en Base Decimale:\n");
    for(int i=0; i<strlen(chaine)-1; i++){
        printf("%c = %d\n", chaine[i], chaine[i]);
    }

    return 0;
}
```



```
F:\UAM\SublimeText\test.exe
Code ASCII en Base Decimale:
A = 65
a = 97
B = 66
b = 98
C = 67
c = 99

Process returned 0 (0x0) execution time : 0.047 s
Press any key to continue.
```


3.1.3. Tableaux Multidimensionnels

De manière similaire, on peut déclarer un tableau à plusieurs dimensions. Par exemple, pour un tableau à deux dimensions :

type nom-du-tableau[nombre-lignes][nombre-colonnes] ;

En fait, un tableau à deux dimensions est un tableau unidimensionnel dont chaque élément est lui-même un tableau. On accède à un élément du tableau par l'expression « **tableau[i][j]** ».

Pour initialiser un tableau à plusieurs dimensions à la compilation, on utilise une liste dont chaque élément est une liste de constantes :

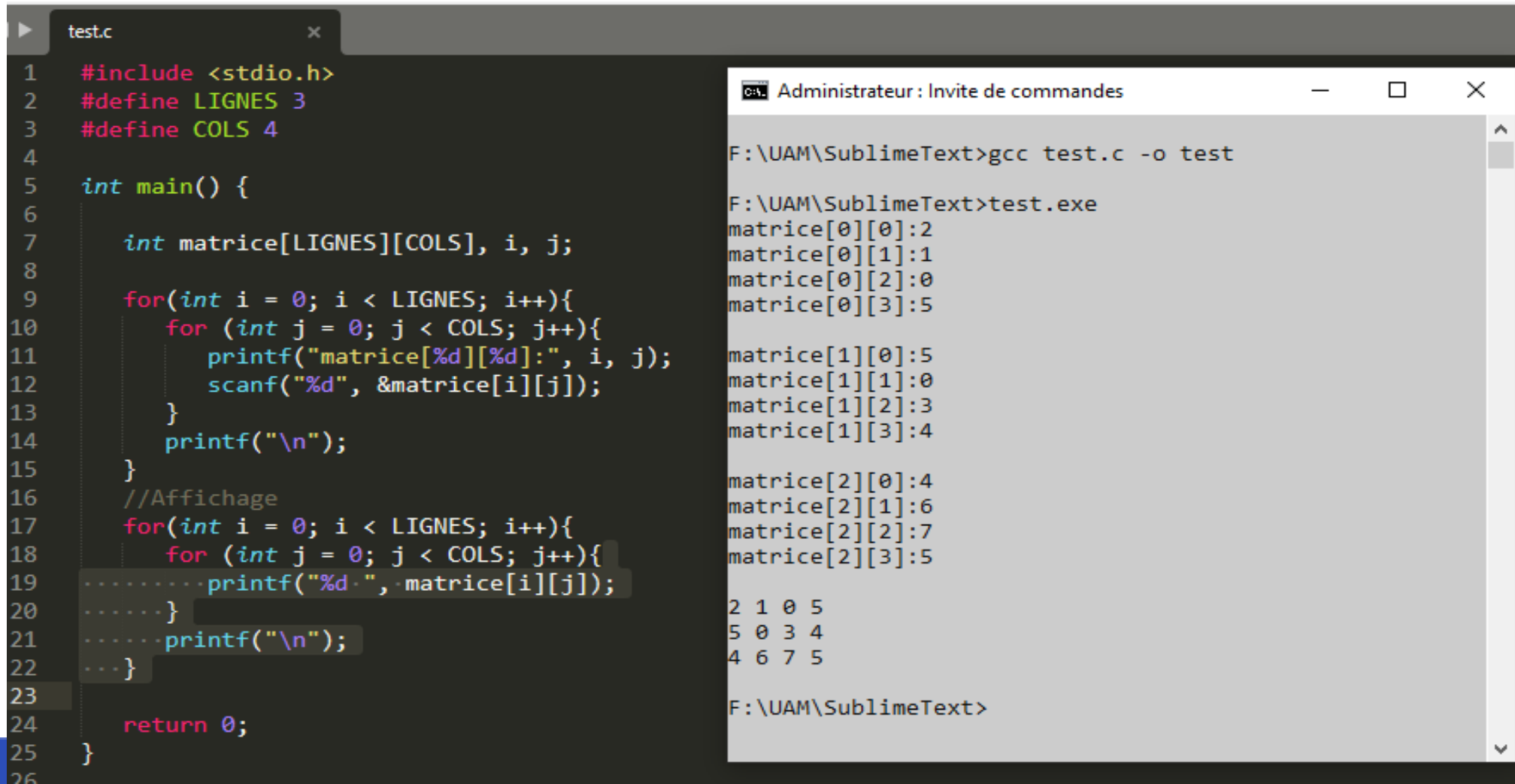
```
#include <stdio.h>
#define M 3
#define N 3

int tab[M][N] = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
```

3.1.3. Tableaux Multidimensionnels

❖ Saisie et Affichage d'un tableau à 2 dimensions:

Exemple: Un programme qui lit et affiche une matrice d'entiers de 3 lignes et 4 colonnes.



The image shows a C program in a text editor and its execution in a command prompt. The program defines a 3x4 matrix and reads values from the user. The output shows the matrix being filled row by row.

```
test.c
1  #include <stdio.h>
2  #define LIGNES 3
3  #define COLS 4
4
5  int main() {
6
7      int matrice[LIGNES][COLS], i, j;
8
9      for(int i = 0; i < LIGNES; i++){
10         for (int j = 0; j < COLS; j++){
11             printf("matrice[%d][%d]:", i, j);
12             scanf("%d", &matrice[i][j]);
13         }
14         printf("\n");
15     }
16     //Affichage
17     for(int i = 0; i < LIGNES; i++){
18         for (int j = 0; j < COLS; j++){
19             printf("%d ", matrice[i][j]);
20         }
21         printf("\n");
22     }
23
24     return 0;
25 }
26
```

```
Administrateur : Invite de commandes
F:\UAM\SublimeText>gcc test.c -o test
F:\UAM\SublimeText>test.exe
matrice[0][0]:2
matrice[0][1]:1
matrice[0][2]:0
matrice[0][3]:5
matrice[1][0]:5
matrice[1][1]:0
matrice[1][2]:3
matrice[1][3]:4
matrice[2][0]:4
matrice[2][1]:6
matrice[2][2]:7
matrice[2][3]:5
2 1 0 5
5 0 3 4
4 6 7 5
F:\UAM\SublimeText>
```

3.2. Les Structures

Les tableaux permettent de définir le type de variables qui peut contenir plusieurs éléments de données du **même type**.

De même, une structure est un autre type de données défini par l'utilisateur qui permet de combiner des éléments de **types différents**.

Une **structure** est, donc, une suite finie d'objets de **types différents**.

Contrairement aux tableaux, les différents éléments d'une structure n'occupent pas nécessairement des zones contiguës en mémoire.

Chaque élément de la structure, appelé **membre** ou **champ**, est désigné par un identificateur.

3.2. Les Structures

On distingue la déclaration d'un **modèle de structure** de celle d'un objet de **type structure** correspondant à un modèle donné.

Pour définir **modèle de structure**, on utilise l'instruction **struct**. L'instruction **struct** définit un **nouveau type** de données avec plusieurs membres.

Le format de l'instruction **struct** est le suivant:

```
struct modele
{
    type_1 membre_1;
    type_2 membre_2;
    ...
    type_n membre_n;
};
```

La déclaration d'un modèle de structure dont l'identificateur est **modele**

3.2. Les Structures

Pour déclarer un objet de type structure correspondant au modèle précédent, on utilise la syntaxe :

```
struct modele objet;
```

ou bien, si le modèle n'a pas été déclaré au préalable :

On accède aux différents membres d'une structure grâce à l'opérateur membre de structure, noté ".".

Le **i-ème** membre de objet est désigné par l'expression **objet.membre_i**

On peut effectuer sur le **i-ème** membre de la structure toutes les opérations valides sur des données de type **type_i**.

3.2. Les Structures

Par exemple, le programme suivant définit la structure **etudiant**, composée de 3 champs, **prenom**(string), **nom** (string), et **note**(double).

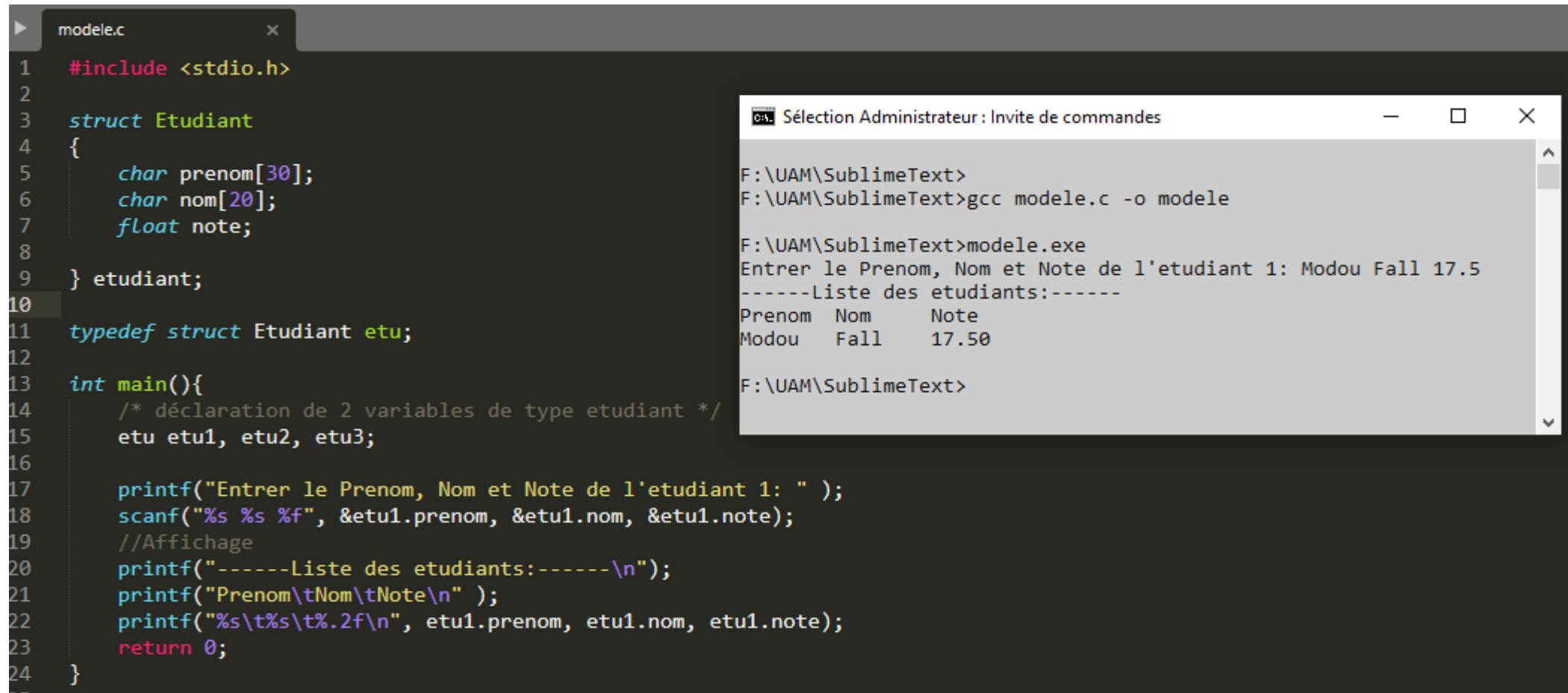
```
modele.c
1 #include <stdio.h>
2
3 struct Etudiant
4 {
5     char prenom[30];
6     char nom[20];
7     float note;
8 } etudiant;
9
10
11 int main()
12 {
13     /* déclaration de 2 variables de type etudiant */
14     struct Etudiant etu1, etu2;
15
16     printf("Entrer le Prenom, Nom et Note de l'etudiant 1: ");
17     scanf("%s %s %f", &etu1.prenom, &etu1.nom, &etu1.note);
18
19     printf("Entrer le Prenom, Nom et Note de l'etudiant 2: ");
20     scanf("%s %s %f", &etu2.prenom, &etu2.nom, &etu2.note);
21     //Affichage
22     printf("-----Liste des etudiants:-----\n");
23     printf("Prenom\tNom\tNote\n");
24     printf("%s\t%s\t%.2f\n", etu1.prenom, etu1.nom, etu1.note);
25     printf("%s\t%s\t%.2f\n", etu2.prenom, etu2.nom, etu2.note);
26     return 0;
27 }
28
```

```
Administrateur : Invite de commandes
F:\UAM\SublimeText>gcc modele.c -o modele
F:\UAM\SublimeText>modele.exe
Entrer le Prenom, Nom et Note de l'etudiant 1: Modou Fall 17
Entrer le Prenom, Nom et Note de l'etudiant 2: Fatou Ndiaye 18
-----Liste des etudiants:-----
Prenom Nom      Note
Modou   Fall      17.00
Fatou   Ndiaye    18.00
F:\UAM\SublimeText>
```

3.2. Les Structures

3.2.1. Types composés avec typedef

Pour alléger l'écriture des programmes, on peut affecter un nouvel identificateur à un type composé à l'aide de **typedef** : **typedef type synonyme;**



The image shows a C program in a file named `modele.c` and its execution output in a command prompt window.

```
1  #include <stdio.h>
2
3  struct Etudiant
4  {
5      char prenom[30];
6      char nom[20];
7      float note;
8  } etudiant;
9
10
11 typedef struct Etudiant etu;
12
13 int main(){
14     /* déclaration de 2 variables de type etudiant */
15     etu etu1, etu2, etu3;
16
17     printf("Entrer le Prenom, Nom et Note de l'etudiant 1: ");
18     scanf("%s %s %f", &etu1.prenom, &etu1.nom, &etu1.note);
19     //Affichage
20     printf("-----Liste des etudiants:-----\n");
21     printf("Prenom\tNom\tNote\n");
22     printf("%s\t%s\t%.2f\n", etu1.prenom, etu1.nom, etu1.note);
23     return 0;
24 }
```

The command prompt window shows the following output:

```
Sélection Administrateur : Invite de commandes
F:\UAM\SublimeText>
F:\UAM\SublimeText>gcc modele.c -o modele
F:\UAM\SublimeText>modele.exe
Entrer le Prenom, Nom et Note de l'etudiant 1: Modou Fall 17.5
-----Liste des etudiants:-----
Prenom  Nom    Note
Modou   Fall    17.50
F:\UAM\SublimeText>
```

3.3. Union

Union est un type de données en programmation C qui permet de stocker **différents types** de données dans les **mêmes emplacements mémoire** .

Union fournit un moyen efficace de **réutiliser** l'emplacement mémoire, car un seul de ses membres est accessible à la fois.

Une **union** est utilisée presque de la même manière que vous déclareriez et utiliseriez une **structure**.

La **taille** d'une union en C est déterminée par son **plus grand membre**.

Par exemple, si une union contient un **entier** et un **tableau de 20 entiers**, la taille de l'union sera la taille du plus grand membre, soit **80 octets (20 éléments * 4 octets chacun)**.

3.3. Union

3.3.1. Syntaxe pour déclarer une Union

La syntaxe pour définir une union en langage C

```
union nom_union{  
    type membre_1;  
    type membre_2;  
    ...  
    type membre_n;  
} [union variables]; //Facultatif
```

Chaque définition de membre est une définition de variable normale, telle que « **int i;** » ou « **float f;** » ou toute autre définition de variable valide.

À la fin de la définition de l'union, avant le point-virgule final, vous pouvez spécifier une ou plusieurs variables d'union.

3.3. Union

3.3.2. Accès aux membres d'une union

Tout comme la **Structure**, pour accéder à n'importe quel **membre** d'une union, nous utilisons l'opérateur d'accès aux membres (.).

L'opérateur d'accès aux membres est codé sous la forme d'un point entre le **nom de la variable d'union** et le **membre de l'union** auquel nous souhaitons accéder.

Vous utiliserez le mot-clé **union** pour définir des variables de type union.

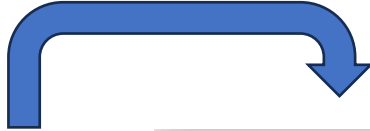
Syntaxe:

```
nom_union.membre_1;           // accès au membre_1 de l'union  
nom_union.membre_n;           // accès au membre_n de l'union  
nom_union.membre_1 = valeur1; //initialisation du membre_1  
nom_union.membre_2 = valeur2; //initialisation du membre_2
```

3.3. Union

Exemple1:

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4
5  union Donnees{
6      int code_etudiant;
7      char nom[50];
8      char ufr[10];
9      float note;
10 };
11
12 int main(){
13     union Donnees etudiant;
14
15     etudiant.code_etudiant = 1001; // initialisation du membre code_etudiant
16     etudiant.note = 18.5; // initialisation du membre note
17     strcpy(etudiant.nom, "Idrissa Wade"); // initialisation du membre nom avec strcpy
18     strcpy(etudiant.ufr, "UFR-STA");
19
20     printf("Code: %d \n", etudiant.code_etudiant);
21     printf("Note: %f \n", etudiant.note);
22     printf("nom: %s \n", etudiant.nom);
23     printf("UFR: %s \n", etudiant.ufr);
24
25     return 0;
26 }
```



```
C:\> Administrateur : Invite de commandes

F:\UAM>gcc unionC.c -o unionC

F:\UAM>unionC.exe
Code: 760366677
Note: 0.000000
nom: UFR-STA
UFR: UFR-STA

F:\UAM>
```

Ici, on constate que les valeurs de **code**, **note** et **nom** (membres de l'union) affichent des valeurs erronées car la valeur finale attribuée à la variable a occupé l'emplacement mémoire et c'est la raison pour laquelle la valeur du membre **ufr** est très bien imprimée.

3.3. Union

Exemple2:

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4
5  union Donnees{
6      int code_etudiant;
7      char nom[50];
8      char ufr[10];
9      float note;
10 };
11
12 int main(){
13     union Donnees etudiant;
14
15     etudiant.code_etudiant = 1001;
16     printf("Code: %d \n", etudiant.code_etudiant);
17     strcpy(etudiant.nom, "Idrissa Wade");
18     printf("nom: %s \n", etudiant.nom);
19     strcpy(etudiant.ufr, "UFR-STA");
20     printf("UFR: %s \n", etudiant.ufr);
21     etudiant.note = 18.5;
22     printf("Note: %f \n", etudiant.note);
23
24     return 0;
25 }
```



```
Administrateur : Invite de commandes

F:\UAM>gcc unionC.c -o unionC

F:\UAM>unionC.exe
Code: 1001
nom: Idrissa Wade
UFR: UFR-STA
Note: 18.500000

F:\UAM>
```

Ici, les valeurs de tous les membres du syndicat sont très bien imprimées car un membre est utilisé à la fois.

3.3. Union

3.3.3. Différence entre Structure et Union

Structures et les **Union** sont des types de données composés en programmation C.

- La différence la plus significative entre une structure et une union est la façon dont elles stockent leurs données. Une structure stocke chaque membre dans des emplacements mémoire **distincts**, tandis qu'une union stocke tous ses membres dans le **même** emplacement mémoire.
- La définition d'une union est similaire à la définition d'une structure.
- Une définition du type d'union appelé **myunion** et une du type de structure appelé **mystruct**:

```
union myunion{  
    int a;  
    double b;  
    char c;  
};|
```

```
struct mystruct{  
    int a;  
    double b;  
    char c;  
};
```

3.3. Union

3.3.3. Différence entre Structure et Union

- La principale différence entre une structure et une union est la **taille des variables**.
- Le compilateur alloue la mémoire à une variable de structure, pour pouvoir stocker les valeurs de tous les éléments.
- Dans **mystruct**, il y a trois éléments – un **int**, un **double** et un **char**, nécessitant 13 octets ($4 + 8 + 1$). Par conséquent, **sizeof(struct mystruct)** renvoie **13**.
- En revanche, pour une variable de type union, le compilateur alloue un bloc de mémoire d'une taille suffisante pour accueillir l'élément de la **plus grande taille en octets**.
- Le type **myunion** possède un élément **int**, un **double** et un **char**. Parmi les trois éléments, la taille de la variable **double** est la plus grande, c'est-à-dire **8**. Ainsi, **sizeof(union myunion)** renvoie **8**.
- Un autre point à prendre en considération est qu'une variable d'union ne peut contenir que la valeur d'un seul de ses éléments. Lorsque vous attribuez une valeur à un élément, les autres éléments ne sont pas définis. Si vous essayez d'utiliser les autres éléments, cela produira des résultats erronés.

3.4. Les Enumérations

Les **énumérations** permettent de définir un **type** par la liste des valeurs qu'il peut prendre.

Un objet de type énumération est défini par le mot-clef **enum** et un identificateur de **modèle**, suivis de la liste des valeurs que peut prendre cet objet :

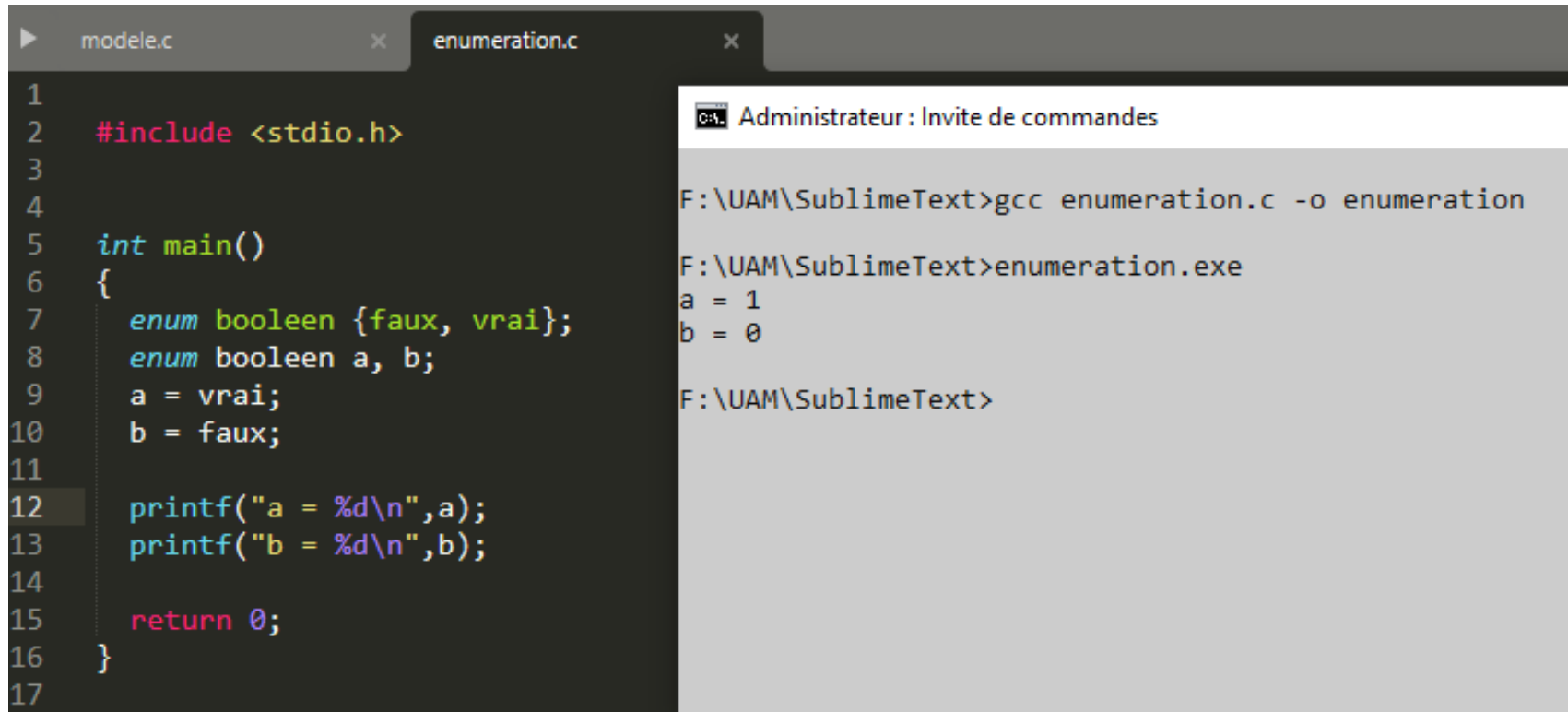
```
enum modele {constante-1, constante-2,...,constante-n};
```

3.4. Les Enumérations

En réalité, les objets de type **enum** sont représentés comme des **int**.

Les valeurs possibles **constante-1**, **constante-2**,...,**constante-n** sont codées par des entiers de **0** à **n-1**.

Par exemple, le type **enum booleen** défini dans le programme suivant associe l'entier **0** à la valeur **faux** et l'entier **1** à la valeur **vrai**.



```
modele.c x enumeration.c x
1
2  #include <stdio.h>
3
4
5  int main()
6  {
7      enum booleen {faux, vrai};
8      enum booleen a, b;
9      a = vrai;
10     b = faux;
11
12     printf("a = %d\n",a);
13     printf("b = %d\n",b);
14
15     return 0;
16 }
17
```

```
Administrateur : Invite de commandes
F:\UAM\SublimeText>gcc enumeration.c -o enumeration
F:\UAM\SublimeText>enumeration.exe
a = 1
b = 0
F:\UAM\SublimeText>
```


3.4. Les Enumérations

En On peut modifier le codage par défaut des valeurs de la liste lors de la déclaration du type énuméré. Exemple : **enum booleen {faux = 12, vrai = 23};**

```
modele.c x enumeration.c x
1  #include <stdio.h>
2
3  int main()
4  {
5      enum boolee {faux = 500, vrai = 200};
6      enum boolee a, b;
7      a = vrai;
8      b = faux;
9
10     printf("a = %d\n",a);
11     printf("b = %d\n",b);
12
13     return 0;
14 }
15
```

```
Administrateur : Invite de commandes
F:\UAM\SublimeText>gcc enumeration.c -o enumeration
F:\UAM\SublimeText>enumeration.exe
a = 200
b = 500
F:\UAM\SublimeText>
```

Fin chapitre 3

Chapitre 4: Les pointeurs

- Compréhension des pointeurs et de l'adressage mémoire
- Opérations sur les pointeurs
- Pointeurs et Tableaux
- Pointeurs et Fonctions

4.1 Compréhension des pointeurs et de l'adressage mémoire

- Toute variable manipulée dans un programme est stockée quelque part en mémoire centrale. Cette mémoire est constituée d'octets qui sont identifiés de manière univoque par un numéro qu'on appelle **adresse**.
- Pour retrouver une variable, il suffit donc de connaître l'**adresse** de l'octet où elle est stockée (ou, s'il s'agit d'une variable qui recouvre plusieurs octets contigus, l'adresse du premier de ces octets).
- Pour des raisons évidentes de lisibilité, on désigne souvent les variables par des identificateurs, et non par leur adresse.
- C'est le compilateur qui fait alors le lien entre l'identificateur d'une variable et son adresse en mémoire. Toutefois, il est parfois très pratique de manipuler directement une variable par son adresse.

4.1.1 Adresse et valeur d'un objet

On appelle **Lvalue** (left value) tout objet pouvant être placé à gauche d'un opérateur d'affectation. Une **Lvalue** est caractérisée par :

- son adresse, c'est-à-dire l'adresse-mémoire à partir de laquelle l'objet est stocké ;
- sa valeur, c'est-à-dire ce qui est stocké à cette adresse.

Dans l'exemple,

int **i**, **j**;

i = 3;

j = **i**;

Si le compilateur a placé la variable **i** à l'adresse **1000** en mémoire, et la variable **j** à l'adresse **1002**, on a :

Objet	Adresse	Valeur
i	1000	3
j	1002	3

4.1.1 Adresse et valeur d'un objet

Deux variables différentes ont des adresses différentes. L'affectation $i = j;$ n'opère que sur les valeurs des variables. Les variables i et j étant de type int, elles sont stockées sur 4 octets. Ainsi la valeur de i est stockée sur les octets d'adresse **1000** à **1003**.

L'adresse d'un objet étant un numéro d'octet en mémoire, il s'agit d'un entier quelque soit le type de l'objet considéré. Le format interne de cet entier (16 bits, 32 bits ou 64 bits) dépend des architectures.

L'opérateur $\&$ permet d'accéder à l'adresse d'une variable. Toutefois $\&i$ n'est pas une **Lvalue** mais une constante : on ne peut pas faire figurer $\&i$ à gauche d'un opérateur d'affectation.

Pour pouvoir manipuler des adresses, on doit donc recourir un nouveau type d'objets, les **pointeurs.**

4.1.2 Notion de Pointeur en C

Un pointeur est un objet (**Lvalue**) dont la **valeur** est égale à l'**adresse** d'un autre objet. On déclare un pointeur par l'instruction :

type ***nom-du-pointeur**;

où **type** est le type de l'objet pointé. Cette déclaration déclare un identificateur, **nom-du-pointeur**, associé à un objet dont la valeur est l'adresse d'un autre objet de type **type**.

L'identificateur **nom-du-pointeur** est donc en quelque sorte un identificateur d'adresse.

Comme pour n'importe quelle **Lvalue**, sa valeur est modifiable.

4.1.2 Notion de Pointeur en C

Même si la valeur d'un pointeur est toujours un entier (éventuellement un entier long), le type d'un pointeur dépend du type de l'objet vers lequel il pointe. Cette distinction est indispensable à l'interprétation de la valeur d'un pointeur.

En effet, pour un pointeur sur un objet de type **char**, la valeur donne l'adresse de l'octet où cet objet est stocké. Par contre, pour un pointeur sur un objet de type **int**, la valeur donne l'adresse du premier des 2 octets où l'objet est stocké.

Dans l'exemple suivant, on définit un pointeur **ptr** qui pointe vers un entier **i** :

```
int i = 3;  
int *ptr;  
ptr = &i;
```

On se trouve dans la configuration :



Objet	Adresse	Valeur
i	1000	3
ptr	1002	1000

4.1.2 Notion de Pointeur en C

L'opérateur ***** permet d'accéder directement à la valeur de l'objet pointé. Ainsi, si **ptr** est un pointeur vers un entier **i**, ***p** désigne la **valeur de i**.

Exemple:

```
#include <stdio.h>

int main(int argc, char const *argv[])
{
    int i = 3;
    int *ptr;
    ptr = &i;
    printf("*ptr = %d \n", *ptr);

    return 0;
}
```

Affiche ***p = 3**

Objet	Adresse	Valeur
i	1000	3
ptr	1002	1000
*ptr	1000	3

Dans ce programme, les objets **i** et ***ptr** sont identiques : ils ont mêmes adresses et valeurs. Cela signifie en particulier que toute modification de ***ptr** modifie **i**.

Ainsi, si l'on ajoute l'instruction ***ptr = 0;** à la fin du programme précédent, la valeur de **i** devient nulle.

4.1.2 Notion de Pointeur en C

On peut donc dans un programme manipuler à la fois les objets **ptr** et ***ptr**. Ces deux manipulations sont très différentes. Comparons par exemple les deux programmes suivants :

```
#include <stdio.h>

int main()
{
    int i = 3, j = 6;
    int *p1, *p2;
    p1 = &i;
    p2 = &j;
    *p1 = *p2;

    return 0;
}
```

```
#include <stdio.h>

int main()
{
    int i = 3, j = 6;
    int *p1, *p2;
    p1 = &i;
    p2 = &j;
    p1 = p2;

    return 0;
}
```

4.1.2 Notion de Pointeur en C

Avant la dernière affectation de chacun de ces programmes, on est dans une configuration du type :

Objet	Adresse	Valeur
i	100	3
j	102	6
p1	104	100
p2	106	102

Après l'affectation `*p1 = *p2;` du premier programme, on a :

Objet	Adresse	Valeur
i	100	6
j	102	6
p1	104	100
p2	106	102

Par contre, l'affectation `p1 = p2` du second programme, conduit à la situation :

Objet	Adresse	Valeur
i	100	3
j	102	6
p1	104	102
p2	106	102

4.2 Opérations sur les Pointeurs

La valeur d'un pointeur étant un entier, on peut lui appliquer un certain nombre d'opérateurs arithmétiques classiques. Les seules opérations arithmétiques valides sur les pointeurs sont :

- l'**addition** d'un entier à un pointeur. Le résultat est un pointeur de même type que le pointeur de départ ;
- la **soustraction** d'un entier à un pointeur. Le résultat est un pointeur de même type que le pointeur de départ ;
- la **différence** de deux pointeurs pointant tous deux vers des objets de même type. Le résultat est un entier.

Si **i** est un entier et **p** est un pointeur sur un objet de type **type**, l'expression **p + i** désigne un pointeur sur un objet de type **type** dont la valeur est égale à la valeur de **p** incrémentée de **i * sizeof(type)**. Il en va de même pour la soustraction d'un entier à un pointeur, et pour les opérateurs d'incrément et de décrémentation **++** et **--**. Par exemple, le programme suivant affiche : **p1 = 100 p2 = 104**

```
#include <stdio.h>

int main()
{
    int i = 3;
    int *p1, *p2;
    p1 = &i;
    p2 = p1 + 1;
    printf("p1 = %ld \t p2 = %ld\n",p1,p2);

    return 0;
}
```

Objet	Adresse	Valeur
i	100	3
p1	104	100
p2	106	104

4.2 Opérations sur les Pointeurs

Rappel:

Il existe en langage C des types dits scalaires (de base)

char : caractère (octet)

int : entier (32 ou 64 bits)

float : réel 32 bits format IEEE 754

double : réel 64 bits format IEEE 754

4.2 Opérations sur les Pointeurs

Les opérateurs de comparaison sont également applicables aux pointeurs, à condition de **comparer des pointeurs qui pointent vers des objets de même type**.

L'utilisation des opérations arithmétiques sur les pointeurs est particulièrement utile pour parcourir des tableaux. Ainsi, le programme suivant imprime les éléments du tableau tab dans l'ordre croissant puis décroissant des indices.

```
#include <stdio.h>
#define N 5

int tab[5] = {1, 2, 6, 0, 7};

int main(){
    int *p;

    printf("\n Ordre croissant:\n");

    for (p = &tab[0]; p <= &tab[N-1]; p++)
        printf(" %d ",*p);

    printf("\n Ordre decroissant:\n");

    for (p = &tab[N-1]; p >= &tab[0]; p--)
        printf(" %d ",*p);

    return 0;
}
```



```
F:\UAM\pointeur>gcc pointeur.c -o pointeur
F:\UAM\pointeur>pointeur.exe

Ordre croissant:
1  2  6  0  7

Ordre decroissant:
7  0  6  2  1

F:\UAM\pointeur>
```

4.3 Allocation dynamique

Avant de manipuler un pointeur, et notamment de lui appliquer l'opérateur d'indirection *****, il faut l'initialiser. Sinon, par défaut, la valeur du pointeur est égale à une constante symbolique notée **NULL** définie dans **stdio.h**. En général, cette constante vaut **0**. Le test **p == NULL** permet de savoir si le pointeur **p** pointe vers un objet.

On peut initialiser un pointeur **p** par une affectation sur **p**. Par exemple, on peut affecter à **p** l'adresse d'une autre variable.

Il est également possible d'affecter directement une valeur à ***p**. Mais pour cela, il faut d'abord réserver à ***p** un espace-mémoire de taille adéquate. L'**adresse** de cet espace-mémoire sera la **valeur** de **p**.

Cette opération consistant à réserver un espace-mémoire pour stocker l'objet pointé s'appelle **allocation dynamique**. Elle se fait en C par la fonction **malloc** de la librairie standard **stdlib.h**.

Sa syntaxe est: **malloc(nombre-octets);**

4.3 Allocation dynamique

`malloc(nombre-octets);`

Cette fonction retourne un pointeur de type `char *` pointant vers un objet de taille `nombre-octets` octets.

Pour initialiser des pointeurs vers des objets qui ne sont pas de type `char`, il faut convertir le type de la sortie de la fonction `malloc` à l'aide d'un `cast`. L'argument `nombre-octets` est souvent donné à l'aide de la fonction `sizeof()` qui renvoie le nombre d'octets utilisés pour stocker un objet.

Ainsi, pour initialiser un pointeur vers un entier, on écrit :

```
#include <stdlib.h>
int *p;
p = (int*)malloc(sizeof(int));
```

On aurait pu écrire également:

```
p = (int*)malloc(4);
```

puisque un objet de type `int` est stocké sur **4 octets**. Mais on préférera la première écriture qui a l'avantage d'être portable.

Remarque : Le programme ci-dessous ne nécessite pas d'allocation dynamique puisque l'espace-mémoire à l'adresse `&i` est déjà réservé pour un entier.

```
main()
{
    int i = 3;
    int *p;
    p = &i;
}
```


4.3 Allocation dynamique

Ce programme suivant définit un pointeur **p** sur un objet ***p** de type int, et affecte à ***p** la valeur de la variable **i**

```
#include <stdio.h>
#include <stdlib.h>
main()
{
    int i = 3;
    int *p;
    printf("valeur de p avant initialisation = %ld\n",p);
    p = (int*)malloc(sizeof(int));
    printf("valeur de p apres initialisation = %ld\n",p);
    *p = i;
    printf("valeur de *p = %d\n",*p);
}
```

Sortie:

valeur de p avant initialisation = -1191501240
valeur de p apres initialisation = 1007234560
valeur de *p = 3

Avant l'allocation dynamique, ***p** n'a aucun sens. En particulier, toute manipulation de la variable ***p** générerait une violation mémoire, détectable à l'exécution par le message d'erreur **Segmentation fault**.

L'allocation dynamique a pour résultat d'attribuer une valeur à **p** et de réserver à cette adresse un espace-mémoire composé de 4 octets pour stocker la valeur de ***p**.

***p** est maintenant définie mais sa valeur n'est pas initialisée. Cela signifie que ***p** peut valoir n'importe quel entier (celui qui se trouvait précédemment à cette adresse). L'affectation ***p = i;** a enfin pour résultat d'affecter à ***p** la valeur de **i**.

4.3 Allocation dynamique

La fonction `calloc` de la librairie `stdlib.h` a le même rôle que la fonction `malloc` mais elle initialise en plus l'objet pointé `*p` à **zéro**.

Sa syntaxe est : `calloc(nb-objets, taille-objets)`

Ainsi, si `p` est de type `int*`, l'instruction

```
p = (int*)calloc(N, sizeof(int));
```

est strictement équivalente à :

```
p = (int*)malloc(N * sizeof(int));  
for (i = 0; i < N; i++)  
    *(p + i) = 0;
```

Remarque: L'emploi de `calloc` est simplement plus rapide.

4.3 Allocation dynamique

Enfin, lorsque l'on n'a plus besoin de l'espace-mémoire alloué dynamiquement (c'est-à-dire quand on n'utilise plus le pointeur `p`), il faut libérer cette place en mémoire.

Ceci se fait à l'aide de l'instruction **free** qui a pour syntaxe:

free(nom-du-pointeur);

A toute instruction de type **malloc** ou **calloc** doit être associée une instruction de type **free**.

4.4 Pointeurs et Tableaux

4.4.1 Pointeurs et Tableaux à une dimension

L'usage des pointeurs en C est, en grande partie, orienté vers la manipulation des tableaux.

Tout tableau en C est en fait un pointeur constant. Dans la déclaration `int tab[10];` **tab** est un pointeur constant (non modifiable) dont la valeur est l'**adresse du premier élément du tableau**. Autrement dit, **tab** a pour valeur **&tab[0]**. On peut donc utiliser un pointeur initialisé à **tab** pour parcourir les éléments du tableau.

On accède à l'élément d'indice **i** du tableau **tab** grâce à l'opérateur d'indexation `[]`, par l'expression **tab[i]**.

Cet opérateur d'indexation peut en fait s'appliquer à tout objet **p** de type pointeur. Il est lié à l'opérateur d'indirection `*` par la formule

$$p[i] = *(p + i)$$

Programme 1	Programme 2
<pre>#define N 5 int tab[5] = {1, 2, 6, 0, 7}; main() { int i; int *p; p = tab; for (i = 0; i < N; i++) { printf(" %d \n", *p); p++; } }</pre>	<pre>#define N 5 int tab[5] = {1, 2, 6, 0, 7}; main() { int i; int *p; p = tab; for (i = 0; i < N; i++) printf(" %d \n", p[i]); }</pre>

4.4.1 Pointeurs et Tableaux à une dimension

Pointeurs et **tableaux** se manipulent donc exactement de même manière. Les deux programmes précédents sont donc équivalents.

Toutefois, la manipulation de tableaux, et non de pointeurs, possède certains **inconvénients** dus au fait **qu'un tableau est un pointeur constant**.

Ainsi

- on ne peut pas créer de tableaux dont la taille est une variable du programme.
- on ne peut pas créer de tableaux bidimensionnels dont les lignes n'ont pas toutes le même nombre d'éléments.

Ces opérations deviennent possibles dès que l'on manipule des **pointeurs alloués dynamiquement**. Ainsi, pour créer un tableau d'entiers à **n** éléments où **n** est une variable du programme, on écrit :

Si on veut en plus que tous les éléments du tableau **tab** soient initialisés à zéro, on remplace l'allocation dynamique avec **malloc** par

```
tab = (int*)calloc(n, sizeof(int));
```

Les éléments de **tab** sont manipulés avec l'opérateur d'indexation **[]**, exactement comme pour les tableaux.

```
#include <stdlib.h>
main()
{
    int n;
    int *tab;

    ...
    tab = (int*)malloc(n * sizeof(int));
    ...
    free(tab);
}
```

4.4.1 Pointeurs et Tableaux à une dimension

Les deux différences principales entre un tableau et un pointeur sont :

- un **pointeur** doit toujours être initialisé, soit par une **allocation dynamique**, soit par affectation d'une expression adresse, par exemple ***p = &i ;***
- un **tableau** n'est pas une **Lvalue** ; il ne peut donc pas figurer à gauche d'un opérateur d'affectation. En particulier, un tableau ne supporte pas l'**arithmétique** (on ne peut pas écrire `tab++`;

4.4.2 Pointeurs et Tableaux à plusieurs dimensions

Un tableau à deux dimensions est, par définition, un **tableau de tableaux**. Il s'agit donc en fait d'un **pointeur** vers un **pointeur**. Considérons le tableau à deux dimensions défini par :

```
int tab[M][N];
```

tab est un **pointeur**, qui pointe vers un objet lui-même de type pointeur d'entier. **tab** a une valeur constante égale à l'adresse du premier élément du tableau, **&tab[0][0]**.

De même **tab[i]** , pour **i** entre **0** et **M-1**, est un pointeur constant vers un objet de type entier, qui est le premier élément de la ligne d'indice **i**. **tab[i]** a donc une valeur constante qui est égale à **&tab[i][0]**.

4.4.2 Pointeurs et Tableaux à plusieurs dimensions

Exactement comme pour les tableaux à une **dimension**, les **pointeurs de pointeurs** ont de nombreux avantages sur les tableaux multi-dimensionnés.

On déclare un pointeur qui pointe sur un objet de type **type *** (deux dimensions) de la même manière qu'un **pointeur**, c'est-à-dire: **type **nom-du-pointeur;**

De même un pointeur qui pointe sur un objet de type **type **** (équivalent à un tableau à 3 dimensions) se déclare par **type ***nom-du-pointeur;**

Par exemple, pour créer avec un pointeur de pointeur une matrice à **k lignes** et **n colonnes** à coefficients entiers, on écrit :

```
int main(){
    int k, n; int **tab;

    tab = (int**)malloc(k * sizeof(int*));
    for (i = 0; i < k; i++)
        tab[i] = (int*)malloc(n * sizeof(int));

    for (i = 0; i < k; i++)
        free(tab[i]);

    free(tab);

    return 0;
}
```

La première allocation dynamique réserve pour l'objet pointé par tab l'espace-mémoire correspondant à k pointeurs sur des entiers. Ces k pointeurs correspondent aux lignes de la matrice. Les allocations dynamiques suivantes réservent pour chaque pointeur tab[i] l'espace-mémoire nécessaire pour stocker n entiers.

Si on désire en plus que tous les éléments du tableau soient initialisés à zéro, il suffit de remplacer l'allocation dynamique dans la boucle for par

tab[i] = (int*)calloc(n, sizeof(int));

Contrairement aux tableaux à deux dimensions, on peut choisir des tailles différentes pour chacune des lignes tab[i]. Par exemple, si l'on veut que tab[i] contienne exactement i+1 éléments, on écrit

for (i = 0; i < k; i++)

tab[i] = (int*)malloc((i + 1) * sizeof(int));

4.5 Pointeurs et Chaines de caractères

On a vu précédemment qu'une **chaîne de caractères** était un **tableau à une dimension** d'objets de type **char**, se terminant par le caractère nul **'\0'**. On peut donc manipuler toute chaîne de caractères à l'aide d'un **pointeur** sur un objet de type **char**.

On peut faire subir à une chaîne définie par

char *chaine;

des affectations comme

chaine = "ceci est une chaine";

et toute opération valide sur les pointeurs, comme l'instruction **chaine++**. Ainsi, le programme suivant imprime le nombre de caractères d'une chaîne (sans compter le caractère nul).

```
#include <stdio.h>
main()
{
    int i;
    char *chaine = "chaine de caracteres";
    for (i = 0; *chaine != '\0'; i++)
        chaine++;
    printf("nombre de caracteres = %d\n",i);
}
```

La fonction donnant la longueur d'une chaîne de caractères, définie dans la librairie standard **string.h**, procède de manière identique. Il s'agit de la fonction **strlen** dont la syntaxe est **strlen(chaine)**;

où **chaine** est un **pointeur** sur un objet de type **char**. Cette fonction renvoie un entier dont la valeur est égale à la longueur de la chaîne passée en argument (moins le caractère **'\0'**).

4.5 Pointeurs et Chaines de caractères

L'utilisation de pointeurs de caractère et non de tableaux permet par exemple de créer une chaîne correspondant à la concaténation de deux chaînes de caractères :

On remarquera l'utilisation d'un pointeur intermédiaire `p` qui est indispensable dès que l'on fait des opérations de type incrémentation.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
main()
{
    int i;
    char *chaine1, *chaine2, *res, *p;

    chaine1 = "chaine ";
    chaine2 = "de caracteres";
    res = (char*)malloc((strlen(chaine1) + strlen(chaine2)) *
sizeof(char));
    p = res;
    for (i = 0; i < strlen(chaine1); i++)
        *p++ = chaine1[i];
    for (i = 0; i < strlen(chaine2); i++)
        *p++ = chaine2[i];
    printf("%s\n",res);
}
```

4.5 Pointeurs et Chaines de caractères

En effet, si on avait incrémenté directement la valeur de **res**, on aurait évidemment « **perdu** » la référence sur le premier caractère de la chaîne. Par exemple:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
main(){
    int i;
    char *chaine1, *chaine2, *res;

    chaine1 = "chaine ";
    chaine2 = "de caracteres";
    res = (char*)malloc((strlen(chaine1) + strlen(chaine2)) *
sizeof(char));
    for (i = 0; i < strlen(chaine1); i++)
        *res++ = chaine1[i];
    for (i = 0; i < strlen(chaine2); i++)
        *res++ = chaine2[i];
    printf("\nnombre de caracteres de res = %d\n",strlen(res));
}
```

imprime la valeur **0**, puisque **res** a été modifié au cours du programme et pointe maintenant sur le caractère **nul**.

4.6 Pointeurs et Structures

Contrairement aux **tableaux**, les objets de type **structure** en C sont des **Lvalues**.

Ils possèdent une **adresse**, correspondant à l'adresse du premier élément du premier **membre de la structure**.

On peut donc manipuler des pointeurs sur des structures.

Ainsi, le programme suivant crée, à l'aide d'un pointeur, un tableau d'objets de type **structure**.

4.6 Pointeurs et Structures

```
#include <stdlib.h>
#include <stdio.h>

struct eleve
{
    char nom[20];
    int date;
};

typedef struct eleve *classe;

main() {
    int n, i;
    classe tab;

    printf("nombre d'eleves de la classe = ");
    scanf("%d",&n);
    tab = (classe)malloc(n * sizeof(struct eleve));
```

```
    for (i = 0 ; i < n; i++)
    {
        printf("\n saisie de l'eleve numero %d\n",i);
        printf("nom de l'eleve = ");
        scanf("%s",&tab[i].nom);
        printf("\n date de naissance JJMMAA = ");
        scanf("%d",&tab[i].date);
    }

    printf("\n Entrez un numero ");
    scanf("%d",&i);
    printf("\n Eleve numero %d:",i);
    printf("\n nom = %s",tab[i].nom);
    printf("\n date de naissance = %d\n",tab[i].date);

    free(tab);
}
```



4.6 Pointeurs et Structures

Si **p** est un **pointeur** sur une **structure**, on peut accéder à un **membre** de la structure pointé par l'expression :

(*p).membre

L'usage de parenthèses est ici indispensable car l'opérateur d'indirection ***** à une priorité plus élevée que l'opérateur de membre de structure. Cette notation peut être simplifiée grâce à l'opérateur pointeur de membre de structure, noté **->**. L'expression précédente est strictement équivalente à :

p->membre

Ainsi, dans le programme précédent, on peut remplacer **tab[i].nom** et **tab[i].date** respectivement par **(tab + i)->nom** et **(tab + i)->date**.

4.7 Pointeurs et Fonctions

Voir chapitre sur les fonctions!

Chapitre 5: Les fonctions

5. Les Fonctions

Comme dans la plupart des langages, on peut en **C** découper un programme en plusieurs **fonctions**. Une seule de ces fonctions existe obligatoirement ; c'est la **fonction principale** appelée **main**.

Cette **fonction principale** peut, éventuellement, appeler une ou plusieurs fonctions secondaires.

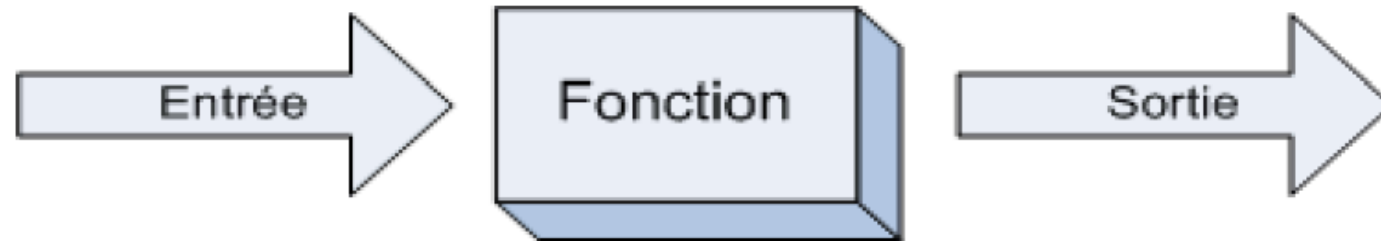
De même, chaque fonction secondaire peut appeler d'autres fonctions secondaires ou s'appeler elle-même (dans ce dernier cas, on dit que la fonction est **récursive**).

Une fonction exécute des actions et renvoie un résultat.

5. Les Fonctions

Une fonction est un morceau de code qui sert à faire quelque chose de précis. On dit qu'une fonction possède une entrée et une sortie.

Schématiquement :



Lorsqu'on appelle une fonction, il y a 3 étapes :

- 1. L'entrée:** on fait "rentrez" des informations dans la fonction (en lui donnant des informations avec lesquelles travailler)
- 2. Les calculs:** grâce aux informations qu'elle a reçues en entrée, la fonction travaille.
- 3. La sortie:** une fois qu'elle a fini ses calculs, la fonction renvoie un résultat. C'est ce qu'on appelle la sortie, ou encore le retour.

5. Les Fonctions

Concrètement, on peut imaginer par exemple une fonction appelée "triple" qui calcule le triple du nombre qu'on lui donne (en le multipliant par 3):



La fonction « tripe » multiplie le nombre en entrée par 3

Bien entendu, les fonctions seront en général plus compliquées.

Le but des fonctions est donc de **simplifier le code source**, pour ne pas avoir à **retaper** le même code plusieurs fois d'affilée.

5.1 Définition d'une fonction

La définition d'une **fonction** est la donnée du texte de son algorithme, qu'on appelle corps de la fonction. Elle est de la forme:

```
type nom-fonction (type-1 arg-1, ..., type-n arg-n)  
{  
    [déclarations de variables locales ]  
    liste d'instructions  
}
```

- La première ligne de cette définition est l'**en-tête** de la **fonction**.
- Dans cet en-tête, **type** désigne le type de la **fonction**, c'est-à-dire le type de la valeur qu'elle retourne. Contrairement à d'autres langages, il n'y a pas en C de notion de procédure ou de sous-programme. Une **fonction** qui ne renvoie pas de valeur est une fonction dont le type est spécifié par le mot-clef **void**. Les **arguments** de la **fonction** sont appelés **paramètres**. Les paramètres peuvent être de n'importe quel type. Leurs identificateurs n'ont d'importance qu'à l'intérieur de la fonction. Enfin, si la fonction ne possède pas de **paramètres**, on remplace la liste de **paramètres** par le mot-clef **void**.

5.1 Définition d'une fonction

- Le **corps** de la fonction débute éventuellement par des déclarations de variables, qui sont **locales** à cette fonction. Il se termine par l'instruction de retour à la fonction appelante, **return**, dont la syntaxe est ***return(expression);***
- La valeur de ***expression*** est la valeur que retourne la fonction. Son **type** doit être le même que celui qui a été spécifié dans l'**en-tête** de la fonction. Si la fonction ne retourne pas de valeur (**fonction de type void**), le corps de la fonction n'a pas de **return** (ou ***return ;***)
- Plusieurs instructions **return** peuvent apparaître dans une fonction. Le retour au programme appelant sera alors provoqué par le premier **return** rencontré lors de l'exécution.

5.1 Définition d'une fonction

Exemple : La fonction "triple«

Cette fonction reçoit un nombre entier de type **int** et elle renvoie un nombre entier aussi de type **int**. Cette fonction calcule donc le triple du nombre qu'on lui donne en paramètre:

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int triple(int nombre)
5  {
6      int resultat = 0;
7      // On multiplie le nombre qu'on nous a transmis par 3
8      resultat = 3 * nombre;
9      // Retourne la variable résultat qui vaut le triple de nombre
10     return resultat;
11 }
12
13 int main(int argc, char const *argv[])
14 {
15     /* code */
16     return 0;
17 }
```

5.2 Déclaration d'une fonction

Le C n'autorise pas les fonctions imbriquées. La définition d'une fonction secondaire doit donc être placée soit avant, soit après la fonction principale `main()`.

Toutefois, il est indispensable que le compilateur **connaisse** la fonction au moment où celle-ci est appelée.

Si une fonction est définie après son premier appel (en particulier si sa définition est placée après la fonction `main`), elle doit impérativement être déclarée au préalable.

Une fonction secondaire est déclarée par son prototype, qui donne le type de la fonction et celui de ses paramètres, sous la forme :

type nom-fonction(**type-1** ,..., **type-n**);

5.2 Déclaration d'une fonction

Les fonctions secondaires peuvent être déclarées indifféremment avant ou au début de la fonction main. Par exemple, on écrira:

```
1  #include <stdio.h>
2
3  //déclaration de la fonction
4  int puissance (int, int );
5
6  int puissance (int a, int n)
7  {
8      if (n == 0)
9          return(1);
10
11     return(a * puissance(a, n-1));
12 }
13
14 void main()
15 {
16     int a = 2, b = 5;
17     printf("%d\n", puissance(a,b));
18 }
```

Même si la déclaration est parfois facultative (par exemple quand les fonctions sont définies avant la fonction **main()** et dans le bon ordre), elle seule permet au compilateur de vérifier que le nombre et le **type** des paramètres utilisés dans la définition concordent bien avec le **prototype**.

5.2 Déclaration d'une fonction

De plus, la présence d'une déclaration permet au compilateur de mettre en place d'éventuelles **conversions des paramètres effectifs**, lorsque la fonction est appelée avec des **paramètres** dont les **types** ne correspondent pas aux types indiqués dans le **prototype**. Ainsi les fichiers d'extension **.h** de la **librairie standard** (fichiers headers) contiennent notamment les **prototypes** des fonctions de la librairie standard. Par exemple, on trouve dans le fichier **math.h** le **prototype** de la fonction **pow()** (élévation à la puissance) :

```
extern double pow(double , double );
```

La directive au préprocesseur **#include <math.h>** permet au préprocesseur d'inclure la déclaration de la fonction **pow()** dans le fichier source. Ainsi, si cette fonction est appelée avec des paramètres de type **int**, ces paramètres seront convertis en **double** lors de la compilation.

Par contre, en l'absence de directive au préprocesseur, le compilateur ne peut effectuer la conversion de **type**. Dans ce cas, l'appel à la fonction **pow()** avec des paramètres de **type int** peut produire un résultat faux !

5.3 Fonction avec plusieurs paramètres

La fonction "**triple**" contient un **paramètre**, mais il est possible de créer des fonctions prenant plusieurs **paramètres**.

Par exemple, une fonction addition qui additionne deux nombres a et b :

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int addition(int a, int b)
5  {
6      int s;
7      // On additionne a et b puis on met le résultat dans s
8      s = a + b;
9      // Retourne la variable s qui vaut la somme des deux nombres
10     return s;
11 }
12
13 int main(int argc, char const *argv[])
14 {
15     /* code */
16     return 0;
17 }
```

5.4 Fonction sans paramètre

Certaines fonctions ne prennent **aucun paramètre** en entrée. Ces fonctions feront généralement toujours la **même chose**. En effet, si elles n'ont pas de nombres sur lesquels travailler, ces fonctions serviront juste à effectuer certaines actions, comme afficher du texte à l'écran.

Imaginons une fonction "**affiche**" qui affiche juste bonjour à l'écran :

```
void affiche()  
{  
    printf("Bonjour");  
}
```

5.5 Appel d'une fonction

L'appel d'une fonction se fait par l'expression

nom-fonction(**para-1**, **para-2**, ..., **para-n**)

L'**ordre** et le **type** des **paramètres** effectifs de la fonction doivent concorder avec ceux donnés dans l'**en-tête** de la fonction. Les paramètres effectifs peuvent être des **expressions**.

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int addition(int a, int b) {
5      int s;
6      s = a + b;
7      return s;
8  }
9
10 int triple(int nombre) {
11     int resultat = 0;
12     resultat = 3 * nombre;
13     return resultat;
14 }
15
16 int main(int argc, char const *argv[]){
17     //appel de la fonction triple
18     printf("Le triple du nombre est %d\n", triple(4));
19     //appel de la fonction addition
20     printf("La somme des deux nombre est %d\n", addition(10, 20));
21     return 0;
22 }
```

```
F:\UAM\fonction>
F:\UAM\fonction>gcc addition.c -o application

F:\UAM\fonction>application.exe
Le triple du nombre est 12
La somme des deux nombre est 30

F:\UAM\fonction>
```



5.5 Portée des variables

Les **variables** manipulées dans un programme **C** ne sont pas toutes traitées de la même manière. En particulier, elles n'ont pas toutes la même **durée de vie**. On distingue deux catégories de variables:

1. Les variables permanentes (ou statiques)

Une **variable permanente** occupe un emplacement en mémoire qui reste le **même** durant toute l'exécution du programme. Cet emplacement est alloué une fois pour toutes lors de la compilation.

La partie de la mémoire contenant les **variables permanentes** est appelée **segment de données**.

Par défaut, les variables permanentes sont **initialisées à zéro** par le compilateur. Elles sont caractérisées par le mot-clef ***static***.

5.6 Portée des variables

2. Les variables temporaires

Les **variables temporaires** se voient allouer un emplacement en mémoire de façon **dynamique** lors de l'exécution du programme. Elles **ne sont pas initialisées par défaut**. Leur emplacement en mémoire est libéré par exemple à la fin de l'exécution d'une fonction secondaire.

Par défaut, les **variables temporaires** sont situées dans la partie de la mémoire appelée ***segment de pile***.

La durée de vie des variables est liée à leur portée, c'est-à-dire à la portion du programme dans laquelle elles sont définies.

5.6 Portée des variables: Variables globales

On appelle **variable globale** une variable **déclarée en dehors de toute fonction**.

Une variable globale est connue du compilateur dans toute la portion de code qui suit sa déclaration.

Les variables globales sont systématiquement permanentes.

Dans le programme suivant, **n** est une variable globale :

5.6 Portée des variables: Variables globales

Dans le programme suivant, **n** est une variable globale :

```
1  #include <stdio.h>
2
3  int n; // variable globale
4
5  void fonction()
6  {
7      n++;
8      printf("appel numero %d\n",n);
9  }
10
11 void main() {
12     int i;
13
14     for (i = 0; i < 5; i++)
15         fonction();
16 }
```

```
F:\UAM\fonction>
F:\UAM\fonction>gcc var_globale.c -o var_globale

F:\UAM\fonction>var_globale.exe
appel numero 1
appel numero 2
appel numero 3
appel numero 4
appel numero 5

F:\UAM\fonction>
```

La variable **n** est **initialisée à zéro** par le compilateur et il s'agit d'une ***variable permanente***.

5.6 Portée des variables: Variables locales

On appelle **variable locale** une variable **déclarée à l'intérieur d'une fonction** (ou d'un bloc d'instructions) du programme.

Par défaut, les **variables locales** sont **temporaires**.

Quand une fonction est appelée, elle place ses **variables locales** dans la pile. A la sortie de la fonction, les variables locales sont dépilées et donc perdues.

5.6 Portée des variables: Variables locales

Les **variables locales** n'ont en particulier aucun lien avec des **variables globales** de **même nom**. Par exemple:

```
1  #include <stdio.h>
2
3  int n = 10; // variable globale
4
5  void fonction()
6  {
7      int n = 0;
8      n++;
9      printf("appel numero %d\n",n);
10 }
11
12 void main() {
13     int i;
14
15     for (i = 0; i < 5; i++)
16         fonction();
17 }
```

```
F:\UAM\fonction>
F:\UAM\fonction>gcc var_locale.c -o var_locale

F:\UAM\fonction>var_locale.exe
appel numero 1
appel numero 1
appel numero 1
appel numero 1
appel numero 1

F:\UAM\fonction>
```

Les **variables locales** à une fonction ont une **durée de vie limitée** à une seule exécution de cette fonction. Leurs valeurs ne sont pas conservées d'un appel au suivant.

5.6 Portée des variables: Variables locales

Il est toutefois possible de créer une variable locale de classe statique en faisant précéder sa déclaration du mot-clef **static** :

static type nom-de-variable;

Une telle variable reste **locale** à la fonction dans laquelle elle est déclarée, mais sa valeur est conservée d'un appel au suivant. Elle est également **initialisée à zéro** à la compilation.

Par exemple, dans le programme suivant, **n** est une **variable locale** à la fonction secondaire nommée **fonction**, mais de classe **statique**.

5.6 Portée des variables: Variables locales

```
1  #include <stdio.h>
2
3  int n = 10; // variable globale
4
5  void fonction()
6  {
7      static int n; //pas d'initialisation
8      n++;
9      printf("appel numero %d\n",n);
10
11     return;
12 }
13
14 void main() {
15     int i;
16
17     for (i = 0; i < 5; i++)
18         fonction();
19 }
```

Ce programme affiche

```
F:\UAM\fonction>
F:\UAM\fonction>gcc var_locale.c -o var_locale

F:\UAM\fonction>var_locale.exe
appel numero 1
appel numero 1
appel numero 1
appel numero 1
appel numero 1

F:\UAM\fonction>
```

On voit que la **variable locale** **n** est de classe **statique** (elle est initialisée à zéro, et sa valeur est conservée d'un appel à l'autre de la fonction).

Par contre, il s'agit bien d'une **variable locale**, qui n'a aucun lien avec la **variable globale** du même nom.

5.7 Transmission des paramètres d'une fonction:

5.7.1 Passage de paramètres par valeur

Les **paramètres** d'une fonction sont traités de la même manière que les **variables locales** de classe automatique : lors de l'appel de la fonction, les **paramètres effectifs** sont copiés dans le segment de pile. La fonction travaille alors uniquement sur cette copie. Cette copie disparaît lors du retour au programme appelant. Cela implique en particulier que, si la fonction **modifie la valeur** d'un de ses paramètres, seule la copie sera modifiée ; la variable du programme appelant, elle, ne sera pas modifiée.

On dit que les paramètres d'une fonction sont transmis par valeurs.

5.7 Transmission des paramètres d'une fonction:

5.7.1 Passage de paramètres par valeur

Exemple:

```
1  #include <stdio.h>
2
3  void echange (int, int );
4
5  void echange (int a, int b)
6  {
7      int t;
8      printf("debut fonction :\n a = %d \t b = %d\n",a,b);
9      t = a;
10     a = b;
11     b = t;
12     printf("fin fonction :\n a = %d \t b = %d\n",a,b);
13     return;
14 }
15
16 void main()
17 {
18     int a = 2, b = 5;
19     printf("debut programme principal :\n a = %d \t b = %d\n",a,b);
20     echange(a,b);
21     printf("fin programme principal :\n a = %d \t b = %d\n",a,b);
22 }
```

```
F:\UAM\fonction>
F:\UAM\fonction>gcc passage_par_valeur.c -o passage_par_valeur

F:\UAM\fonction>passage_par_valeur.exe
debut programme principal :
a = 2    b = 5
debut fonction :
a = 2    b = 5
fin fonction :
a = 5    b = 2
fin programme principal :
a = 2    b = 5

F:\UAM\fonction>
```

5.7 Transmission des paramètres d'une fonction:

5.7.1 Passage de paramètres par adresse(référence)

Pour qu'une fonction **modifie la valeur** d'un de ses arguments, il faut qu'elle ait pour paramètre l'adresse de cet objet et non sa **valeur**.

On dit, dans ce cas, que les paramètres de la fonction sont transmis par adresse.

5.7 Transmission des paramètres d'une fonction:

5.7.1 Passage de paramètres par adresse(référence)

Par exemple, pour échanger les valeurs de deux variables, il faut écrire :

```
1  #include <stdio.h>
2
3  void exchange (int *, int *);
4
5  void exchange (int *adr_a, int *adr_b) {
6      int t;
7      t = *adr_a;
8      *adr_a = *adr_b;
9      *adr_b = t;
10     return;
11 }
12
13 void main(){
14     int a = 2, b = 5;
15     printf("debut programme principal :\n a = %d \t b = %d\n",a,b);
16     exchange(&a,&b);
17     printf("fin programme principal :\n a = %d \t b = %d\n",a,b);
18 }
```

```
F:\UAM\fonction>
F:\UAM\fonction>gcc passage_par_adresse.c -o passage_par_adresse

F:\UAM\fonction>passage_par_adresse.exe
debut programme principal :
a = 2   b = 5
fin programme principal :
a = 5   b = 2

F:\UAM\fonction>
```


5.7 Transmission des paramètres d'une fonction:

5.7.1 Passage de paramètres par adresse(référence)

Rappelons qu'un **tableau est un pointeur** (sur le premier élément du tableau).

Lorsqu'un **tableau** est transmis comme **paramètre** à une fonction secondaire, ses éléments sont donc **modifiés** par la fonction.

Par exemple, le programme ci-contre initialise les éléments du tableau **tab**:

```
1  #include <stdio.h>
2
3  void init (int *, int );
4
5  void init (int *tab, int n)
6  {
7      int i;
8      for (i = 0; i < n; i++)
9          tab[i] = i;
10     return;
11 }
12
13 main()
14 {
15     int i, n = 5;
16     int *tab;
17     tab = (int*)malloc(n * sizeof(int));
18     init(tab,n);
19 }
```

5.8 La fonction main

- La fonction principale **main** est une fonction comme les autres. Nous avons jusqu'à présent considéré qu'elle était de **type void**, ce qui est toléré par le compilateur.
- Toutefois l'écriture **main()** provoque un **message d'avertissement** lors de la compilation.
- En fait, la fonction **main** est de **type int**. Elle doit retourner un entier dont la valeur est transmise à l'environnement d'exécution. Cet entier indique si le programme s'est ou non déroulé **sans erreur**. La valeur de retour **0** correspond à une **terminaison correcte**, toute valeur de retour **non nulle** correspond à une **terminaison sur une erreur**.
- On peut utiliser comme valeur de retour les deux constantes symboliques **EXIT_SUCCESS** (égale à **0**) et **EXIT_FAILURE** (égale à **1**) définies dans **stdlib.h**.

5.8 La fonction main

- L'instruction ***return(statut);*** dans la fonction **main**, où **statut** est un entier spécifiant le type de terminaison du programme, peut être remplacée par un appel à la fonction **exit** de la librairie standard (**stdlib.h**).
- La fonction **exit**, de prototype ***void exit(int statut);*** provoque une terminaison normale du programme en notifiant un **succès** ou un **échec** selon la **valeur** de l'entier **statut**.

Lorsqu'elle est utilisée sans arguments, la fonction **main** a donc pour prototype ***int main(void);***

On s'attachera désormais dans les programmes à respecter ce prototype et à spécifier les valeurs de retour de main.

5.8 La fonction main

- La fonction **main** peut également posséder des **paramètres formels**.
- En effet, un programme C peut recevoir une **liste d'arguments au lancement de son exécution**. La ligne de commande qui sert à lancer le programme est, dans ce cas, composée du nom du **fichier exécutable** suivi par des **paramètres**. La fonction main reçoit tous ces éléments de la part de l'interpréteur de commandes.
- En fait, la fonction **main** possède **deux paramètres formels**, appelés par convention **argc** (argument count) et **argv** (argument vector).

argc est une variable de **type int** dont la valeur est égale **au nombre de mots** composant la ligne de commande (y compris le nom de l'exécutable). Elle est donc égale au **nombre de paramètres effectifs de la fonction + 1**.

argv est un **tableau de chaînes de caractères** correspondants chacune à un **mot** de la ligne de commande. Le premier élément **argv[0]** contient donc le **nom de la commande** (du fichier exécutable, **gcc**), le second **argv[1]** contient le **premier paramètre**.

5.8 La fonction main

Le second **prototype** valide de la fonction **main** est donc

int main (int argc, char *argv[]);

Ainsi, le programme suivant calcule le produit de deux entiers, entrés en arguments de l'exécutable :

```
F:\UAM\fonction>gcc fonction_main.c -o fonction_main
F:\UAM\fonction>fonction_main 5 4

Le produit de 5 par 4 vaut : 20

F:\UAM\fonction>
F:\UAM\fonction>fonction_main 5

Erreur : nombre invalide d'arguments
Usage: fonction_main int int
F:\UAM\fonction>
```

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int main(int argc, char *argv[]) {
5      int a, b;
6
7      if (argc != 3) {
8          printf("\nErreur : nombre invalide d'arguments");
9          printf("\nUsage: %s int int\n",argv[0]);
10         return(EXIT_FAILURE);
11     }
12
13     a = atoi(argv[1]);
14     b = atoi(argv[2]);
15     printf("\nLe produit de %d par %d vaut : %d\n", a, b, a * b);
16
17     return(EXIT_SUCCESS);
18 }
```

Ici, **argv** sera un tableau de 3 chaînes de caractères **argv[0]**, **argv[1]** et **argv[2]** qui, dans notre exemple, valent respectivement « **fonction_main** », « **5** » et « **4** ».

Enfin, la fonction de la librairie standard **atoi()**, déclarée dans **stdlib.h**, prend en argument une chaîne de caractères et retourne l'entier dont elle est l'écriture décimale.

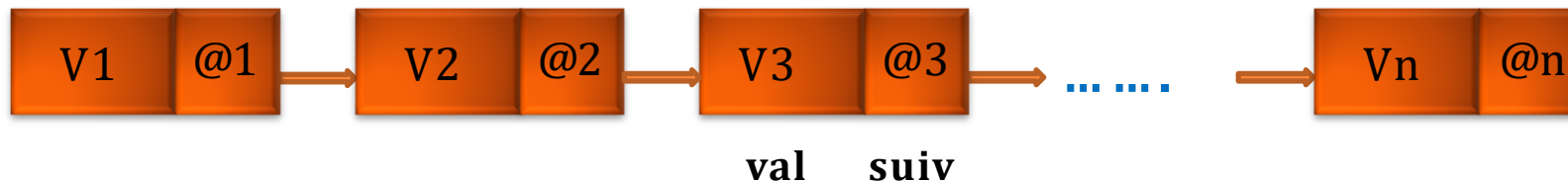
Chapitre 6: Les listes chaînées

PLAN

- ❑ Définitions
- ❑ Modèle des LLC (M_{LLC})
- ❑ Algorithmes sur les listes
- ❑ Listes particulières
- ❑ Conclusion

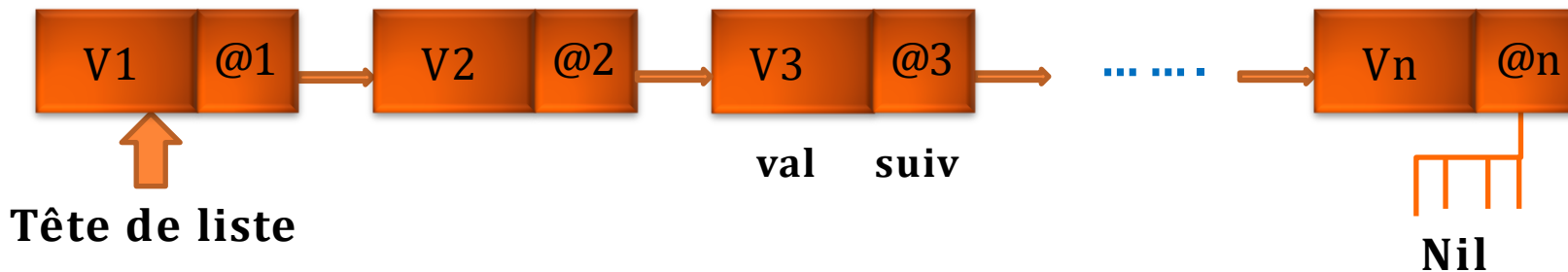
DÉFINITIONS

- ❖ Une Liste Linéaire Chaînée (LLC) est un ensemble de **maillons** reliés entre eux.



- ❖ Un maillon est une entité renfermant de l'information et ayant un pointeur sur le maillon qui la suit. C'est toujours une structure (enregistrement) avec deux champs :
 - un champ **Valeur** : contenant l'information
 - un champ **Adresse** : donnant l'adresse du prochain maillon.

DÉFINITIONS

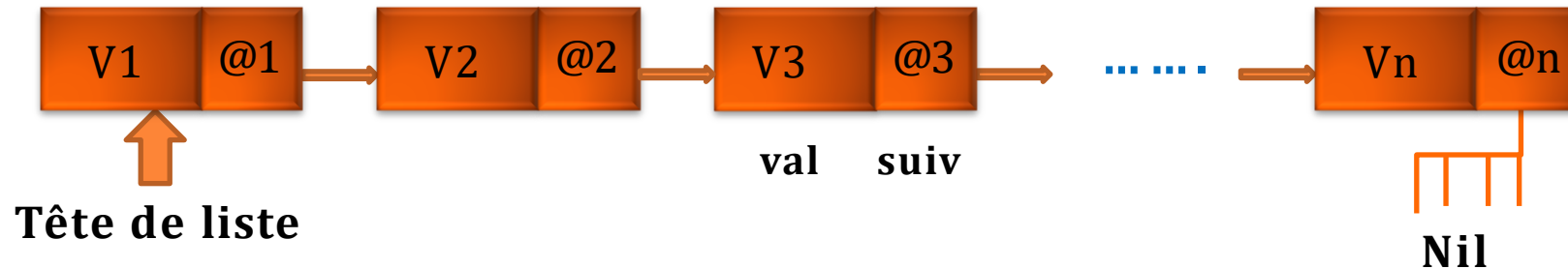


❖ Une LLC est caractérisée par :

- L'adresse de son premier élément (la tête de liste). Elle doit toujours être sauvegardée dans une variable pour pouvoir manipuler la liste.
- NIL constitue l'adresse qui ne pointe aucun maillon (indiquant par convention la fin de la liste).
- Si la liste est vide (ne contient aucun maillon), la tête doit alors être positionnée à NIL.

Tête

DÉFINITIONS



Langage Algorithmique	Langage C
<pre><i>type Maillon = Structure</i> <i>val : typeqq</i> <i>suiv: * Maillon</i> <i>fin</i></pre>	<pre><i>struct Maillon {</i> <i>typeqq val ;</i> <i>struct Maillon*suiv ;</i> <i>};</i></pre>
<pre><i>Var Tête: *Maillon</i></pre>	<pre><i>struct Maillon* Tête;</i></pre>

MODÈLE DE LLC (M_{LLC})

- ❖ On définit un ensemble d'opérations que l'on va utiliser pour écrire des algorithmes sur les listes. Cet ensemble d'opérations s'appelle le **Modèle des Listes Linéaires Chaînées (M_{LLC})** :

Modules	Rôle
Allouer(P)	C'est une procédure qui alloue (dynamiquement) un nouveau maillon et affecte son adresse dans le pointeur P.
Libérer(P)	C'est une procédure qui détruit le maillon pointé par P.
Valeur(P)	C'est une fonction qui retourne le contenu du champ « val » du maillon pointé par P (<i>retourner (*P.val)</i>)
Suivant(P)	C'est une fonction qui retourne le contenu du champ « suiv » du maillon pointé par P (<i>retourner (*P.adr)</i>)
Aff_Adr(P, Q)	C'est une procédure qui affecte le pointeur Q dans le champ « suiv » du maillon pointé par P (<i>*P.suiv ← Q</i>)
Aff_Val(P,v)	C'est une procédure qui affecte la valeur v dans le champ « val » du maillon pointé par P (<i>*P.val ← v</i>)

MODÈLE DE LLC (M_{LLC})

- ❖ Voici l'équivalence en langage C, des fonctions et des procédures du modèle :

Modules	Implémentation en langage C
Allouer(P)	<pre>struct Maillon *Allouer () { return ((struct Maillon *) malloc(sizeof(struct Maillon))); }</pre>
Libérer(P)	<pre>Void Libérer (struct Maillon *P) {free (p);}</pre>
Valeur(P)	<pre>typeqq Valeur(struct Maillon *P) { return(P->val) ; //équivalent à *P.val}</pre>
Suivant(P)	<pre>struct Maillon *Suivant(struct Maillon *P) { return(P->suiv); //équivalent à *P.suiv}}</pre>
Aff_Adr(P, Q)	<pre>void Aff_Adr(struct Maillon *P, struct Maillon *Q) { P->suiv = Q; //équivalent à *P.suiv=Q}</pre>
Aff_Val(P,v)	<pre>void Aff_Val(struct Maillon *P, int v) { P->val =v; //équivalent à *P.val = v}</pre>

ALGORITHMES SUR LES LLCs

❖ **Parcours :**

- ***Accès par valeur*** : il s'agit de rechercher (séquentiellement à partir de la tête) une valeur v dans la liste.
- ***Accès par position*** : il s'agit de rechercher (séquentiellement à partir de la tête) le maillon (son adresse) qui se trouve à la position donnée. La position est le numéro d'ordre du maillon dans la liste (entier).

ALGORITHMES SUR LES LLCs

❖ Mises à jour :

- ***Construction d'une liste*** à partir de n valeurs données.
- ***Insertion d'une valeur (v) à une position donnée (i)*** : allouer un nouveau maillon contenant v et l'insérer dans la liste de telle sorte qu'il se retrouve à la i ème position.
- ***Insertion d'une valeur (v) dans une liste ordonnée*** : allouer un nouveau maillon contenant v et l'insérer dans la liste de telle sorte que la liste reste ordonnée après l'insertion.

ALGORITHMES SUR LES LLCs

❖ Mises à jour :

- *Suppression du maillon se trouvant à une position donnée*
(i) : rechercher par position le maillon i et le libérer.
- **Suppression d'une valeur v dans la liste** : rechercher par valeur et supprimer le maillon trouvé en mettant à jour le précédent (s'il existe). Si la liste contient plusieurs occurrences de la même valeur, on pourra les supprimer en faisant un seul parcours de la liste.
- **Destruction de tous les maillons d'une liste.**

ALGORITHMES SUR LES LLCs

- ❖ **Tri sur les LLC** : Les mêmes algorithmes de tri utilisable pour les tableaux, en prenant en compte que l'accès par position coûte beaucoup plus cher que dans un tableau.
- ❖ **Algorithmes sur plusieurs LLC** :
 - *Fusion (ou Interclassement) de deux listes ordonnées* : à partir de 2 listes ordonnées, construire une liste ordonnée contenant tous leurs éléments (sans allouer de nouveaux maillons).
 - *Eclatement d'une liste en 2 listes distinctes* : à partir d'une liste L et d'un critère (un prédicat) donné, on construit deux listes, l'une contenant toutes les valeurs de L vérifiant le critère, l'autre, celles qui ne le vérifient pas.

ALGORITHMES SUR LES LLCs

❖ Exercice 1:

Développer les algorithmes suivants qui permettent de :

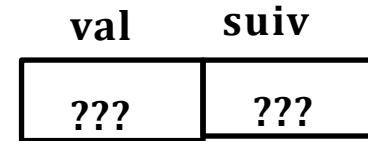
1. Créer une LLC à partir d'une suite de n valeurs (entières) lues,
2. Imprimer la liste ainsi créée.
3. Détruire la liste ainsi créée

ALGORITHMES SUR LES LLCs

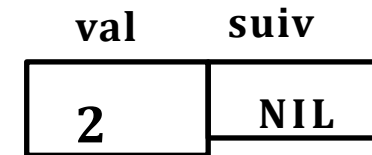
CRÉATION UNE LLC À PARTIR D'UNE SUITE DE N VALEURS LUES

❖ Pour créer cette liste:

1. Allouer un nouveau maillon



2. Remplir le champ valeur de ce maillon



3. Insérer dans la liste. Dans ce cas il y a deux cas:

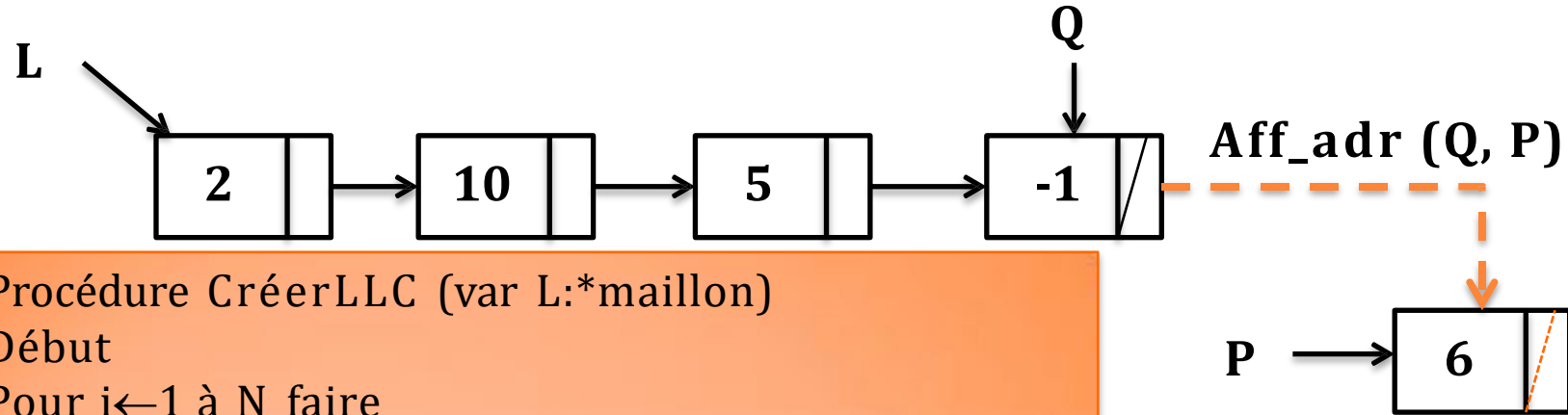
- a. Insertion à la fin

- b. Insertion au début et la liste crée est à l'envers

ALGORITHMES SUR LES LLCs

CRÉATION UNE LLC À PARTIR D'UNE SUITE DE N VALEURS LUES

❖ **Cas 1: Insertion à la fin.** Soient $N = 5$ et les valeurs suivantes: 2, 10, 5, -1, 6



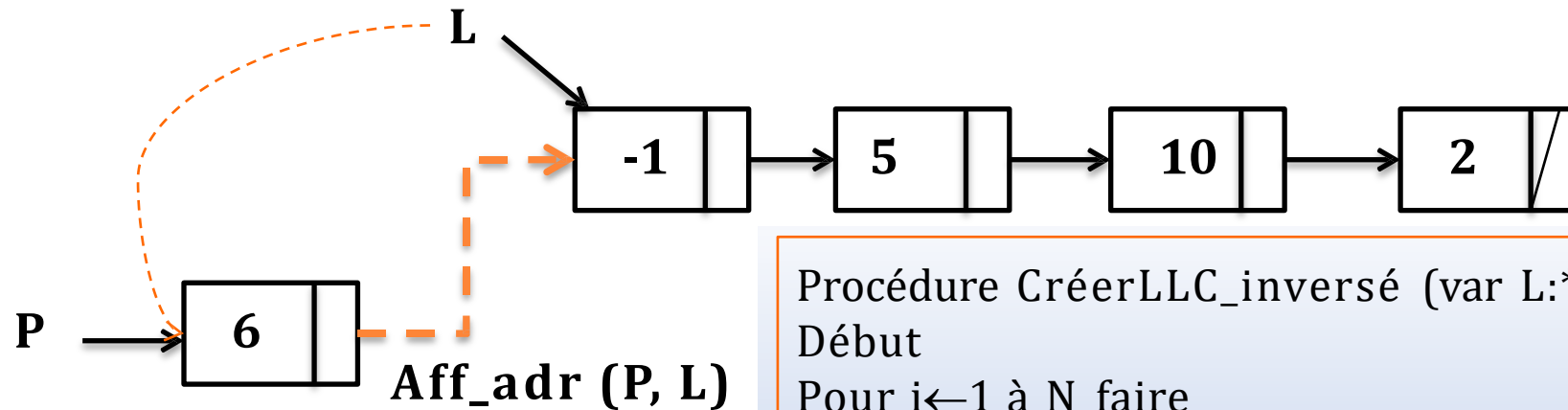
```
Procédure CréerLLC (var L:*maillon)
Début
Pour i ← 1 à N faire
  DP
    lire (x)
    Allouer (P)
    aff_val(P, x)
    aff_adr(P, null)
  Si (L=null) alors //la liste est vide
    L ← P //mettre à jour l'entête
  Sinon
    aff_adr(Q, P)
  Q ← P //sauvegarder l'@ du dernier maillon
FP
Fin
```

ALGORITHMES SUR LES LLCs

CRÉATION UNE LLC À PARTIR D'UNE SUITE DE N VALEURS LUES

❖ Soient $N = 5$ et les valeurs suivantes: **2, 10, 5, -1, 6**

❖ **Cas 1: Insertion au début**



Procédure CréerLLC_inversé (var L:*maillon)

Début

Pour $i \leftarrow 1$ à N faire

DP

lire (x)

Allouer (P)

aff_val(P, x)

aff_adr(P, null)

aff_adr(P, L)

$L \leftarrow P$ //mettre à jour la tête de la liste

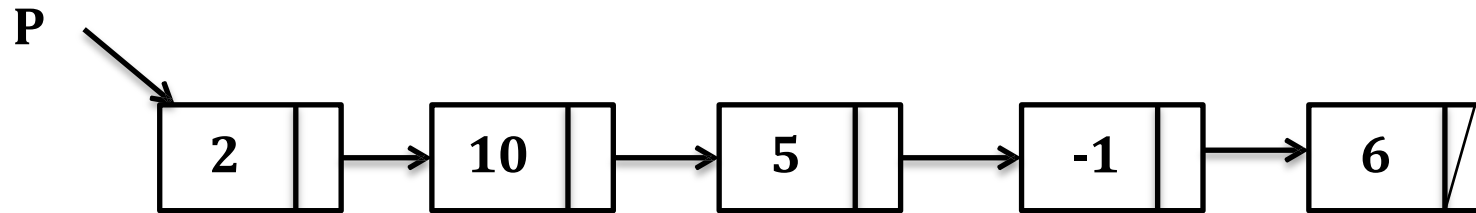
FP

Fin

ALGORITHMES SUR LES LLCs

IMPRESSION D'UNE LLC

❖ Soit la liste suivante:



Procédure Imprimer_LLC (P:*maillon)

Début

Tant que (P≠Null) faire //la liste n'est pas vide

DTQ

 écrire (valeur (P))

 P ←suivant (P)

FTQ

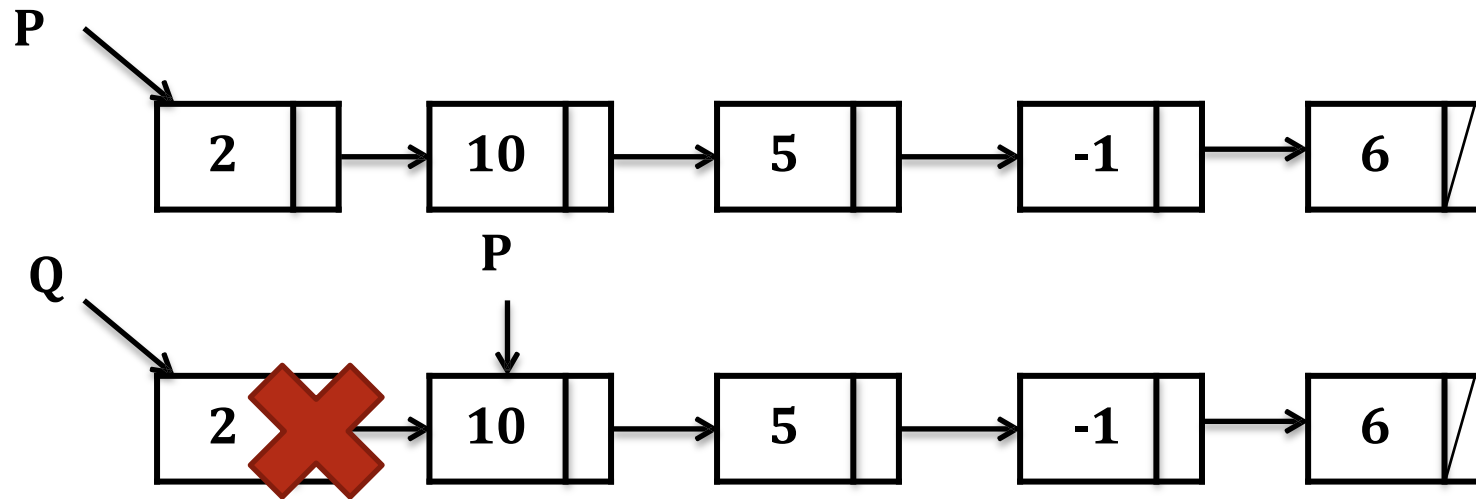
Fin

❖ La liste imprimé est 2, 10, 5, -1 et 6

ALGORITHMES SUR LES LLCs

DESTRUCTION D'UNE LLC

❖ Soit la liste suivante:



Procédure Détruire (Var P:*maillon)

Début

Tant que (P≠Null) faire //la liste n'est pas vide

DTQ

Q ← P

P ← suivant (P)

libérer (Q)

FTQ

Fin

ALGORITHMES SUR LES LLCs

❖ Exercice 2:

Soit L une liste d'entiers non ordonnée. Développer les modules suivants:

1. Rechercher une valeur « v » dans la liste
2. Rechercher la valeur du maillon qui se trouve à la position K
3. Insérer une valeur « v »:
 - a. Au début de la liste
 - b. À la fin de la liste
 - c. À une position donnée

ALGORITHMES SUR LES LLCs

❖ Exercice 2:

Soit L une liste d'entier non ordonnée. Développer les modules suivants:

4. Supprimer un maillon qui se trouve:

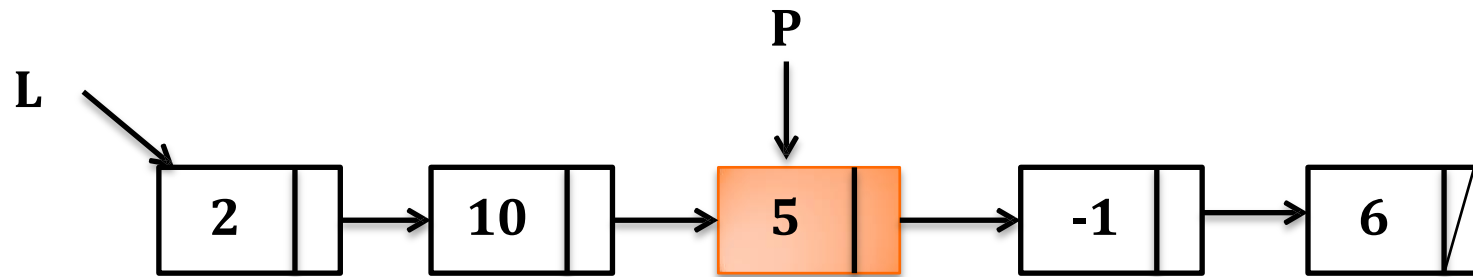
- a. Au début de la liste
- b. À la fin de la liste
- c. À une position donnée

5. Supprimer une valeur v dans la liste.

ALGORITHMES SUR LES LLCs

RECHERCHE DANS UNE LLC

- ❖ Rechercher une valeur « $v=5$ » dans la liste suivante:



- ❖ Si $v=1$ alors

la fonction retourne null.

```
Fonction Rech (L:*maillon, v:entier): *maillon
Début
  P ← L
  Tant que (P≠Null) faire //la liste n'est pas vide
  DTQ
    Si valeur (P) = v alors
      retourner (P)
    Sinon
      P ←suivant (P)
  FTQ
  Retourner (Null)
Fin
```

ALGORITHMES SUR LES LLCs

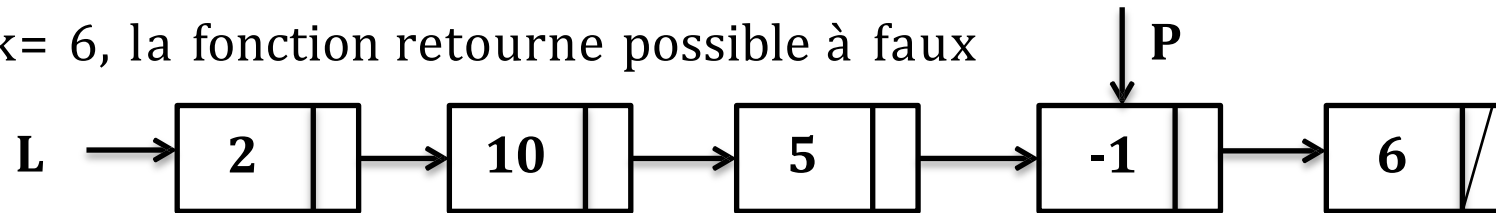
RECHERCHE DANS UNE LLC

- ❖ Pour rechercher la valeur du maillon qui se trouve à la position K , deux cas se présentent:
 - Cas 1: $K > \text{longueur}(\text{Liste})$, donc le maillon de la position K n'existe même pas.
 - Cas 2: $1 \leq K \leq \text{longueur}(\text{Liste})$, donc il est possible d'accéder au maillon de la position K

ALGORITHMES SUR LES LLCs

RECHERCHE DANS UNE LLC

- ❖ Pour rechercher la valeur du maillon qui se trouve à la position $k = 4$, la fonction retourne la valeur -1 (possible est à vrai).
- ❖ Si $k = 6$, la fonction retourne possible à faux



Procédure Rech_pos(L: *maillon, K: entier, var v: *entier, var possible: *booléen)

Debut

$P \leftarrow L$

Si $K > \text{longueur (liste)}$ alors

 possible $\leftarrow$ faux

Sinon

Dsinon

 *possible $\leftarrow$ vrai Pour

 i $\leftarrow 1$ à K faire

 P $\leftarrow$ suivant (P)

 *v $\leftarrow$ valeur (P)

Fsinon

Fin

ALGORITHMES SUR LES LLCs

RECHERCHE DANS UNE LLC

- ❖ Rechercher la valeur du maillon qui se trouve à la position k

2 ème méthode (sans l'utilisation de la fonction longueur)

Procédure Rech_pos(L: *maillon, K: entier, var v: entier, var possible: booléen)

Debut

P $\leftarrow$ L

i $\leftarrow$ 0

Tant que (i < K) et (P $\neq$ Null) faire //accéder au K ème maillon

DTQ

 P $\leftarrow$ suivant (P)

 i++

FTQ

Si (P = Null) alors //le cas où K > longueur (liste)

 possible $\leftarrow$ faux

Sinon

Dsinon

 possible $\leftarrow$ vrai

 v $\leftarrow$ valeur (P)

Fsinon

Fin

ALGORITHMES SUR LES LLCs

INSERTION DANS UNE LLC

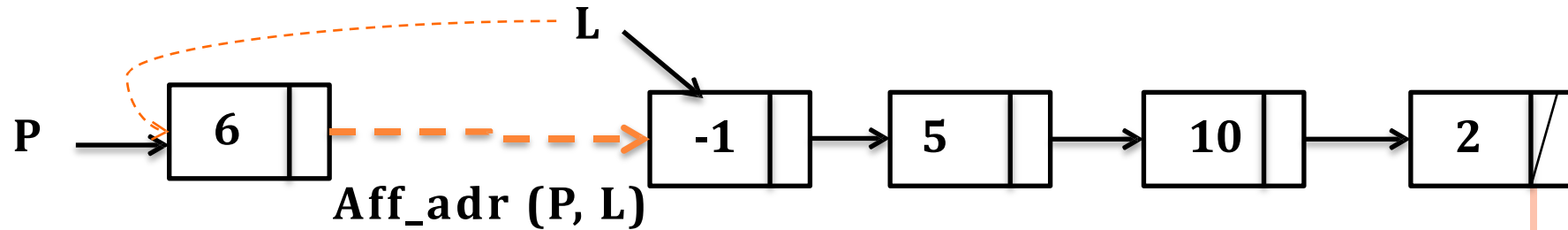
❖ Pour ajouter un élément dans la liste il y a plusieurs situations :

1. Insertion au début de la liste
2. Insertion à la fin de la liste
1. Insertion à une position donnée

ALGORITHMES SUR LES LLCs

INSERTION DANS UNE LLC

❖ Cas 1: Insertion au début



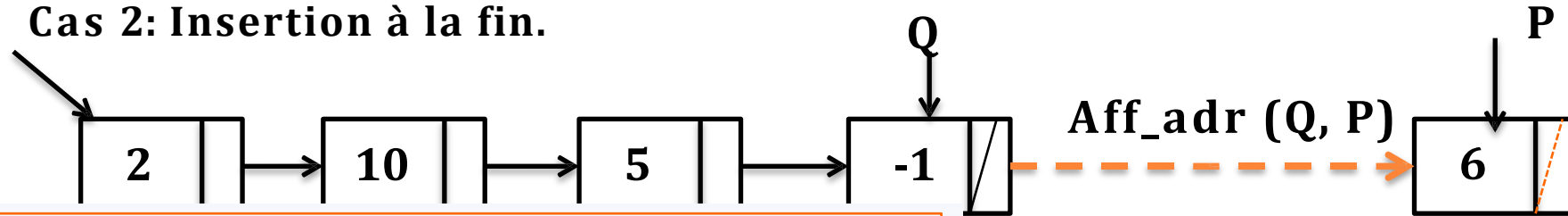
```
Procédure Insérer_Deb(Var L:*maillon, v:entier)
Début
Allouer (P)
aff_val(P, v)
aff_adr(P, null)
aff_adr(Q, L)
L ← P //mettre à jour la tête de la liste
Fin
```

```
Procédure CréerLLC_inversé (var L:*maillon)
Début
Pour i ← 1 à N faire
    lire (v)
    Insérer_Deb( L, v)
Fin
```

ALGORITHMES SUR LES LLCs

INSERTION DANS UNE LLC

L ♦ Cas 2: Insertion à la fin.



Procédure Insérer_Fin (var L:*maillon, v:entier)

Début

Allouer (P)

aff_val(P, v)

aff_adr(P, null)

Si (L=null) alors //la liste est vide

 L ← P //mettre à jour l'entête

Sinon

 Dsinon

 Q ← L

 Tant que (suivant (Q) ≠ Null) faire
 //accéder au dernier maillon

 Q ← suivant (Q)

 aff_adr (Q, P)

 Fsinon

Fin

Procédure CréerLLC (var L:*maillon)

Début

Pour i ← 1 à N faire

 DP

 lire (v)

 Insérer_Fin(L, v)

 FP

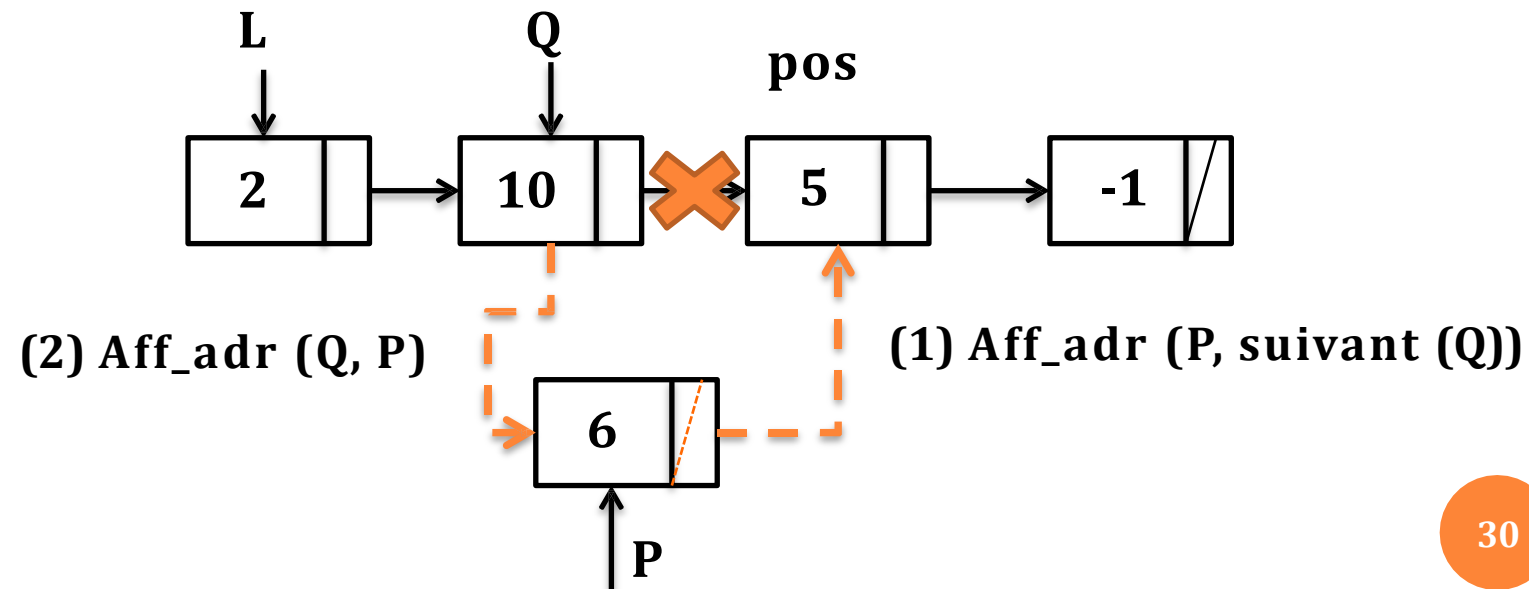
Fin

ALGORITHMES SUR LES LLCs

INSERTION DANS UNE LLC

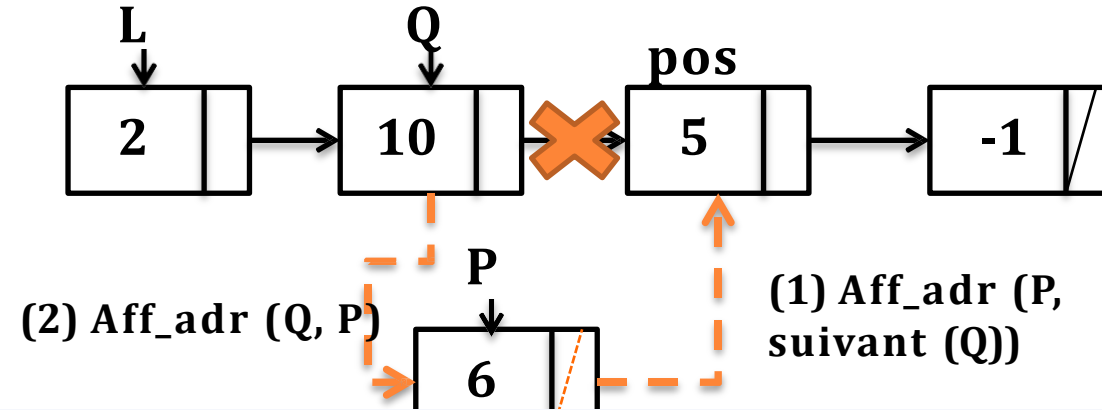
❖ Cas 2: Insertion à une position donné « pos ».

- On suppose que pos est entre 1 et longueur(Liste)
- On doit accéder d'abord au maillon de position « pos -1 », ensuite refaire le chaînage comme suit;



ALGORITHMES SUR LES LLCs

INSERTION DANS UNE LLC



```
Procédure Insérer_Pos (var L:*maillon, pos: entier, v:entier)
Début
Allouer (P)
aff_val(P, v)
Si (k=1) alors
    aff_adr(P, L)
    L ← P //mettre à jour l'entête
Sinon
    Q ← L
    Pour i←1 à pos-1 faire //accéder au dernier maillon
        Q ←suivant (Q)
    Aff_adr (P, suivant (Q))
    aff_adr (Q, P)
Fin
```

ALGORITHMES SUR LES LLCs

INSERTION DANS UNE LLC

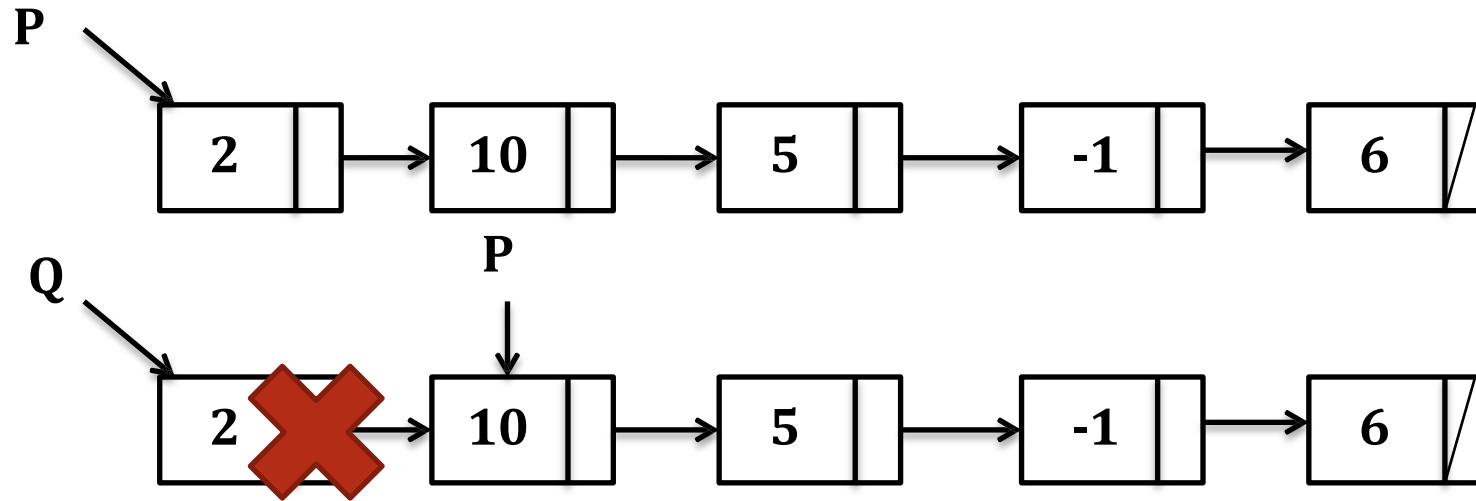
❖ Pour supprimer un élément dans la liste il y a plusieurs situations :

1. Suppression au début de la liste
2. Suppression à la fin de la liste
1. Suppression à une position donnée

ALGORITHMES SUR LES LLCs

SUPPRESSION DANS UNE LLC

❖ Cas 1: Suppression au début



Procédure Supprimer_Deb (Var P:*maillon)

Début

Si $P \neq \text{Null}$ alors

$Q \leftarrow P$

$P \leftarrow \text{suivant}(P)$

 libérer (Q)

Fin

Procédure Détruire (Var P:*maillon)

Début

Tant que ($P \neq \text{Null}$) faire

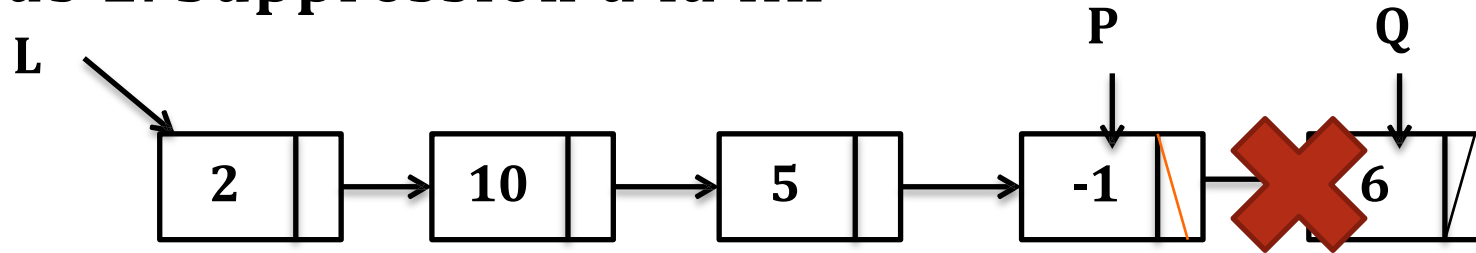
 Supprimer_Deb (P)

Fin

ALGORITHMES SUR LES LLCs

SUPPRESSION DANS UNE LLC

❖ Cas 2: Suppression à la fin



Procédure Supprimer_Fin (Var L:*maillon)

Début

Si L ≠ Null alors

 Si (suivant (L) = null) alors //le cas où la liste contient un seul maillon
 libérer (L)
 L ← null

 Sinon

 P ← L; Q ← L

 Tant que (suivant(Q) ≠ Null) faire

 P ← Q

 Q ← suivant (Q)

 Aff_adr(P, null)

 libérer (Q)

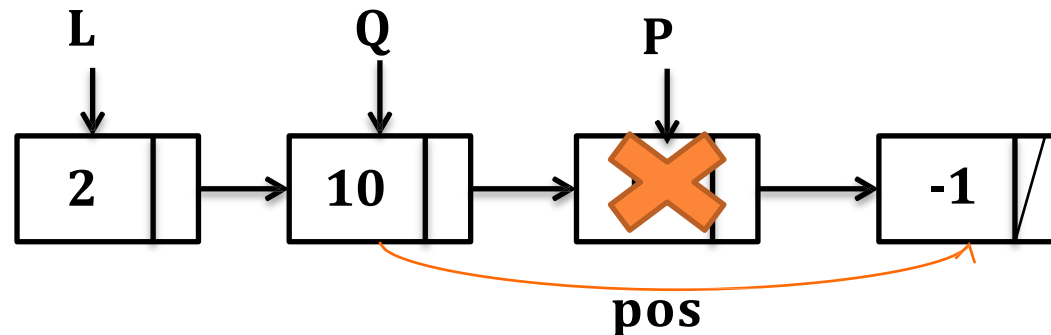
Fin

ALGORITHMES SUR LES LLCs

SUPPRESSION DANS UNE LLC

❖ Cas 3: suppression à une position donné « pos ».

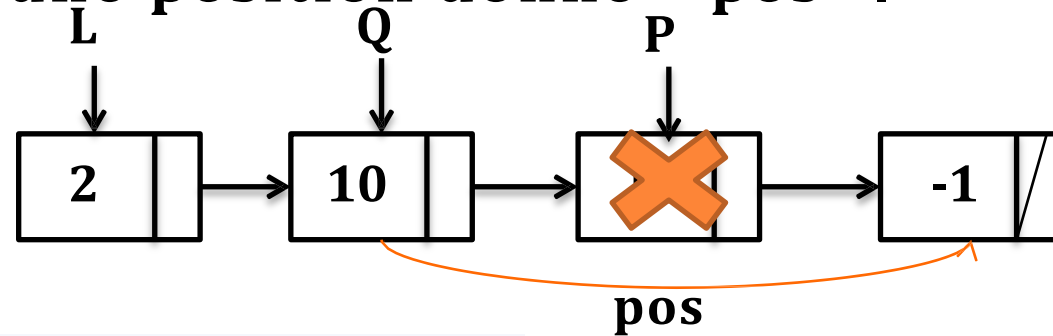
- On suppose que pos est entre 1 et longueur(Liste)
- Deux situations se présentent:
 - ❑ $K = 1$, suppression au début
 - ❑ $K \neq 1$, On doit accéder d'abord au maillon de position « pos » en sauvegardant l'adresse du maillon « pos-1 », ensuite refaire le chaînage comme suit;



ALGORITHMES SUR LES LLCs

SUPPRESSION DANS UNE LLC

❖ **Cas 3: suppression à une position donné « pos ».**



Procédure Supprimer_pos (Var L:*maillon, pos:entier)

Début

Si (pos = 1) alors //supprimer le premier maillon

Supprimer_Deb(L)

Sinon

P ← L

Q ← L

Pour i ← 1 à pos faire

Q ← P

P ← suivant (P)

Aff_adr(Q, suivant (P))

libérer (P)

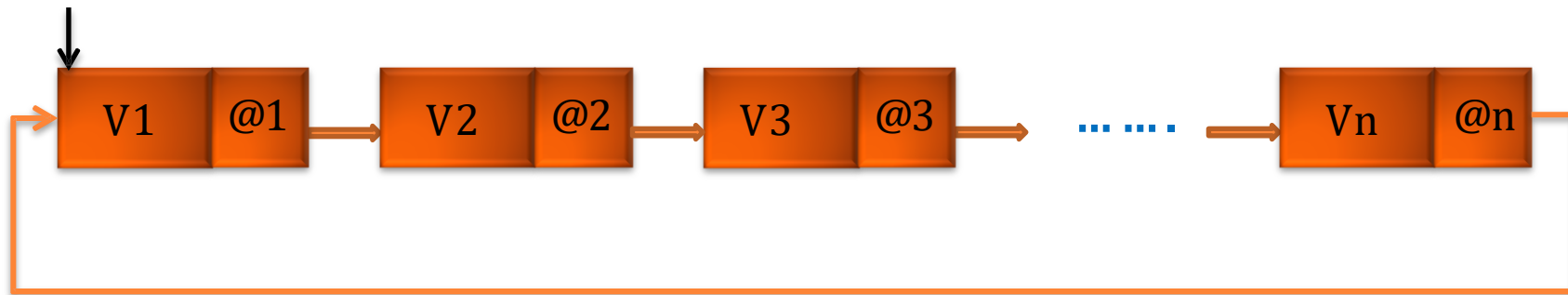
Fin

LISTES PARTICULIÈRES

LISTE CIRCULAIRE

- ❖ **Liste circulaire** est une LLC où le dernier maillon pointe le premier, formant ainsi un cercle.

Tête



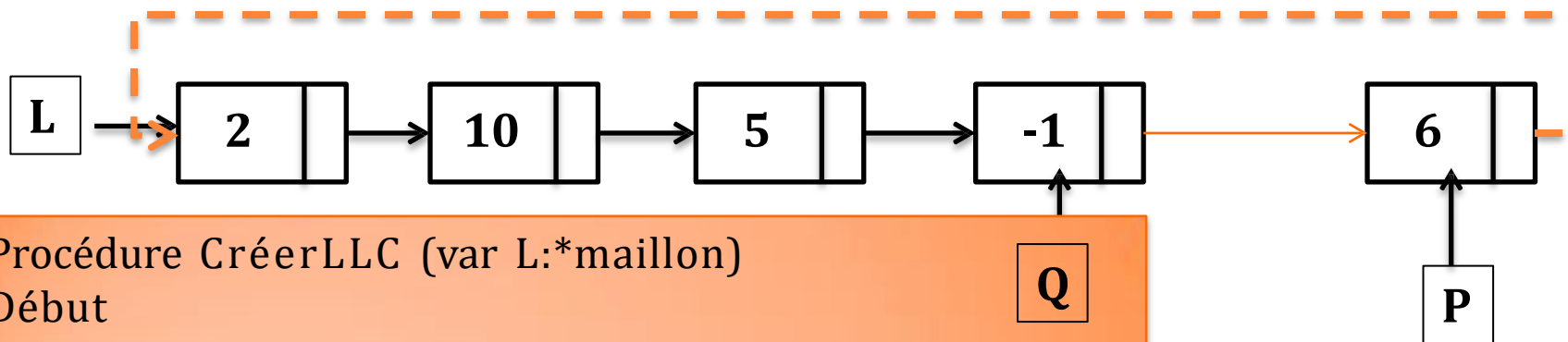
- ❖ La tête de la liste est l'adresse de n'importe quel maillon de la liste circulaire.
- ❖ Le même modèle des LLC est utilisé pour écrire des algorithmes sur ce type de listes: $M_{LLC_c} = M_{LLC}$

LISTES PARTICULIÈRES

LISTE CIRCULAIRE (QUELQUES ALGORITHMES)

❖ Construction d'une LLC circulaire

- Construire une LLC unidirectionnel
- À la fin, chaîner le dernier maillon avec le premier

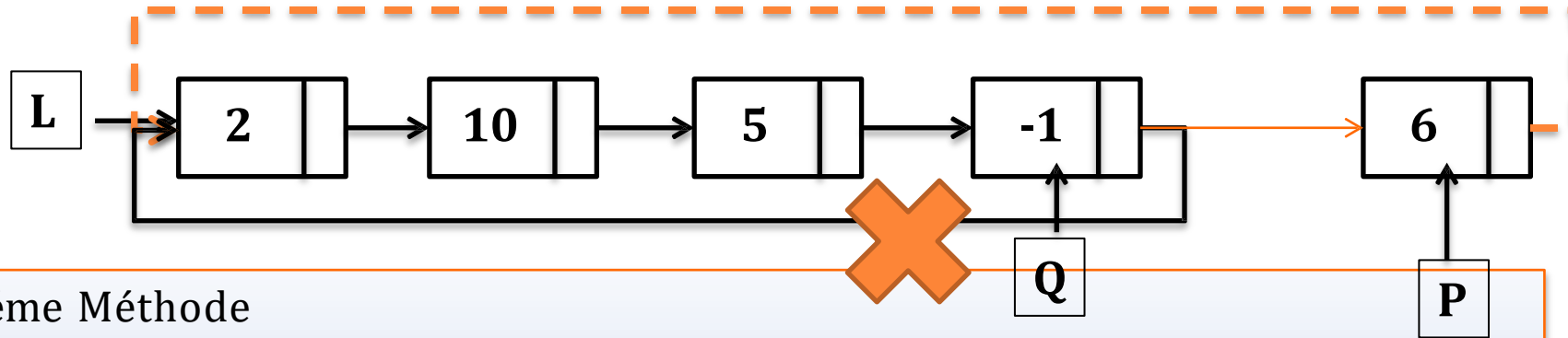


```
Procédure CréerLLC (var L:*maillon)
Début
  Pour i ← 1 à N faire
    lire (x)
    Allouer (P); aff_val(P, x) ; aff_adr(P, null)
    Si (L=null) alors L ← P //mettre à jour l'entête
    Sinon   aff_adr(Q, P)
           Q ← P //sauvegarder l'@ du dernier maillon
  Aff_adr (P, L);
Fin
```


LISTES PARTICULIÈRES

LISTE CIRCULAIRE (QUELQUES ALGORITHMES)

❖ Construction d'une LLC circulaire



2^{ème} Méthode

Procédure CréerLLCcir (var L:*maillon)

Début

Pour $i \leftarrow 1$ à N faire

lire (x); Allouer (P); aff_val(P, x)

Si (L=null) alors //la liste est vide

aff_adr(P, P)

$L \leftarrow P$ //mettre à jour l'entête

Sinon

aff_adr(Q, P)

aff_adr(P, L) // lier le dernier maillon au premier

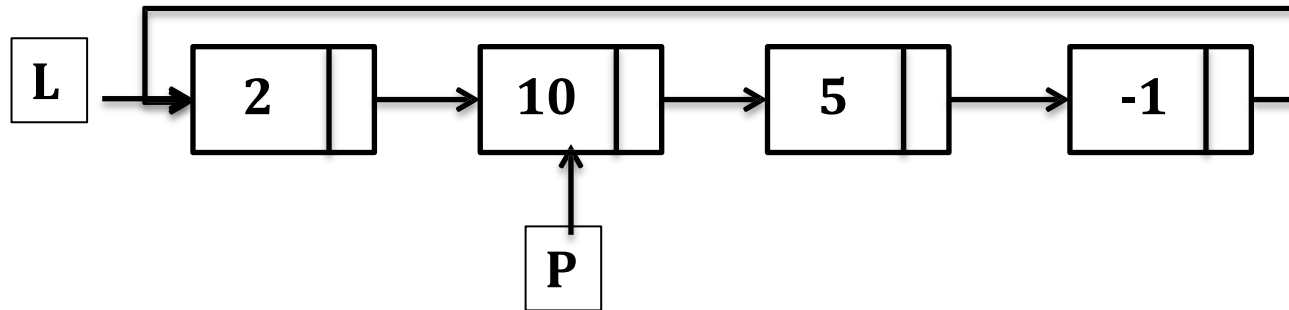
$Q \leftarrow P$ //sauvegarder l'@ du dernier maillon

Fin

LISTES PARTICULIÈRES

LISTE CIRCULAIRE (QUELQUES ALGORITHMES)

❖ Impression d'une LLC circulaire



Procédure Imprimer_cir (var L:*maillon)

Début

Si $L \neq \text{Null}$ alors

$P \leftarrow L$

Répéter

Écrire (valeur (P))

$P \leftarrow \text{suivant}(P)$

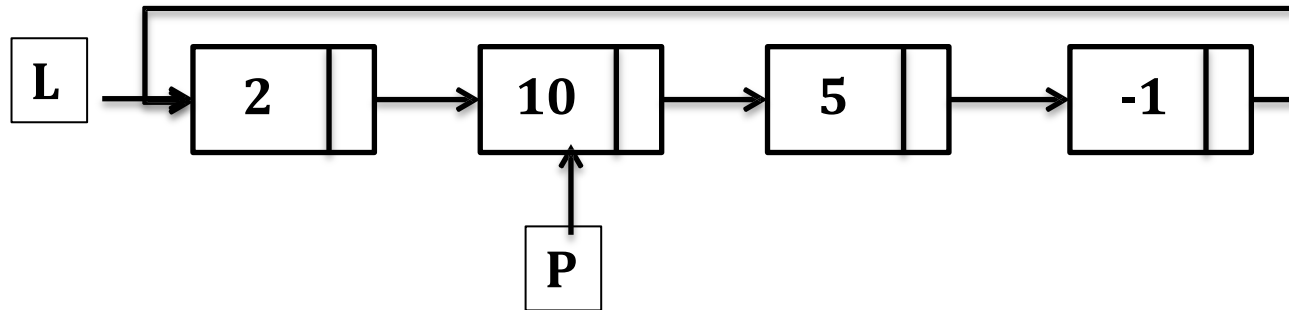
Jusqu'à $P=L$

Fin

LISTES PARTICULIÈRES

LISTE CIRCULAIRE (QUELQUES ALGORITHMES)

❖ Recherche d'une valeur dans une LLC circulaire

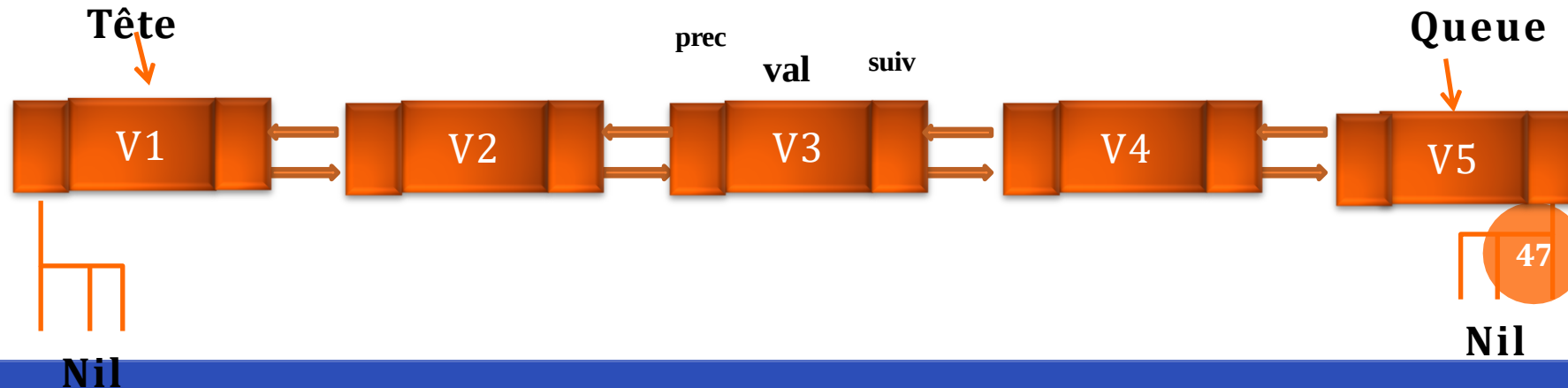


```
Fonction Rechercher_cir (var L:*maillon, v:entier):  
    maillon  
Début  
    Si L ≠ Null alors  
        P ← L  
        Répéter  
            Si (valeur (P) = v) alors  
                retourner (P)  
            P ← suivant(P)  
        Jusqu'à P=L  
    Retourner (null)  
Fin
```

LISTES PARTICULIÈRES

LISTE BIDIRECTIONNELLE

- ❖ **Liste bidirectionnelle ou Liste Doublement Chaînées** est une liste que l'on peut parcourir dans les deux sens : de gauche à droite (pour aller au maillon suivant) et de droite à gauche (pour aller au maillon précédent).
- ❖ Elle est définie par sa tête, aussi que par sa queue.



LISTES PARTICULIÈRES

LISTE BIDIRECTIONNELLE

- ❖ Chaque maillon d'une liste bidirectionnelle comporte trois champs:
 - Un champ contenant la donnée.
 - Un pointeur vers l'élément suivant de la liste.
 - Un pointeur vers l'élément précédent de la liste.



```
type Maillon = Structure  
    val : typeqq  
    suiv: * Maillon  
    prec:*Maillon  
fin
```

48

```
Var Tête, queue: *Maillon
```

LISTES PARTICULIÈRES

LISTE BIDIRECTIONNELLE



- ❖ Le modèle des LLC bidirectionnelles est donc étendu par les opérations suivantes :

$$M_{LLCb} = M_{LLC} - \{Aff_Adr\} + \{Aff_prec, Aff_suiv, Précédent\}$$

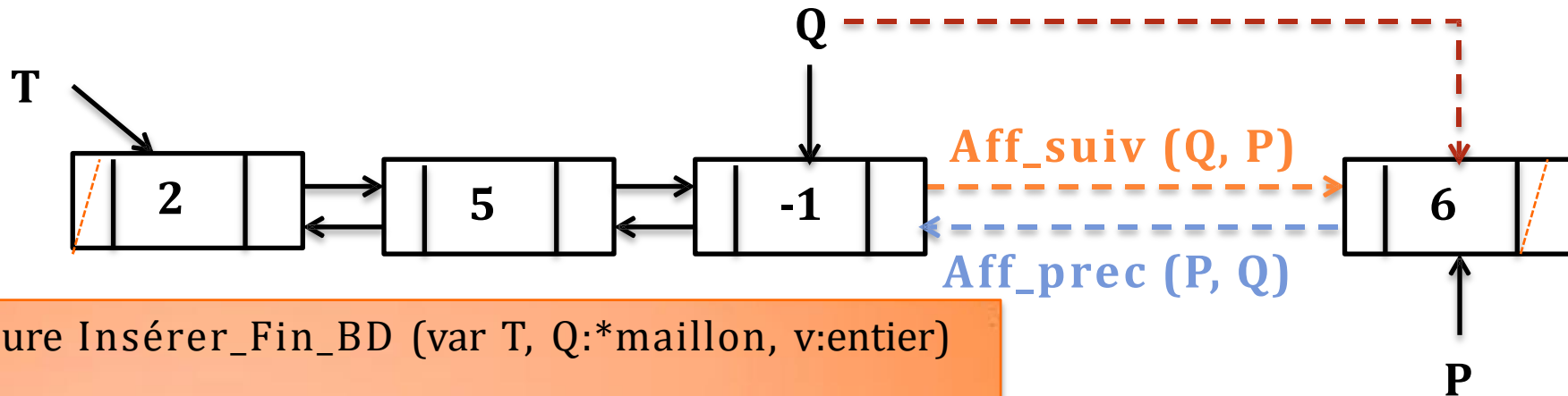
Fonction	Rôle
Précédent(P)	Accéder au champ « prec » du maillon d'adresse P.
Aff_prec(P, Q)	Modifier le champ « prec » du maillon d'adresse P en lui affectant le pointeur Q.
Aff_suiv(P, Q)	Modifier le champ « suiv » du maillon d'adresse P en lui affectant le pointeur Q.

LISTES PARTICULIÈRES

LISTE BIDIRECTIONNELLE (QUELQUES ALGORITHMES)

❖ Construction d'une LLC Bidirectionnelle

- **Cas 1: Insertion à la fin.** Soient $N = 4$ et les valeurs suivantes: 2, 5, -1, 6



Procédure Insérer_Fin_BD (var T , Q :*maillon, v :entier)

Début

Allouer (P); $\text{aff_val}(P, x)$;

Si $Q = \text{Null}$ //la liste est vide

$\text{aff_suiv}(P, \text{null})$; $\text{aff_prec}(P, \text{null})$

$T \leftarrow P$; $Q \leftarrow P$

Sinon

$\text{aff_suiv}(P, \text{null})$

$\text{aff_suiv}(Q, P)$; $\text{aff_prec}(P, Q)$

$Q \leftarrow P$ //mettre à jour la queue

Fin

Procédure CréerLLC_BDQ(var T , Q :*mail

Début

Pour $i \leftarrow 1$ à N faire

$\text{lire}(x)$;

$\text{Insérer_Fin_BD}(T, Q, x)$;

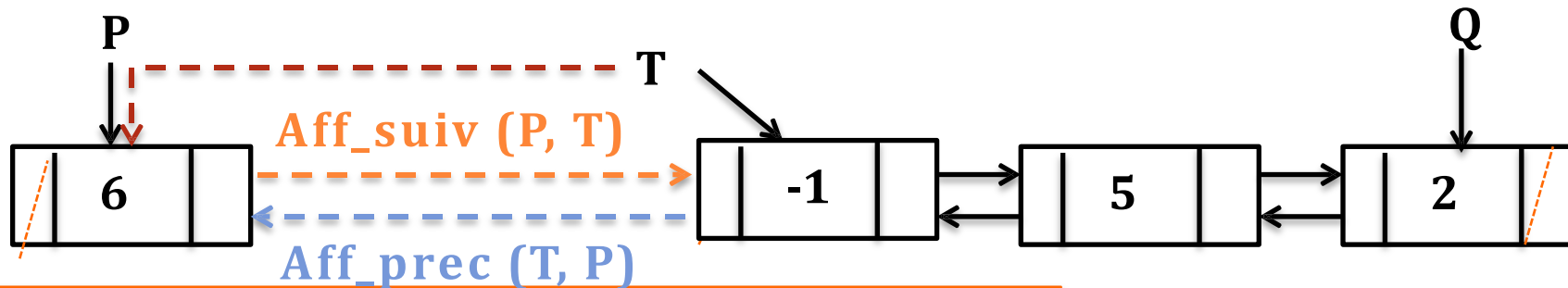
Fin

LISTES PARTICULIÈRES

LISTE BIDIRECTIONNELLE (QUELQUES ALGORITHMES)

❖ Construction d'une LLC Bidirectionnelle

- Cas 2: Insertion au début. Soient $N = 4$ et les valeurs suivantes: 2, 5, -1, 6



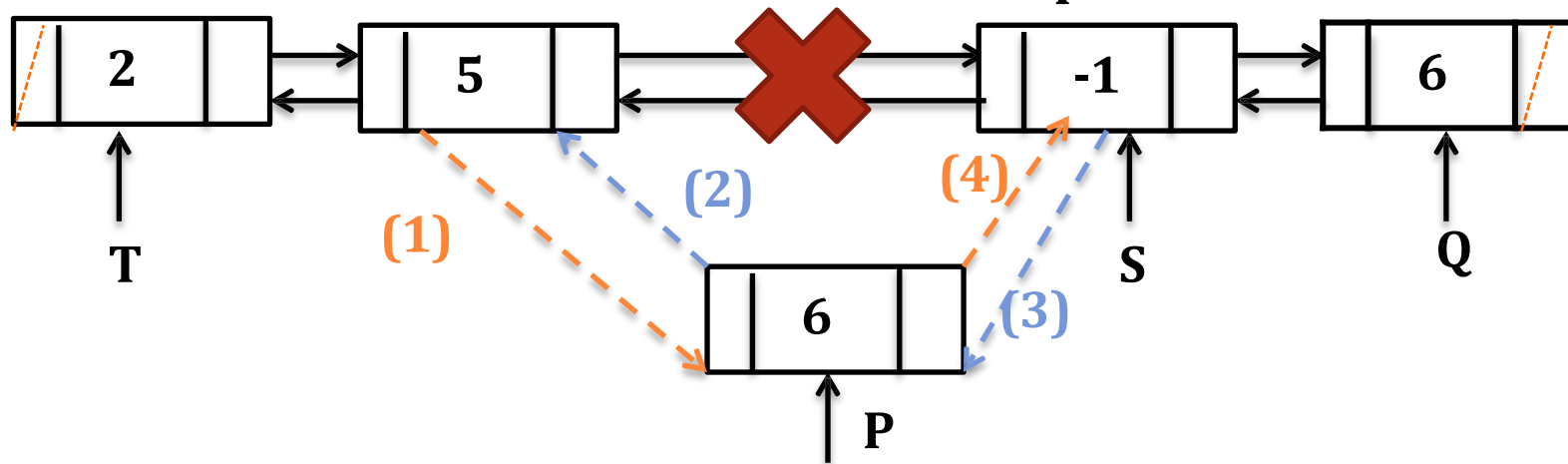
```
Procédure Insérer_Deb_BD (var T, Q:*maillon, v:entier)
Début
Allouer (P); aff_val(P, v);
Si L = Null alors //si la liste est vide
    aff_suiv(P, null); aff_prec(P, null)
    T ← P; Q ← P
Sinon
    aff_prec (P, null);
    aff_suiv (P, T) ; aff_prec (T, P)
    T ← P //mettre à jour la tête
Fin
```

```
Procédure CréerLLC_BD T(var T, Q:*mail
Début
Pour i ← 1 à N faire
    lire (x);
    Insérer_Deb_BD(T, Q, x);
Fin
```


LISTES PARTICULIÈRES

LISTE BIDIRECTIONNELLE (QUELQUES ALGORITHMES)

❖ Insertion à une position donnée (avant)

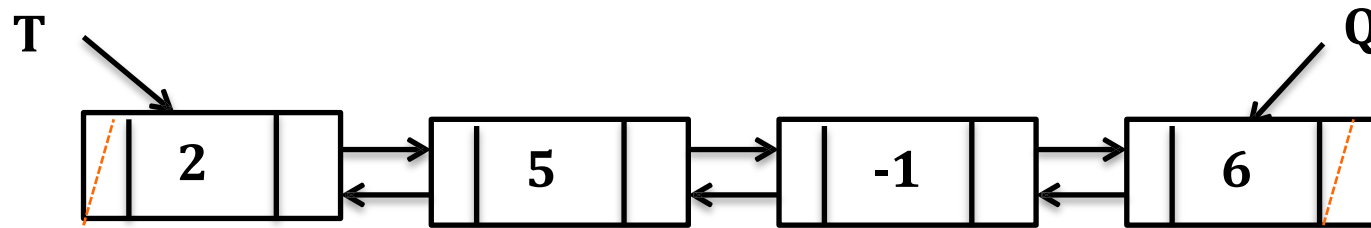


```
Procédure Insérer_pos_BD (var T, Q:*maillon, v:entier)
Si pos = 1 alors Insérer_Deb_BD(T, Q, v)
Sinon
    S ← T;
    Pour i ← 1 à pos faire S ←suivant (S); //accéder au maillon « pos »
    Allouer (P); aff_val(P, v);
    aff_suiv (Precedent (S), P) ; aff_prec (P, Precedent (S));
    aff_suiv (P, S) ;aff_prec (S, P) ;
```

LISTES PARTICULIÈRES

LISTE BIDIRECTIONNELLE (QUELQUES ALGORITHMES)

❖ Impression d'une LLC Bidirectionnelle



Procédure Imprimer_LLC_BDT (T:*maillon)

Début

Tant que (T≠Null) faire //la liste n'est pas vide
 écrire (valeur (T))
 T ←suivant (T)

Fin

La liste imprimée est 2, 5, -1, 6

Procédure Imprimer_LLC_BDQ (Q:*maillon)

Début

Tant que (Q≠Null) faire //la liste n'est pas vide
 écrire (valeur (Q))
 Q ←précédent (Q)

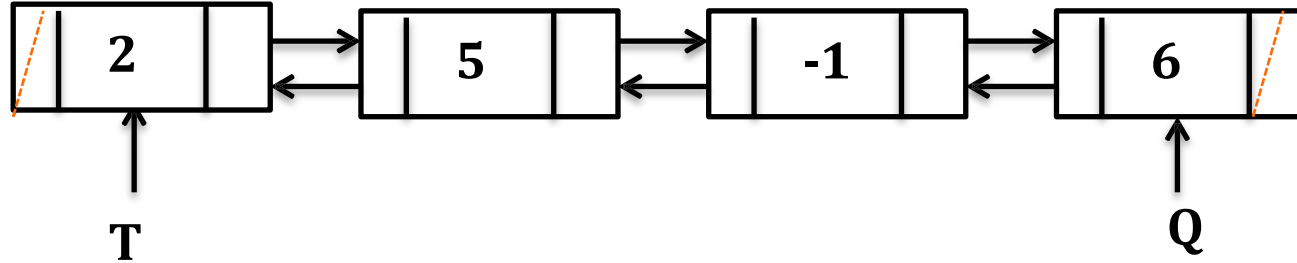
Fin

La liste imprimée est 6, -1, 5, 2

LISTES PARTICULIÈRES

LISTE BIDIRECTIONNELLE (QUELQUES ALGORITHMES)

❖ Recherche d'une valeur dans une LLC Bidirectionnelle



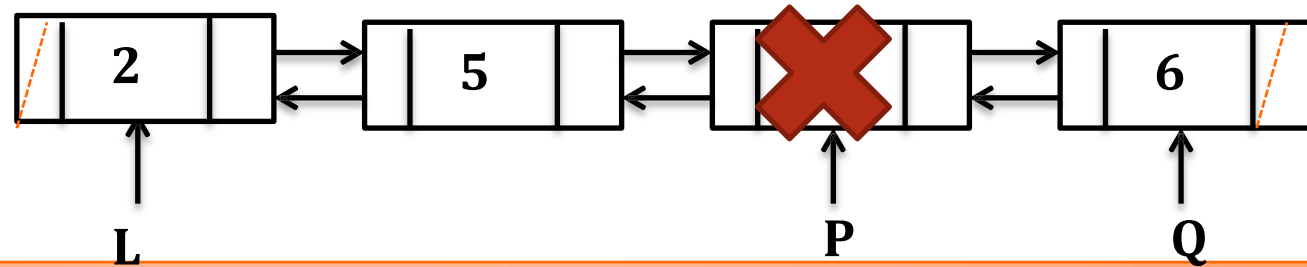
```
Fonction Rech_BD T(T:*maillon, v:entier): *maillon
Début
P ← T
Tant que (P≠Null) faire //la liste n'est pas vide
    Si valeur (P) = v alors retourner (P)
    Sinon    P ← suivant (P)
Retourner (Null)
Fin
```

```
Fonction Rech_BDQ (Q:*maillon, v:entier): *maillon
Début
P ← Q
Tant que (P≠Null) faire //la liste n'est pas vide
    Si valeur (P) = v alors retourner (P)
    Sinon    P ← précédent (P)
Retourner (Null)
Fin
```

LISTES PARTICULIÈRES

LISTE BIDIRECTIONNELLE (QUELQUES ALGORITHMES)

❖ Suppression du maillon contenant la valeur v



Procédure Supprimer_BD (Var T, Q:*maillon, v:entier)

Si (L ≠ Null) alors **//la liste n'est pas vide**

 P ← Rech_BDT (T, v)

 Si P ≠ Null alors **//il existe un maillon contenant la valeur v**

 Si P = T = Q alors **//la liste contient un seul maillon**

 T ← Null; Q ← Null **//la liste devient vide**

 Sinon Si P = T alors **//suppression au début**

 T ← suivant (P); aff_prec(T, null);

 Sinon Si P = Q alors **//suppression à la fin**

 Q ← précédent (P); aff_suiv(Q, null)

 Sinon

 aff_suiv(précédent (P), suivant(P));

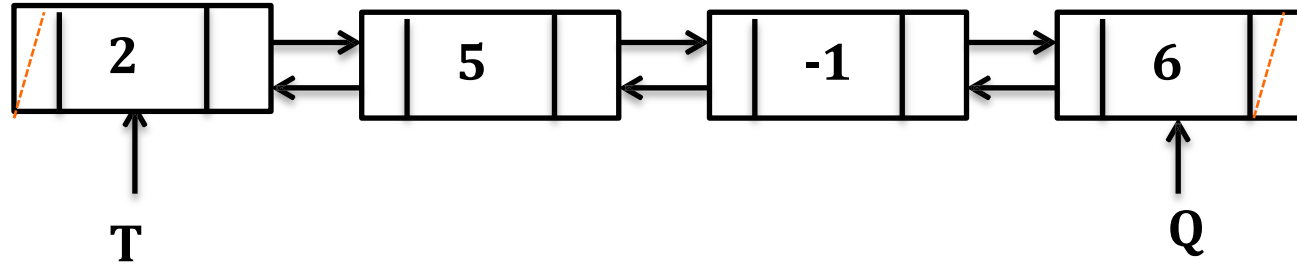
 aff_prec(suivant(P), précédent (P));

libérer (P)

LISTES PARTICULIÈRES

LISTE BIDIRECTIONNELLE (QUELQUES ALGORITHMES)

❖ Destruction d'une LLC Bidirectionnelle



```
Procédure Détruire_BD (Var T:*maillon)
Début
  Tant que (T≠Null) faire //la liste n'est pas vide
    P←T
    T ←suivant (T)
    libérer (P)
Fin
```

```
Procédure Détruire_BD (Var Q:*maillon)
Début
  Tant que (Q≠Null) faire //la liste n'est pas vide
    P←Q
    Q ←précédent (Q)
    libérer (P)
Fin
```

Chapitre 7: Les Entrées-Sorties

7. Les Entrées-Sorties: Gestions des fichiers

- Durant l'exécution d'un programme, le processeur a besoin de **communiquer** avec le reste du **matériel**. Ces échanges d'informations sont les **entrées** et les **sorties** (ou **input** et **output**), souvent abrégées **E/S**(ou **I/O**).
- Les **entrées** permettent de **recevoir** une donnée en provenance de certains périphériques. Les données fournies par ces entrées peuvent être une information envoyée par le disque dur, la carte réseau, le clavier, la souris, un CD, un écran tactile, bref par n'importe quel périphérique. Par exemple, notre clavier va transmettre des informations sur les touches enfoncées au processeur : notre clavier est donc une entrée.
- À l'inverse, les **sorties** vont **transmettre** des données vers ces périphériques. On pourrait citer l'exemple de l'écran : notre ordinateur lui envoie des informations pour qu'elles soient affichées.

7. Les Entrées-Sorties: Gestions des fichiers

- Langage C offre la possibilité de **lire** et d'**écrire** des données dans un **fichier**.
- Pour des raisons d'efficacité, les **accès** à un **fichier** se font par l'intermédiaire d'une **mémoire-tampon (buffer)**, ce qui permet de réduire le nombre d'accès aux périphériques (disque, ram, cpu, etc).
- Pour pouvoir manipuler un **fichier**, un programme a besoin d'un certain nombre d'informations : l'**adresse** de l'endroit de la **mémoire-tampon** où se trouve le fichier, la **position** de la tête de lecture, le **mode d'accès** au fichier (lecture ou écriture).

*Ces informations sont rassemblées dans une structure dont le type, **FILE***, est défini dans **stdio.h**.*

*Un objet de type **FILE*** est appelé flot de données (en anglais, **stream**).*

7.1 Ouverture et Fermeture de fichiers

7.1.1 Fonction `fopen`:

- Cette fonction, de type **FILE*** ouvre un **fichier** et lui associe un flot de données. Sa syntaxe est :

`fopen("nom-de-fichier", "mode-accès");`

- La valeur retournée par `fopen` est un **flot de données**.
- Si l'exécution de cette fonction ne se déroule pas normalement, la valeur retournée est le **pointeur NULL**.
- Il est donc recommandé de toujours tester si la valeur renvoyée par la fonction `fopen` est égale à **NULL** afin de détecter les erreurs (lecture d'un fichier inexistant).

7.1 Lecture et écriture de fichiers

7.1.1 Fonction `fopen`:

- Le premier argument, *nom-de-fichier*, de `fopen` est le **nom du fichier concerné**, fourni sous forme d'une chaîne de caractères. On préférera définir le nom du fichier par une constante symbolique au moyen de la directive *#define* plutôt que d'expliciter le nom de fichier dans le corps du programme.
- Le second argument, *mode-accès*, est une chaîne de caractères qui spécifie le **mode d'accès au fichier**. Les spécificateurs de mode d'accès diffèrent suivant le type de fichier considéré.

7.1 Lecture et écriture de fichiers

7.1.1 Fonction `fopen`:

On distingue :

- **les fichiers textes**, pour lesquels les **caractères de contrôle** (retour à la ligne) seront interprétés en tant que tels lors de la lecture et de l'écriture ;
- **les fichiers binaires**, pour lesquels les **caractères de contrôle** ne sont pas interprétés.

7.1 Lecture et écriture de fichiers

7.1.1 Fonction **fopen**: Les différents modes d'accès sont les suivants :

"r"	ouverture d'un fichier texte en lecture
"w"	ouverture d'un fichier texte en écriture
"a"	ouverture d'un fichier texte en écriture à la fin
"rb"	ouverture d'un fichier binaire en lecture
"wb"	ouverture d'un fichier binaire en écriture
"ab"	ouverture d'un fichier binaire en écriture à la fin
"r+"	ouverture d'un fichier texte en lecture/écriture
"w+"	ouverture d'un fichier texte en lecture/écriture
"a+"	ouverture d'un fichier texte en lecture/écriture à la fin
"r+b"	ouverture d'un fichier binaire en lecture/écriture
"w+b"	ouverture d'un fichier binaire en lecture/écriture
"a+b"	ouverture d'un fichier binaire en lecture/écriture à la fin

7.1 Lecture et écriture de fichiers

7.1.1 Fonction `fopen`:

Ces **modes d'accès** ont pour particularités :

- Si le **mode** contient la lettre **r**, le fichier doit **exister**.
- Si le **mode** contient la lettre **w**, le fichier **peut ne pas exister**. Dans ce cas, il sera **créé**. Si le fichier existe déjà, son ancien contenu sera **perdu**.
- Si le mode contient la lettre **a**, le fichier **peut ne pas exister**. Dans ce cas, il sera **créé**. Si le fichier existe déjà, les nouvelles données seront **ajoutées** à la **fin** du fichier précédent.

7.1 Lecture et écriture de fichiers

7.1.2 Fonction **fclose**:

Elle permet de **fermer** le **flot** qui a été associé à un **fichier** par la fonction **fopen**. Sa syntaxe est :

fclose (flot) ;

où **flot** est le flot de type **FILE*** retourné par la fonction **fopen** correspondant.

La fonction **fclose** retourne un **entier** qui vaut **zéro** si l'opération s'est déroulée normalement (et une valeur **non nulle** en cas **d'erreur**).

7.2 Les entrées-sorties formatées

7.2.1 Fonction `fprintf`:

La fonction `fprintf`, analogue à `printf`, permet d'écrire des données dans un fichier. Sa syntaxe est :

`fprintf(flot, "chaîne de contrôle", expression-1, ..., expression-n);`

où **flot** est le flot de données retourné par la fonction `fopen`.

Les spécifications de **format** utilisées pour la fonction `fprintf` sont les mêmes que pour `printf`.

7.2 Les entrées-sorties formatées

7.2.2 Fonction `fscanf`:

La fonction `fscanf`, analogue à `scanf`, permet de **lire** des données dans un **fichier**. Sa syntaxe est semblable à celle de `scanf` :

`fscanf(flot, "chaîne de contrôle", argument-1, ..., argument-n);`

où `flot` est le flot de données retourné par `fopen`.

Les spécifications de **format** sont ici les mêmes que celles de la fonction `scanf`.

7.3 Les entrées-sorties binaires

- Les fonctions **d'entrées-sorties binaires** permettent de transférer des données dans un fichier **sans transcodage**. Elles sont donc **plus efficaces** que les fonctions d'**entrée-sortie standard**, mais les fichiers produits ne sont pas portables puisque le codage des données dépend des machines.
- Elles sont notamment utiles pour manipuler des données de grande taille ou ayant un type composé.
- Leurs prototypes sont :

```
fread(void *pointeur, size_t taille, size_t nombre, FILE *flot);  
fwrite(void *pointeur, size_t taille, size_t nombre, FILE *flot);
```

où **pointeur** est l'adresse du début des données à transférer, **taille** la taille des objets à transférer, **nombre** leur nombre.

- La fonction **fread** lit les données sur le flot **flot** et la fonction **fwrite** les écrit.

7.3 Les entrées-sorties binaires

Exemple:

Le programme ci-après écrit un tableau d'entiers (contenant les 50 premiers entiers) avec *fwrite* dans le fichier sortie, puis lit ce fichier avec *fread* et affiche les éléments du tableau.

```
#include <stdio.h>
#include <stdlib.h>

#define NB 50
#define F_SORTIE "sortie"

void main(void)
{
    FILE *f_in, *f_out;
    int *tab1, *tab2;
    int i;
    tab1 = (int*)malloc(NB * sizeof(int));
    tab2 = (int*)malloc(NB * sizeof(int));
    for (i = 0 ; i < NB; i++)
        tab1[i] = i;

    /* ecriture du tableau dans F_SORTIE */
    f_out = fopen(F_SORTIE, "w")
    fprintf(stderr, "\nImpossible d'ecrire dans le fichier %s\n", F_SORTIE);
    return(EXIT_FAILURE);
    fwrite(tab1, NB * sizeof(int), 1, f_out);
    fclose(f_out);

    /* lecture dans F_SORTIE */
    f_in = fopen(F_SORTIE, "r")
    fread(tab2, NB * sizeof(int), 1, f_in);
    fclose(f_in);
    for (i = 0 ; i < NB; i++)
        printf("%d\t", tab2[i]);
    printf("\n");
}
```

7.4 Gestion des erreurs d'entrée/sortie

En C, les **erreurs d'entrée/sortie** surviennent lorsqu'une opération sur un fichier **échoue**.

Ces **erreurs** peuvent être détectées à l'aide des mécanismes suivants :

- Vérification des valeurs de retour des fonctions.
- Utilisation de **errno**, une variable globale définie dans **<errno.h>**.
- Utilisation de la fonction **perror()** pour afficher un message d'erreur détaillé.

7.4 Gestion des erreurs d'entrée/sortie

7.4.1 Vérification des valeurs de retour:

La plupart des fonctions de manipulation de fichiers (comme **fopen**, **fread**, **fwrite**, **fclose**) retournent une valeur indiquant si l'opération a réussi ou échoué.

7.4 Gestion des erreurs d'entrée/sortie

7.4.1 Vérification des valeurs de retour:

Exemple : Ouverture de fichier avec vérification d'erreur.

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    FILE *file = fopen("exemple.txt", "r");

    // Vérification d'erreur
    if (file == NULL) {
        fprintf(stderr, "Erreur : Impossible d'ouvrir le fichier.\n");
        return EXIT_FAILURE;
    }

    printf("Fichier ouvert avec succès.\n");
    fclose(file);
    return EXIT_SUCCESS;
}
```

7.4 Gestion des erreurs d'entrée/sortie

7.4.2 Utilisation de `errno` et `perror`:

Lorsque certaines fonctions échouent, elles définissent une valeur dans la variable globale `errno`. Vous pouvez utiliser `perror()` ou `strerror()` pour afficher le message correspondant.

Exemple : Ouverture de fichier avec vérification d'erreur.

7.4 Gestion des erreurs d'entrée/sortie

7.4.2 Utilisation de `errno` et `perror`:

Exemple : Ouverture de fichier avec vérification d'erreur.

```
#include <stdio.h>
#include <errno.h>
#include <string.h>

int main() {
    FILE *file = fopen("inexistant.txt", "r");

    if (file == NULL) {
        perror("Erreur détectée");
        printf("Code d'erreur : %d\n", errno);
        printf("Description : %s\n", strerror(errno));
        return errno;
    }

    fclose(file);
    return 0;
}
```

7.4 Gestion des erreurs d'entrée/sortie

7.4.3 Gestion des erreurs lors de la lecture/écriture:

Lorsque vous **lisez** ou **écrivez** dans un fichier, il est important de vérifier si l'opération a été effectuée correctement.

Exemple: Lecture avec gestion

```
#include <stdio.h>
#include <stdlib.h>
int main() {

    FILE *file = fopen("exemple.txt", "r");
    if (file == NULL) {
        perror("Erreur lors de l'ouverture du fichier");
        return EXIT_FAILURE;
    }

    char buffer[256];
    if (fgets(buffer, sizeof(buffer), file) == NULL) {
        if (feof(file)) {
            printf("Fin du fichier atteinte.\n");
        } else {
            perror("Erreur lors de la lecture");
        }
        fclose(file);
        return EXIT_FAILURE;
    }

    printf("Contenu lu : %s\n", buffer);
    fclose(file);

    return EXIT_SUCCESS;
}
```


7.4 Gestion des erreurs d'entrée/sortie

7.4.4 Utilisation de `ferror` et `clearerr`:

La fonction `ferror()` permet de vérifier si une erreur s'est produite lors d'une opération sur un fichier. La fonction `clearerr()` permet de réinitialiser les indicateurs d'erreur.

Exemple: Lecture avec gestion

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    FILE *file = fopen("exemple.txt", "r");
    if (file == NULL) {
        perror("Erreur lors de l'ouverture du fichier");
        return 1;
    }

    char ch;
    while ((ch = fgetc(file)) != EOF) {
        putchar(ch);
    }

    if (ferror(file)) {
        perror("Erreur lors de la lecture");
    }

    fclose(file);
    return 0;
}
```

7.4 Gestion des erreurs d'entrée/sortie

7.4.5 Erreurs fréquentes et leur gestion:

- ❖ **Erreur : Accès refusé:** Cela peut se produire si le fichier n'a pas les permissions nécessaires.

Solution :

```
FILE *file = fopen("fichier_protege.txt", "w");  
if (file == NULL) {  
    perror("Accès refusé");  
    return 1;  
}
```

- ❖ **Erreur : Espace disque insuffisant:** Cela peut se produire si le fichier n'a pas les permissions nécessaires.

Solution : Vérifier la valeur retournée par fwrite.

FIN DU COURS DE LANGAGE C
Semestre 2 pour étudiants L1 MPI & SML
UFR STA – UAM

BONNE CHANCE POUR LA SUITE
A bientôt pour la L2

Pr AD Gueye
Dr L Yade
M. S Anne
M. PA Diop